Empirical Software Engineering
(submitted manuscript)

Guidelines for Empirical Studies in Software Engineering involving Large Language Models

Sebastian Baltes · Florian Angermeir · Chetan Arora · Marvin Muñoz Barón · Chunyang Chen · Lukas Böhme · Fabio Calefato · Neil Ernst · Davide Falessi · Brian Fitzgerald · Davide Fucci · Junda He · Christoph Treude · Marcos Kalinowski · Stefano Lambiase · Daniel Russo · Mircea Lungu · Cristina Martinez Montes · Lutz Prechelt · Paul Ralph · ~~Christoph Treude~~ Rijnard van Tonder · Stefan Wagner

Submitted: 26/08/2025
Revised: 27/02/2026

Sebastian Baltes
~~University of Bayreuth~~Heidelberg University, Germany
E-mail: sebastian.baltes@uni-heidelberg.de
(corresponding author)

Florian Angermeir
fortiss, Germany
Blekinge Institute of Technology, Sweden
E-mail: angermeir@fortiss.org

Chetan Arora
Monash University, Australia
E-mail: chetan.arora@monash.edu

Marvin Muñoz Barón · Chunyang Chen
Technical University of Munich, Germany
E-mail: {marvin.munoz-baron, chun-yang.chen}@tum.de

Lukas Böhme
Hasso-Plattner-Institut, Germany
University of Potsdam, Germany
E-mail: lukas.boehme@hpi.de

Fabio Calefato
University of Bari, Italy
E-mail: fabio.calefato@uniba.it

Neil Ernst
University of Victoria, Canada
E-mail: nernst@uvic.ca

Davide Falessi

Abstract Large ~~language models~~ Language Models (LLMs) are ~~increasingly being integrated into~~ now ubiquitous in software engineering (SE) research and practice, yet their non-determinism, opaque training data, and ~~evolving architectures complicate the reproduction and replication~~ rapidly evolving models threaten the reproducibility and replicability of empirical studies. We ~~present a community effort to scope this space, introducing~~ address this challenge through a collaborative effort of 22 researchers, presenting a taxonomy of ~~LLM-based study types~~ seven study types that organizes the landscape of LLM involvement in SE research, together with eight guidelines for designing and reporting ~~empirical studiesinvolving LLMs. The guidelines present essential~~ such studies. Each guideline distinguishes requirements (MUST) ~~criteria as~~

University of Rome Tor Vergata, Italy
E-mail: falessi@ing.uniroma2.it

Brian Fitzgerald
Lero, Ireland
University of Limerick, Ireland
E-mail: brian.fitzgerald@ul.ie

Davide Fucci
Blekinge Institute of Technology, Sweden
E-mail: davide.fucci@bth.se

Junda He · Christoph Treude
Singapore Management University, Singapore
E-mail: {jundahe, ctreude}@smu.edu.sg

Marcos Kalinowski
PUC Rio de Janeiro, Brazil
E-mail: kalinowski@inf.puc-rio.br

Stefano Lambiase · Daniel Russo
Aalborg University in Copenhagen, Denmark
E-mail: {stla, daniel.russo}@cs.aau.dk

Mircea Lungu
IT University of Copenhagen, Denmark
E-mail: mlun@itu.dk

Cristina Martinez Montes
Chalmers University of Technology, Sweden
University of Gothenburg, Sweden
E-mail: montesc@chalmers.se

Lutz Prechelt
Freie Universität Berlin, Germany
E-mail: prechelt@inf.fu-berlin.de

Paul Ralph
Dalhousie University, Canada
E-mail: paulralph@dal.ca

~~Christoph Treude~~ Rijnard van Tonder
~~Singapore Management University, Singapore~~Independent Researcher, Antigua and Barbuda
E-mail: rvantonder@gmail.com

Stefan Wagner
Technical University of Munich, Germany
E-mail: stefan.wagner@tum.de

~~well as desired~~ from recommended practices (SHOULD) ~~criteria and target transparency throughout the research process. Our recommendations, contextualized by our study types , are~~and is contextualized by the study types it applies to. Our guidelines recommend that researchers: (1)~~to~~ declare LLM usage and role; (2)~~to~~ report model versions, configurations, and ~~fine-tuning~~customizations; (3)~~to document tool architectures~~ document the tool architecture beyond the model; (4)~~to disclose prompts~~ disclose prompts, their development, and interaction logs; (5)~~to use human validation~~ validate LLM outputs with humans; (6)~~to employ~~ include an open LLM as a baseline; (7)~~to~~ use suitable baselines, benchmarks, and metrics; and (8)~~to openly~~ articulate limitations and mitigations. ~~Our goal is to enable reproducibility and replicability despite LLM-specific barriers to open science~~We complement the guidelines with an applicability matrix mapping guidelines to study types and a reporting checklist for authors and reviewers. We maintain the study types and guidelines online as a living resource for the community to use and shape (llm-guidelines.org).

Keywords software engineering, large language models, empirical research, meta-science, guidelines

1 Introduction

~~In the short period since the~~ Since the release of ChatGPT in November 2022, large language models (LLMs) have ~~changed the~~ rapidly become a major focus of software engineering (SE) research~~landscape. Although there are numerous opportunities to use LLMs to support SE research and development tasks, solid science needs rigorous empirical evaluations to explore the effectiveness, performance, and robustness of using LLMs to automate different research and development tasks. LLMs can, for example, be used to support literature reviews, reduce development time, and generate software documentation. However, it is often unclear how valid, reproducible, and replicable~~ [63], yet the reproducibility and replicability of empirical studies involving ~~LLM are. This uncertainty poses significant challenges for researchers and practitioners who seek to draw reliable conclusions from empirical studies.~~

~~The importance of open science practices and documenting the study setup is not to be underestimated [56]. One of the primary risks in creating irreproducible and irreplicable results based on studies involving LLMs stems from the variability in model performance due to~~ LLMs remains uncertain. Recent findings indicate a low reproducibility of SE studies involving LLMs [8]. Three characteristics of LLMs make this particularly challenging: their inherent non-determinism ~~, but also due to differences in configuration, training data, model architecture, or evaluation metrics. Slight changes can lead~~ causes variability across runs, with slight changes leading to significantly different results~~. The lack of standardized benchmarks and evaluation protocols further hinders the reproducibility and replicability of study results. Without detailed information on the exact study setup, benchmarks, and metrics used, it is challenging for other researchers to replicate results using different modelsand tools. These issues highlight~~

~~the need for clear guidelines and best practices for designing and reporting studies involving LLMs. While~~ [2, 21, 139, 147]; commercial models evolve beyond version identifiers, so reported performance can change over time [29] ; and even for "open" models, training data and fine-tuning details often remain undisclosed [51]. Moreover, prompt formatting choices alone can shift accuracy by tens of percentage points [142], and configured parameters such as temperature affect output variability [128]. Hence, not reporting these settings directly affects reproducibility.

Although the SE research community has developed guidelines for conducting and reporting specific types of empirical studies such as controlled experiments (e.g.,~~*Experimentation in Software Engineering* [168], *Guide to Advanced Empirical Software Engineering* [143])or~~ [143, 168]), their replications (e.g.,~~*A Procedure and Guidelines for Analyzing Groups of Software Engineering Replications*~~ [134]), ~~we believe that LLMs have specific intrinsic characteristics that require specific guidelines for researchers to achieve an acceptable level of reproducibility and replicability (see also our previous position paper [159]). For example, even if we knew the specific version of a commercial LLM used for an empirical study, the reported task performance could still change over time, since commercial models are known to evolve beyond version identifiers [29]. Moreover, commercial providers do not guarantee the availability of old model or tool versions indefinitely. In addition to version differences, LLM performance varies widely depending on configured parameters such as temperature. Therefore, not reporting the parameter settings severely impacts reproducibility. Even for "open" models such as *Mistral, DeepSeek or Llama*, we do not know how they were fine-tuned for specific tasks and what the exact training data was [51]. A general problem when evaluating LLMs ' performance is that we do not know whether the solution to a certain problem was part of the training data or not~~or empirical studies in general (e.g., the *ACM SIGSOFT Empirical Standards* [123]), none of these address the LLM-specific aspects described above. Previously, a position paper highlighted these issues [159], but there was no comprehensive community-developed guidance for designing and reporting empirical studies involving LLMs in SE.

~~So far, there are no holistic~~ Therefore, we present community-developed guidelines for conducting and reporting studies involving LLMs in SE research~~. With this community effort, we try to fill this gap~~, co-developed by 22 researchers. After outlining our *Scope*, we ~~continue by introducing~~ introduce a taxonomy of *Study Types* ~~before presenting~~ , then present eight *Guidelines* ~~that the authors of this article co-developed. Also, for~~ . We complement these with an applicability matrix mapping guidelines to study types and a reporting checklist for authors and reviewers. For each study type and guideline, we identify relevant ~~research exemplars~~examples, both within ~~the SE literature, but also outside the discipline where relevant; this serves a useful purpose given the vast proliferation of research on this nascent topic. The most recent version is always available online~~ and outside of SE research. We maintain the

guidelines online as a living resource for the community to use and shape.[1] ~~Other researchers can suggest changes via a public GitHub repository.xw~~[2]

2 ~~Scope~~

2 Scope of Guidelines

~~In this section, we outline the scope of the study types and guidelines we present.~~

~~We target *software engineering* (SE) research conducted in academic or industry contexts. While other disciplines have started developing community guidelines for reporting empirical studies involving LLMs, SE has not seen a holistic effort in that direction. In healthcare, for example, Gallifant et al presented the *TRIPOD-LLM* guidelines [49] , which are centered around a checklist for [49]. Although the checklist overlaps with our~~

Software Engineering as our Target Discipline: We target SE research because existing cross-disciplinary guidelines do not address its needs. For instance, Gallifant et al.'s [49] guidelines for LLM use in healthcare research include discipline-specific items irrelevant to most SE studies (e.g., ~~reporting the LLM name and version, comparing performance between LLMs, humans and other benchmarks) , they are healthcare-specific (e.g., they require to and). Instead of a checklist, we distinguish~~ clinical-care usability) while lacking guidance on tool architectures, benchmarking, and ~~criteria and provide brief *tl;dr* summaries highlighting our most important recommendations. SE research utilizes a broader range~~ code-specific evaluation that SE research requires. Moreover, SE research employs a wide variety of empirical methods~~than many other disciplines. Therefore, we are convinced that SE-specific guidelines need to be developed in combination with a taxonomy such as ours. In SE, Sallou et al have presented a~~ [123, 168]. Our work builds on Sallou et al's vision paper on mitigating validity threats in LLM-based SE research~~. Although their guidelines partially overlap with ours (e.g., repeating experiments to handle output variability, considering data leakage), our guidelines cover a broader range of study types and provide more detailed advise. For example, we contextualize our guidelines by study type, distinguish LLMs and LLM-based tools, talk about models, baselines, benchmarks, and metrics, and explicitly address human validation. Moreover, our guidelines are the result of an extensive coordination process between a large number of experts in empirical SE, hence a first step towards true community guidelines~~ [133] and our own position paper [159] , providing more detailed advice for a wider variety of research methods organized into a taxonomy of *Study Types*(Section 4).

[1] `https://llm-guidelines.org`

[2]

Focus on Text-Based Use Cases: ~~We want to clarify that our focus is on LLMs, that is, natural language use cases. Multi-modal foundational models are beyond the scope of our study types and guidelines. We are aware that these foundational models have great potential to support SE research and practice. However, due to the diversity of artifacts that can be generated or used as input (e.g.,~~ While multi-modal foundation models that use or generate images, audio, ~~and video) and the more demanding hardware requirements, we deliberately focus on LLMs only. However, our guidelines could be extended in the future to include foundational models beyond natural language text.~~ or video may also support SE research and practice, we focus on textual use cases of LLMs (e.g., in natural language or programming languages). Many of our guidelines—particularly those concerning model reporting, prompt documentation, and reproducibility—are likely applicable to multi-modal settings as well, but we leave their explicit validation to future work.

Focus on Direct Development or Research Support: ~~Given the exponential growth in LLM usage across all research domains, we also want to define the research contexts in which our guidelines apply. LLMs are already widely used to support several aspects of the overall research process, from fairly simple tasks such as~~ While researchers may use LLMs for many peripheral tasks (e.g., proof-reading, spell-checking, ~~and text translation~~translation), our guidelines focus on their direct role in empirical research and engineering practice. For engineers, ~~to more complex activities such as data coding and synthesis of literature reviews. Regarding tools,~~ we focus on ~~use cases in the area of AI for software engineering (AI4SE)~~the use of LLMs to automate SE tasks, that is, ~~studying the support and automation of SE tasks with the help of~~ artificial intelligence (AI) ~~, more specifically LLMs (see Section~~for software engineering (AI4SE) (see *LLMs for SE*). ~~For research support, we focus on empirical SE research supported by~~ This includes agentic systems that autonomously plan and execute multi-step tasks using LLMs (see~~Section). By research support, we mean the active involvement of LLMs in~~ *Architecture*). For researchers, we focus on the use of LLMs to automate empirical research tasks such as data collection, processing, or analysis ~~. We consider LLMs supporting the study design or the writing process to be out of scope.~~ (see *LLMs for Research*).

Researchers as our Target Audience: ~~Third, our guidelines mainly target SE researchers planning, designing, conducting, and reporting~~ Our guidelines are intended to help SE researchers design, plan, conduct, and report empirical studies involving LLMs~~. Although researchers who review scientific articles written by others can also use our guidelines , for example, to check whether the authors adhere to the essential requirements we present, reviewers are not our main target audience.~~ , and to support scholarly peer review of such studies. Each guideline includes an *Advice for Reviewers* subsection with targeted assessment suggestions. Our guidelines focus on what to report and how; they complement but do not replace methodological guidance for designing specific types of empirical studies.

How to Navigate this Paper: This paper is structured to support different reading strategies depending on the reader's goal. *Researchers planning a new study* may start with the taxonomy of study types (see *Study Types*) to identify which types apply to their planned work, then consult Table 3 to determine which guidelines are requirements (MUST) and which are recommendations (SHOULD) for those study types. Each guideline section opens with a *tl;dr*summary in a shaded box, allowing readers to quickly assess relevance before reading the full text. *Researchers writing up results* may prefer to start with the checklist in Appendix A, which organizes actionable items by typical paper sections (Introduction, Research Design and Methods, Results, etc.). Table 2 provides a quick reference to all eight guidelines, and Table 4 maps each guideline's rationale to its key recommendations. *Reviewers* can use the *Advice for Reviewers* subsection at the end of each guideline for targeted guidance on assessing manuscripts. Table 3 helps reviewers identify which guidelines apply to the study type under review.

3 Methodology

~~The starting point for the development of these guidelines was~~ This project was initiated at the 2024 meeting of the *International Software Engineering Research Network* (ISERN) in Barcelona, Spain~~. The~~ , where the first and last author ~~of this paper~~ organized a session ~~to discuss the need to develop~~ on guidelines for empirical studies involving LLMs. These discussions ~~resulted in~~ led to a position paper ~~published at the~~ at ~~*International Workshop on Methodological Issues with Empirical Studies in Software Engineering(WSESE*~~ *WSESE 2025*~~*) [159]. The first and last author presented a preprint of that paper*~~ [159], which was presented at the *2nd Copenhagen Symposium on Human-Centered Software Engineering AI* (CHASEAI 2024), forming a workstream to ~~lead the collaborative refinement and extension of~~ collaboratively refine and extend the preliminary guidelines~~presented in the position paper~~. Our process follows an established tradition of developing reporting guidelines through expert collaboration, as exemplified by CONSORT for clinical trials [20] and, more recently, TRIPOD-LLM for LLM-based prediction studies in medicine [49] .

We held bi-weekly meetings, in which we decided on the structure of the study type taxonomy and the guidelines. During these discussions, we added a new study type (*LLMs for Synthesis*), refined ~~the guidelines to and to ,~~ and extended the guideline ~~to , and added the new guidelines to~~ sections (in particular *Version and Configuration*, *Architecture*, *Prompts and Logs*) and covered *Benchmarks and Metrics* ~~and to~~ as well as *Limitations and Mitigations*.

Then, at least two co-authors were assigned to each study type and guideline ~~. They refined and extended the contentto align with the intended structure~~to refine and extend the content, followed by a peer review and refinement phase. Afterwards, the first author reviewed all sections and discussed potential changes

with the topic leads. A requirement for co-authorship was that each author contributed or reviewed content and, most importantly, read the complete guidelines and confirmed that they stand behind all recommendations, not only their own contributions.

To select illustrative examples for each study type and guideline, co-author pairs independently searched for relevant papers using academic databases. This initial pool was extended based on suggestions from other co-authors. Identified papers were assessed by one author and subsequently reviewed by a second author; papers that did not add new insights beyond those already covered were not included. Multiple review rounds were then carried out on the guidelines as a whole, during which selected papers were cross-checked by additional authors and, where necessary, further examples were added or replaced. The examples are intended to be illustrative, not the result of a systematic review; the involvement of multiple authors at different stages helped mitigate individual selection bias.

After completing the internal review, we invited external experts in empirical SE to review the study types and guidelines. ~~After incorporating their feedback,~~ [2] ~~we compiled and published this article~~During the review process of the *Empirical Software Engineering* journal, the structure and content were improved based on reviewers' suggestions. We self-assessed our guidelines against the quality dimensions of the *SIGSOFT Empirical Standards* [123]; this assessment is reported in the conclusion.

4 Study Types

~~The development of empirical guidelines for software engineering (SE)~~ Empirical guidelines are needed for SE studies involving LLMs ~~is crucial to ensuring~~ to ensure the validity and reproducibility of their results. However, such guidelines must be tailored to different study types that each pose unique challenges. Therefore, we ~~developed~~ created a taxonomy of study types ~~that we then use to contextualize the recommendationswe provide.~~

to contextualize our recommendations. We use the term *study type* to refer to categories of LLM involvement in empirical research, rather than to research methodologies (e.g., experiments, case studies). A single empirical study may involve multiple such study types. Each study type section starts with a **description**, followed by **examples** from the SE research community and beyond~~as well as the~~ . The **advantages** and **challenges** of using LLMs ~~for the respective study type. The remainder~~ are discussed in a summary subsection at the end of this section~~is structured as follows:~~ , synthesizing cross-cutting themes across all study types. See Table 1 for an overview of study types.

[2] ~~Discussion:~~

Table 1 Overview of study types.

ID	Section	Topic
	4.1	**LLMs as Tools for Software Engineering Researchers**
S1	4.1.1	LLMs as Annotators
S2	4.1.2	LLMs as Judges
S3	4.1.3	LLMs for Synthesis
S4	4.1.4	LLMs as Subjects
	4.2	**LLMs as Tools for Software Engineers**
S5	4.2.1	Studying LLM Usage in Software Engineering
S6	4.2.2	LLMs for New Software Engineering Tools
S7	4.2.3	Benchmarking LLMs for Software Engineering Tasks

4.1 LLMs as Tools for Software Engineering Researchers

LLMs can serve as ~~powerful~~ tools to help researchers conduct empirical studies. They can automate various tasks such as data collection, pre-processing, and analysis. For example, LLMs can apply predefined coding guides to qualitative datasets (*LLMs as Annotators*), assess the quality of software artifacts (*LLMs as Judges*), generate summaries of research papers (*LLMs for Synthesis*), or simulate human behavior in empirical studies (*LLMs as Subjects*). ~~This can significantly~~ If LLMs could complete these tasks well enough, they would reduce the time and effort required to conduct a study. However, ~~besides the many advantages that all these applications bring, they also come with challenges such as potential threats~~ these applications each pose challenges to validity and ~~implications for the reproducibilityof study results~~reproducibility.

4.1.1 LLMs as Annotators (S1)

Description: ~~Just like human annotators, LLMs can label artifacts based on a pre-defined coding guide. However, they can label data much faster than any human could.~~ In qualitative data analysis, manually annotating ("coding") natural language text ~~, e.g., in software artifacts, open-ended survey responses, or~~ (e.g., requirements, interview transcripts, open-ended survey responses) is a time-consuming manual process [16]. LLMs can ~~be used to augment or replace human annotations, provide suggestions for new codes(see Section), or even automate the entire qualitative data analysis process.~~ augment human coding, suggest new codes, and label artifacts based on a pre-defined coding guide much faster than humans can [60]. The extent to which, or under what conditions, LLMs can perform these tasks *effectively* remains an open research question [4]. Indeed, measuring their effectiveness is practically and philosophically challenging. From an interpretivist philosophical perspective, one cannot measure the quality of analysis by comparing one (human or machine) analyst's work to another. From a realist perspective, triangulating across multiple human judges, LLM judges, and other data sources (e.g.,

whether a pull request is marked as having resolved an issue) improves confidence in the findings but does not prove that any one judge is valid.

~~Recent work in software engineering has begun to explore the use of LLMs for annotation tasks.~~

Examples: Huang et al [66] ~~proposed an approach that utilizes~~ utilized multiple LLMs for joint annotation of mobile application reviews. They used three models of comparable size with an absolute majority voting rule (i.e., a label is only accepted if it receives more than half of the total votes from the models). ~~Accordingly, the annotations fell into three categories: exact matches (where all models agreed), partial matches (where a majority agreed), and non-matches (where no majority was reached). The study by~~ This approach slightly outperformed the best individual model tested. Meanwhile, Ahmed et al [4] examined LLMs as annotators in ~~software engineering~~ SE research across five datasets, six LLMs, and ten annotation tasks. They found that model-model agreement strongly correlates with human-model agreement~~, suggesting situations in which LLMs could effectively replace human annotators. Their research showed that for~~ ; models performed poorly in tasks where humans ~~themselves disagreed significantly , models also performed poorly. Conversely, if multiple LLMs reach similar solutions independently, then LLMs are likely suitable for the annotation task.~~ also frequently disagreed. They proposed to use model confidence scores to identify specific samples that could be safely delegated to LLMs, potentially reducing human annotation effort without compromising inter-rater agreement.

~~Recent research demonstrates several advantages of using LLMs as annotators, including their cost effectiveness and accuracy. LLM-based annotation can dramatically reduce costs compared to human labeling, with studies showing cost reductions of 50-96% on various natural language tasks [164]. For example, He et al found in their study that GPT-4 annotation costs only $122.08 compared to $4,508 for a comparable MTurk pipeline [60]. Moreover, the LLM-based approach resulted in a completion time of just 2 days versus several weeks for the crowd-sourced approach. LLMs consistently demonstrate strong performance, with ChatGPT's accuracy exceeding crowd workers by approximately 25% on average [52], achieving impressive results in specific tasks such as sentiment analysis (65% accuracy) [182]. LLMs also show remarkably high inter-rater agreement, higher than crowd workers and trained annotators [52].~~

~~Challenges of using LLMs as annotators include reliability issues, human-LLM interaction challenges, biases, errors, and resource considerations. Studies suggest that while LLMs show promise as annotation tools in SE research, their optimal use may be in augmenting rather than replacing human annotators [60, 164] . LLMs can negatively affect human judgment when labels are incorrect [65] , and their overconfidence requires careful verification [160]. Moreover, in previous studies, LLMs have shown significant variability in annotation quality depending on the dataset and the annotation task [116]. Studies have shown that LLMs are especially unreliable for high-stakes labeling tasks [165] and that LLMs can have notable performance disparities between label categories [182]~~

~~. Recent empirical evidence indicates that LLM consistency in text annotation often falls below scientific reliability thresholds, with outputs being sensitive to minor prompt variations [126]. Context-dependent annotations pose a specific challenge, as LLMs show difficulty in correctly interpreting text segments that require broader contextual understanding [60]. Although pooling of multiple outputs can improve reliability, this approach requires additional computational resources and still requires validation against human-annotated data. While generally cost-effective, LLM annotation requires careful management of per token charges, particularly for longer texts [164]. Furthermore, achieving reliable annotations may require multiple runs of the same input to enable majority voting [126], although the exact cost comparison between LLM-based and human annotation is controversial [60]. Finally, research has identified consistent biases in label assignment, including tendencies to overestimate certain labels and misclassify neutral content [182].~~

4.1.2 LLMs as Judges (S2)

Description: LLMs can ~~act as judges or raters to evaluate~~ rate properties of software artifacts ~~. For example, LLMs can be used to~~ (e.g. assess code readability, adherence to coding standards, or ~~the qualityof code comments. Judgment is distinct from the more qualitative task of assigning a code or label to typically~~ comment quality) or sort multiple solutions by some attribute. These kinds of judgments are distinct from annotating unstructured text (see Section *LLMs as Annotators*). ~~It is also different from using LLMs for general software engineering tasks, as discussed in Section .~~

Example(s): Lubos et al [95] leveraged Llama-2 to evaluate the quality of software requirements statements. They prompted the LLM with the text below, where the words in ~~curly brackets~~ braces reflect the study parameters:

```
Your task is to evaluate the quality of a software requirement.
Evaluate whether the following requirement is {quality_characteristic}.
{quality_characteristic} means: {quality_characteristic_explanation}
The evaluation result must be: 'yes' or 'no'.
Request: Based on the following description of the project: {project_description}
Evaluate the quality of the following requirement: {requirement}.
Explain your decision and suggest an improved version.
```

~~Lubos et al evaluated the LLM 's output against human judges, to assess how well the LLM matched experts. Agreement can be measured in many ways; this study used Cohen's kappa [33]~~ They evaluated LLM output against expert human judges and found moderate agreement for simple requirements and poor agreement for more complex requirements. ~~However, human evaluation of machine judgments may not be the same as evaluation of human judgments and is an open area of research [45]. A crucial decision in studies focusing on LLMs as judges is the number of examples provided to the LLM. This might involve no tuning (zero-shot), several examples (few-shot), or providing many examples (closer to traditional training data). In the example above,~~

~~Lubos et al chose zero-shot tuning, providing no specific guidance besides the provided context; they did not show the LLM what a 'yes' or 'no' answer might look like. In~~ In contrast, Wang et al [163] ~~provided a rubric to an LLM when treating LLMs as a judge to produce interpretable scales (0~~ used an LLM to ~~4) for the overall quality of acceptance criteria generated from user stories. They combined this with a lower-level reward stage to refine acceptance criteria, which significantly improved correctness, clarity, and alignment with the user stories.~~

~~Depending on the model configuration, LLMs can provide relatively consistent evaluations. They can further help mitigate human biases and the general variability that human judges might introduce. This may lead to more reliable and reproducible results in empirical studies, to the extent that these models can be reproduced or checkpointed. LLMs can be much more efficient and scale more easily than the equivalent human approach. With LLM automation, entire datasets can be assessed, as opposed to subsets, and the assessment produced can be at the selected level of granularity, e.g., binary outputs ('yes' or 'no') or defined levels (e.g.,~~ generate acceptance criteria for user stories, and provided a rubric to an LLM to judge the generated acceptance criteria on interpretable scales (0 to 4). ~~However, the main constraint that varies by model and budget is the input context size, i.e., the number of tokens one can pass into a model. For example, the upper bound of the context that can be passed to OpenAi's o1-mini model is 32k tokens.~~

~~When relying on the judgment of LLMs, researchers must build a *reliable* process for generating judgment labels that considers the non-deterministic nature of LLMs and report the intricacies of that process transparently [139]. For example, the order of options has been shown to affect LLM outputs in multiple-choice settings [120]. In addition to reliability, other quality attributes to consider include the *accuracy* of the labels. For example, a reliable LLM might be reliably inaccurate and wrong. Evaluating and judging large numbers of items, for example, to perform fault localization on the thousands of bugs that large open-source projects have to deal with, comes with costs in clock time, compute time, and environmental sustainability. Evidence shows that LLMs can behave differently when reviewing their own outputs [117]. In more human-oriented datasets (such as discussions of pull requests)LLMs may suffer from well-documented biases and issues with fairness [48]. For tasks in which human judges disagree significantly, it is not clear if an LLM judge should reflect the majority opinion or act as an independent judge. The underlying statistical framework of an LLM usually pushes outputs towards the most likely (majority) answer. There is ongoing research on how suitable LLMs are as independent judges. Questions about bias, accuracy, and trust remain [19]. There is reason for concern about LLMs judging student assignments or doing peer review of scientific papers [179]. Even beyond questions of technical capacity, ethical questions remain, particularly if there is some implicit expectation that a human is judging the output. Involving a human in the judgment loop, for example, to contextualize the scoring, is one approach [115]. However, the~~

~~lack of large-scale ground truth datasets for benchmarking LLM performance in judgment studies hinders progress in this area.~~

4.1.3 LLMs for Synthesis (S3)

Description: ~~LLMs can support synthesis tasks in software engineering research by processing and distilling information from qualitative data sources. In this context~~Unlike annotation (see Section *LLMs as Annotators*), which focuses on categorizing or labeling individual data points, synthesis refers to the process of integrating and interpreting information from multiple sources to generate higher-level insights, identify patterns across datasets, and develop conceptual frameworks or theories. ~~Unlike annotation (see Section), which focuses on categorizing or labeling individual data points, synthesis involves connecting and interpreting these annotations to develop a cohesive understanding of the phenomenon being studied. Synthesis tasks can also mean generating synthetic datasets~~ LLMs may be able to support synthesis tasks in SE research by processing and distilling information from qualitative data sources. Although synthesis in the preceding notion refers to abstraction and interpretation across multiple data sources, the term is sometimes also used to refer to generating synthetic content (e.g., source code, bug-fix pairs, requirements, etc.) that are then used in downstream tasks to train, fine-tune, or evaluate existing models or tools. In this case, the synthesis is done primarily using the LLM and its training data; the input is limited to basic instructions and examples.

Example(s): Published examples of applying LLMs for synthesis in ~~the software engineering domain are still scarce. However, recent work has explored the use of LLMs for qualitative synthesis~~ SE remain scarce; however, some recent work in other domains ~~, allowing reflection on how LLMs can be applied for this purpose in software engineering~~is instructive [16]. Barros et al [18] conducted a systematic mapping study on ~~the use of~~ using LLMs for qualitative research ~~. They identified examples in domains such as~~ and found that most studies focused on the healthcare and social sciences~~(see, e.g., [38, 97]) in which LLMs were used to support qualitative analysis, including~~ . In these studies, LLMs supported qualitative methods, such as grounded theory and thematic analysis, by aiding in pattern identification. In SE, de Morais Leca et al. [106] explored how LLMs have been applied for qualitative data analysis (QDA) and proposed general strategies and guidelines for their application. Ornelas et al. [114] complemented this perspective by studying the opportunities and limitations of introducing LLM-based support into QDA, and by formulating recommendations for embedding human–AI collaboration across the thematic analysis phases. Building on these, subsequent work has proposed hybrid frameworks combining LLM support with human-led QDA. Rashid et al. [125] designed an LLM-driven multi-agent system that integrates AI with human decision-making to automate qualitative data analysis methods. Their system generated initial codes, developed themes, and summarized text. ~~Overall, the findings highlight the successful generation of preliminary coding schemes from~~

~~interview transcripts, later refined by human researchers, along with support for pattern identification. This approach was reported not only to expedite the initial coding process but also to allow researchers to focus more on higher-level analysis and interpretation. However, Barros et al emphasize that effective use of LLMs requires structured prompts and careful human oversight.~~ Similarly, ~~de Morais Leça et al [106] conducted a systematic mapping study to investigate how LLMs are used in qualitative analysis and how they can be applied in software engineering research. Consistent with the study by Barros et al [18], de Morais Leça et al identified that LLMs are applied primarily in tasks such as coding, thematic analysis, and data categorization, reducing the time, cognitive demands, and resources required for these processes~~Montes et al. [105] compared the performance of humans and LLMs in coding, theme development, definition, and refinement, creating guidelines for a hybrid-LLM framework. Finally, El-Hajjami and Salinesi's work is an example of using LLMs to create synthetic datasets. They present an approach to generate synthetic requirements, showing that they "*can match or surpass human-authored requirements for specific classification tasks*" [44].

~~LLMs offer promising support for synthesis in SE research by helping researchers process artifacts such as interview transcripts and survey responses, or by assisting literature reviews. Qualitative research in SE traditionally faces challenges such as limited scalability, inconsistencies in coding, difficulties in generalizing findings from small or context-specific samples, and the influence of the researchers' subjectivity on data interpretation [16]. The use of LLMs for synthesis can offer advantages in addressing these challenges [16, 18, 106] . LLMs can reduce manual effort and subjectivity, improve consistency and generalizability, and assist researchers in deriving codes and developing coding guides during the early stages of qualitative data analysis [16, 24]. LLMs can enable researchers to analyze larger datasets, identifying patterns across broader contexts than traditional qualitative methods typically allow. In addition, they can help mitigate the effects of human subjectivity. However, while LLMs streamline many aspects of qualitative synthesis, careful oversight remains essential.~~

~~Although LLMs have the potential to automate synthesis, concerns about overreliance remain, especially due to discrepancies between AI- and human-generated insights, particularly in capturing contextual nuances [15]. Bano et al [15] found that while LLMs can provide structured summaries and qualitative coding frameworks, they may misinterpret nuanced qualitative data due to a lack of contextual understanding. Other studies have echoed similar concerns [16, 18, 106] . In particular, LLMs cannot independently assess the validity of arguments. Critical thinking remains a human responsibility in qualitative synthesis. Peters and Chin-Yee have shown that popular LLMs and LLM-based tools tend to overgeneralize results when summarizing scientific articles, that is, they Compared to human-authored summaries, [119]. In addition, LLMs can produce biased results, reinforcing existing prejudices or omitting essential perspectives, making human oversight crucial to ensure accurate interpretation, mitigate biases, and maintain quality control. Moreover, the proprietary nature of many LLMs limits transparency.~~

~~In particular, it is unknown how the training data might affect the synthesis process. Furthermore, reproducibility issues persist due to the influence of model versions and prompt variations on the synthesis result.~~

4.1.4 LLMs as Subjects (S4)

Description: In empirical studies, data is collected from participants through methods such as surveys, interviews, or controlled experiments. LLMs can serve as virtual *subjects* ~~in empirical studies~~ by simulating human behavior and interactions~~(we use the term "subject" since "participant" implies being human [10]). In this capacity, LLMs~~ . If LLMs can generate responses that approximate those of human participants, ~~which makes them particularly~~ they could be valuable for research involving user interactions, collaborative coding environments, and software usability assessments~~. This approach enables data collection that closely reflects human reactions while avoiding the need for direct human involvement.~~ [178]. To achieve this, prompt engineering techniques are widely employed~~, with a common approach being the use of the~~ ; for instance, the *Personas Pattern* [81] ~~, which~~ involves tailoring LLM responses to align with predefined profiles or roles that emulate specific user archetypes. ~~Zhao et al outline opportunities and challenges of using LLMs as research subjectsin detail [178]. Furthermore, recent sociological studies have emphasized that, to be effectively utilized in this capacity, LLMs—including their agentic versions—should meet four criteria of algorithmic fidelity [9]. Generated~~ To serve as virtual subjects, generated responses should be ~~: (1)~~ indistinguishable from human-produced texts~~(e.g., LLM-generated code reviews should be comparable to those from real developers); (2)~~ , consistent with the attitudes and sociodemographic information of the conditioning context (e.g., ~~LLMs simulating junior developers should exhibit different confidence levels, vocabulary, and concerns compared to senior engineers); (3)~~ junior vs. senior developers), naturally aligned with the form, tone, and content of the ~~provided context (e.g., responses in an agile stand-up meeting simulation should be concise, task-focused, and aligned with sprint objectives rather than long, formal explanations); and (4) reflective of~~ simulated scenario, and reflect patterns in relationships between ideas, demographics, and behavior observed in comparable human data~~(e.g., discussions on software architecture decisions should capture trade-offs typically debated by human developers, such as maintainability versus performance, rather than abstract theoretical arguments)~~ [9].

Example(s): ~~LLMs can be used as subjects in various types of empirical studies, enabling researchers to simulate human participants. The broader applicability of LLM-based studies beyond software engineering has been compiled by~~ Xu et al [173] ~~, who examined various uses in~~ compiled a list of ways LLMs can support social science research~~. Given the socio-technical nature of software development~~, some of ~~these approaches are transferable to empirical software engineering~~ which transfer to empirical SE research. For example,

LLMs can ~~be applied in survey and interview studies to impersonate developers responding to survey questionnaires or interviews, allowing researchers to test the clarity and effectiveness of survey items or to simulate responses under varying conditions, such as different levels of expertise or cultural contexts. For example, Gerosa et al [50] explored persona-based interviews and multi-persona focus groups, demonstrating how LLMs can~~ emulate human responses and behaviors ~~while addressing ethical concerns, biases, and methodological challenges. Another example is usability studies, in which LLMs can simulate end-user feedback, providing insights into potential usability issues and offering suggestions for improvement based on predefined user personas. This aligns with the work of~~ in simulated interviews and focus groups Gerosa et al [50]. Similarly, Bano et al [17], ~~who~~ investigated biases in LLM-generated candidate profiles in SE recruitment processes. ~~Their study, which analyzed both textual and visual inputs, revealed~~ They found biases favoring male ~~Caucasian~~ candidates, lighter skin tones, and slim physiques, particularly for senior roles. LLMs may be able to simulate end-user feedback and behavior in usability studies, identify usability issues and offering suggestions for improvement based on predefined user personas.

~~Using LLMs as subjects can reduce the effort of recruiting human participants, a process that is often time-consuming and costly [96], by augmenting existing datasets or, in some cases, completely replacing human participants. In interviews and survey studies, LLMs can simulate diverse respondent profiles, enabling access to underrepresented populations, which can strengthen the generalizability of findings. In data mining studies, LLMs can generate synthetic data (see) to fill gaps where real-world data is unavailable or underrepresented.~~

~~It is important that researchers are aware of the inherent biases [37] and limitations [58, 161] when using LLMs as study subjects. Schröder et al [138] , who have studied discrepancies between LLM and human responses, even conclude that One critical concern is construct validity. LLMs have been shown to misrepresent demographic group perspectives, failing to capture the diversity of opinions and experiences within a group. The use of identity-based prompts can reduce identities to fixed and innate characteristics, amplifying perceived differences between groups. These biases introduce the risk that studies which rely on LLM-generated responses may inadvertently reinforce stereotypes or misrepresent real-world social dynamics. Alternatives to demographic prompts can be employed when the goal is to broaden response coverage [161]. Beyond construct validity, internal validity must also be considered, particularly with regard to causal conclusions based on studies relying on LLM-simulated responses. Finally, external validity remains a challenge, as findings based on LLMs may not generalize to humans.~~

4.2 LLMs as Tools for Software Engineers

LLM-based assistants have become ~~an essential~~ a widely adopted tool for software engineers [149], supporting them in various tasks such as code generation

and debugging. Researchers have studied how software engineers use LLMs (*Studying LLM Usage*), developed new tools that integrate LLMs (*LLMs for Tools*), and benchmarked LLMs for software engineering tasks (*Benchmarking LLMs*).

4.2.1 Studying LLM Usage in Software Engineering (S5)

Description: Studying how software engineers use LLMs ~~is crucial to understand~~ and LLM-based tools is important for understanding the current state of practice in SE. Researchers can observe software engineers' usage of LLM-based tools in the field, or study if and how they adopt such tools, their usage patterns, as well as perceived benefits and challenges. Surveys, interviews, observational studies, or analysis of usage logs can provide insights into how LLMs are integrated into development processes, how they influence decision making, and what factors affect their acceptance and effectiveness. Such studies can inform improvements for existing LLM-based tools, motivate the design of novel tools, or derive best practices for LLM-assisted software engineering. They can also uncover risks or deficiencies of existing tools.

Example(s): Based on a convergent mixed-methods study, Russo has found that early adoption of generative AI by software engineers is primarily driven by compatibility with existing workflows [132]. Khojah et al investigated the use of ChatGPT (GPT-3.5) by professional software engineers in a week-long observational study [78]. They found that most developers do not use the code generated by ChatGPT directly but instead use the output as a guide to implement their own solutions. Azanza et al~~conducted a case study that evaluated the impact of introducing LLMs on the onboarding process of new software developers [13](GPT-3). Their study identified potential in the use of LLMs for onboarding, as it can allow newcomers to seek information on their own, without the need to senior colleagues. Jahic and Sami surveyed~~ 's case study found that LLMs could "*enhance personalized, instant onboarding support; however, relying on proprietary external LLMs poses significant data privacy risks*" [13]. Jahic and Sami found that most participants from 15 software companies ~~regarding their practices on LLMs in software engineering [70] . They found that the majority of study participants~~ they surveyed had already adopted AI ~~for software engineering tasks; most of them used ChatGPT. Multiple participants~~ (especially ChatGPT) for SE tasks; but cited copyright and privacy issues, as well as inconsistent or low-quality outputs, as barriers to adoption [70]. Retrospective studies that analyze data generated while developers use LLMs can provide additional insights into human-LLM interactions. For example, researchers can employ data mining methods to build large-scale conversation datasets, such as the DevGPT dataset introduced by Xiao et al [171]. Conversations can then be analyzed using quantitative [121] and qualitative [103] analysis methods.

~~Studying the real-world usage of LLM-based tools allows researchers to understand the state of practice and guide future research directions. In field~~

~~studies, researchers can uncover usage patterns, adoption rates, and the influence of contextual factors on usage behavior. Outside of a controlled study environment, researchers can uncover contextual information about LLM-assisted SE workflows beyond the specific LLMs being evaluated. This may, for example, help researchers generate hypotheses about how LLMs impact developer productivity, collaboration, and decision-making processes. In controlled laboratory studies, researchers can study specific phenomena related to LLM usage under carefully regulated, but potentially artificial conditions. Specifically, they can isolate individual tasks in the software engineering workflow and investigate how LLM-based tools may support task completion. Furthermore, controlled experiments allow for direct comparisons between different LLM-based tools. The results can then be used to validate general hypotheses about human-LLM interactions in SE.~~

~~When conducting field studies in real-world environments, researchers have to ensure that their study results are [152] beyond the traditional validity criteria such as internal or construct validity. The usage environment in a real-world context can often be extremely diverse. The integration of LLMs can range from specific LLMs based on company policy to the unregulated use of any available LLM. Both extremes may influence the adoption of LLM by software engineers, and hence need to be addressed in the study methodology. In addition, in longitudinal case studies, the timing of the study may have a significant impact on its result, as LLMs and LLM-based tools are rapidly evolving. Moreover, developers are still learning how to best make use of the new technology, and best practices are still being developed and established. Furthermore, the predominance of proprietary commercial LLM-based tools in the market poses a significant barrier to research. Limited access to telemetry data or other usage metrics restricts the ability of researchers to conduct comprehensive analyses of real-world tool usage. To make study results reliable, researchers must establish that their results are rigorous, for example, by using triangulation to understand and minimize potential biases [152]. When it comes to studying LLM usage in controlled laboratory studies, researchers may struggle with the inherent variability of LLM outputs. Since reproducibility and precision are essential for hypothesis testing, the stochastic nature of LLM responses—where identical prompts may yield different outputs across participants—can complicate the interpretation of experimental results and affect the reliability of study findings.~~

4.2.2 LLMs for New Software Engineering Tools (S6)

Description: LLMs are being integrated into new tools that support software engineers in their daily tasks ~~,~~ (e.g., to assist in code comprehension [175] and test case generation [135]). One way of integrating LLM-based tools into software engineers' workflows ~~are~~ is using GenAI agents. Unlike traditional LLM-based tools, these agents are capable of acting autonomously and proactively, are often tailored to meet specific user needs, and can interact with external environments [154, 167]. From an architectural perspective, GenAI

agents can be implemented in various ways [167]. However, they generally share three key components: (1) a reasoning mechanism that guides the LLM (often enabled by advanced prompt engineering), (2) a set of tools to interact with external systems (e.g., APIs or databases), and (3) a user communication interface that extends beyond traditional chat-based interactions [129, 153, 180]. Researchers can also test and compare different tool architectures to increase artifact quality and developer satisfaction.

Example(s): Yan et al proposed ~~IVIE~~*IVIE*, a tool integrated into the VS Code graphical interface that generates and explains code using LLMs [175]. The authors focused more on the presentation, providing a user-friendly interface to interact with the LLM. Schäfer et al [135] presented a large-scale empirical evaluation on the effectiveness of LLMs for automated unit test generation. They presented ~~TestPilot~~*TestPilot*, a tool that implements an approach in which the LLM is provided with prompts that include the signature and implementation of a function under test, along with usage examples extracted from the documentation. Richards and Wessel introduced a preliminary GenAI agent designed to assist developers in understanding source code by incorporating a reasoning component grounded in the theory of mind [129].

~~From an engineering perspective, developing LLM-based tools is easier than implementing many traditional SE approaches such as static analysis or symbolic execution. Depending on the capabilities of the underlying model, it is also easier to build tools that are independent of a specific programming language. This enables researchers to build tools for a more diverse set of tasks. In addition, it allows them to test their tools in a wider range of contexts.~~

~~Traditional approaches, such as static analysis, are deterministic. LLMs are not. Although the non-determinism of LLMs can be mitigated using configuration parameters and prompting strategies, this poses a major challenge. It can be challenging for researchers to evaluate the effectiveness of a tool, as minor changes in the input can lead to major differences in the performance. Since the exact training data is often not published by model vendors, a reliable assessment of tool performance for unknown data is difficult. From an engineering perspective, while open models are available, the most capable ones require substantial hardware resources. Using cloud-based APIs or relying on third-party providers for hosting, while seemingly a potential solution, introduces new concerns related to data privacy and security.~~

4.2.3 Benchmarking LLMs for Software Engineering Tasks (S7)

Description: Benchmarking is the process of evaluating an LLM's performance ~~using standardized tasksand metrics, which requires high-quality reference datasets.~~ on standardized tasks, using standardized metrics, on a standardized dataset. The LLM output is compared to a supposed ground truth from the benchmark dataset~~using~~. Typical tasks include code generation, code summarization, code completion, and code repair, but also natural language processing tasks, such as anaphora resolution (i.e., the task of identifying the

referring expression of a word or phrase occurring earlier in the text). Metrics may include general metrics for text generation, such as *ROUGE*, *BLEU*, or *METEOR* [63], or task-specific metrics, such as *CodeBLEU* for code generation. ~~For example, *HumanEval* [30] is often used to assess code generation, establishing it as a de facto standard~~Benchmarking requires high-quality reference datasets.

Example(s): In SE, benchmarking may include ~~the evaluation of~~ evaluating an LLM's ability to produce accurate and reliable outputs for a given input ~~,~~ (usually a task description, ~~which may be~~ possibly accompanied by data obtained from curated real-world projects or from synthetic SE-specific datasets~~. Typical tasks include code generation, code summarization, code completion, and code repair, but also natural language processing tasks, such as anaphora resolution (i.e., the task of identifying the referring expression of a word or phrase occurring earlier in the text~~). *RepairBench* [144], for example, contains 574 buggy Java methods and their corresponding fixed versions, which can be used to evaluate the performance of LLMs in code repair tasks. ~~This benchmark~~ It uses the *Plausible@1* metric (i.e., the probability that the first generated patch passes all test cases) and the *AST Match@1* metric (i.e., the probability that the abstract syntax tree of the first generated patch matches the ground truth patch). *SWE-Bench* [74] is a more generic benchmark that contains 2,294 SE Python tasks extracted from GitHub pull requests. To score the LLM's ~~performance on the tasks~~task performance, the benchmark validates whether the generated patch ~~is applicable (i.e., successfully compiles)~~ successfully compiles and calculates the percentage of passed test cases. Meanwhile, *HumanEval* [30] is often used to assess code generation.

4.3 Advantages and Challenges

Using LLMs in empirical SE research, whether as tools for researchers or for software engineers, offers several potential advantages but also raises fundamental challenges that cut across the study types described above.

Advantages: ~~Properly built benchmarks provide objective, reproducible evaluation across different tasks~~ The primary advantage of using LLMs for research tasks is *speed, cost reduction, and scalability*. LLMs can annotate, judge, synthesize, and simulate faster and at lower cost than human researchers, with studies showing cost reductions of 50–96% on various natural language tasks [164] (e.g., He et al found that GPT-4 annotation required only two days and 122.08 USD compared to several weeks and 4,~~enabling a comparison between different models (and versions). In addition, benchmarks built for specific SE tasks can help identify LLMweaknesses and support their optimization and fine-tuning for such tasks. They can foster open science practices by providing a~~ 508 USD for a comparable MTurk pipeline [60]). Similarly, augmenting or replacing human participants with LLM-generated virtual participants would reduce

recruitment effort [96]. This efficiency unlocks scalability: qualitative research traditionally does not scale well to large samples, but LLMs allow researchers to analyze larger datasets [16, 18, 106] and support larger judgment datasets.

LLMs can also *automate* tasks such as coding qualitative data, assessing artifact quality, and generating summaries, reducing cognitive demands and resources required for qualitative research. Some studies suggest that LLMs may improve *consistency*: ChatGPT's accuracy exceeded crowd workers by approximately 25%, and LLMs can achieve higher inter-rater agreement than crowd workers and trained annotators [52]. However, no compelling evidence supports claims that LLMs are less biased than human annotators [52].

LLMs could potentially provide *access to otherwise-inaccessible research contexts. If* virtual participants' behavior is sufficiently similar to human participants, LLMs could access underrepresented and hard-to-reach populations, strengthen generalizability and inclusiveness, impute missing data (see *LLMs for Synthesis*), and enable research that is ethically problematic with real humans (e.g., questions that would force a human to relive past trauma do not harm an LLM).

Studying real-world LLM-based tools allows researchers to *understand the state of practice*, uncovering usage patterns, adoption rates, and contextual factors, and *generate hypotheses* about how LLMs affect developer productivity, collaboration, and decision-making. From an engineering perspective, developing LLM-based tools *requires less task-specific engineering* than traditional SE approaches such as static analysis or symbolic execution, because a single model can handle diverse inputs without language-specific parsers or hand-crafted rules. Good benchmarks provide *standardized evaluation and model comparison*, reducing research effort and fostering open science by providing common ground for sharing data ~~(e.g., as part of the benchmark itself) and results(e.g., of models run against a benchmark)~~and results. Benchmarks built ~~using real-world data can help legitimize research results for practitioners, supporting industry-academia collaboration~~for specific SE tasks can help identify LLM weaknesses and support optimization and fine-tuning.

Challenges: ~~Benchmark contamination, that is, the inclusion of the benchmark in the LLM training data [5], has recently been identified as a problem. The careful selection of samples and the creation of the corresponding input prompts is particularly important, as correlations between prompts may bias the benchmark results [145]. Although LLMs might perform well on a specific benchmark such as~~ The stochastic nature of LLM responses, where identical prompts may yield different outputs, *undermines replicability* across all study types, complicating interpretation of experimental results and test-retest reliability. More broadly, *reliability* is a persistent concern: conceptualizing an LLM-as-judge or LLM-as-annotator as a measurement instrument, researchers should expect validity, test-retest reliability, inter-rater reliability, minimal error, and measurement invariance [124]. However, LLMs show significant variability depending on the dataset and task [116], are sensitive to prompt variations [126] and option

order [120], can behave differently when reviewing their own outputs [117], and can be unreliable for high-stakes labeling [165]. Crucially, *reliability does not imply validity*: a reliable LLM might be reliably inaccurate [179], and for tasks with no single correct answer, the statistical framework pushes outputs toward the most likely answer, which may not be the best one [19]. See *Version and Configuration*and *Limitations and Mitigations*for reporting guidance on replicability and reliability.

The *rapid evolution* of LLMs and LLM-based tools, combined with professionals rapidly learning how to employ them, complicates longitudinal comparisons and may quickly render study findings obsolete (e.g., GPT-4's accuracy on a simple mathematical task dropped from 84% to 51% within three months [29]).

The prevalence of *proprietary tools and opaque training data* limits researchers' ability to assess and mitigate biases, and enables benchmark contamination [5] . LLMs exhibit ~~*HumanEval*~~*bias* ~~or *SWE-bench*, they do not necessarily perform well on other benchmarks. Moreover, benchmarks~~ in multiple forms: tendencies to overestimate certain labels [182], well-documented fairness issues [48], and the potential to reinforce prejudices. When simulating human participants, LLMs are "*likely to both misportray and flatten the representations of demographic groups*" [161]. See *Open LLMs*and *Limitations and Mitigations*for mitigation strategies.

Evaluation difficulty is inherent in studying LLM-based tools and benchmarks. Benchmarks may lack construct validity [124], usually do not capture the full complexity ~~and diversity~~ of software engineering work [27], and may lead to *overconfidence* and overfitting [160]. LLMs are also susceptible to adversarial manipulation where semantics-preserving changes may deceive them into accepting flawed artifacts [59]. See *Benchmarks and Metrics*for detailed guidance on benchmark design, including Cao et al's [26] guidelines for coding task benchmarks.

A fundamental challenge is *philosophical and methodological incongruence* with qualitative research. In a recent open letter, 419 experienced qualitative researchers argued "*that analytic approaches such as reflexive thematic analysis are human research practices requiring a subjective, positioned, and reflexive researcher and therefore the use of GenAI in such approaches is not methodologically congruent*" [75]. LLMs lack the capacity for genuinely reflexive qualitative analysis because they operate on statistical prediction without understanding the meaning of the data being analyzed [15, 75, 105]. Effective use requires structured prompts and careful human oversight [18]; see *Human Validation*and *Limitations and Mitigations*for guidance. ~~Another issue is that metrics used for benchmarks, such as *perplexity* or *BLEU-N*, do not necessarily reflect human judgment. Recently, Cao et al [26] have proposed guidelines for the creation of LLM benchmarks related to coding tasks, grounded in a systematic survey of existing benchmarks. In this process, they highlight current shortcomings related to reliability, transparency, irreproducibility, low data quality, and inadequate validation measures. For more details on benchmarks, see Section .~~

5 ~~Guidelines~~

~~Our guidelines focus on LLMs, that is , foundational models that use~~ There is ~~*text*~~*insufficient evidence* for the effectiveness of LLMs in most research roles. No compelling evidence exists that LLMs can accurately simulate human participants [138] or reliably judge most relevant properties of SE artifacts; gathering such evidence for each specific usage may be quite difficult [58] . LLMs may be better suited for augmenting rather than replacing human researchers [60, 164], but even then, limited evidence exists that augmentation increases *effectiveness* as well as ~~in- and output. We do not address multi-modal foundation models that support other media such as images. However, many of our recommendations apply to these models as well . We consider the extension of our guidelines to explicitly address multi-modal foundational models an important direction for future work. The main~~ *efficiency* [138]. When LLM outputs are incorrect, they can negatively *affect human judgment* [65, 115]; see *Human Validation*for mitigation strategies.

Pooling multiple outputs for majority voting improves reliability but increases *cost and environmental impact* [126, 164]. While open models are available, the most capable ones require substantial hardware; relying on *cloud-based APIs* introduces concerns related to data privacy, security, and replicability. See *Limitations and Mitigations*for sustainability considerations.

Field study findings face *generalizability* challenges because outcomes may be highly sensitive to individual differences, usage patterns, goals, and contexts. Field studies must be "dependable" [152] beyond traditional validity criteria, which complicates methodology; see *Limitations and Mitigations*for a detailed threat taxonomy.

5 Guidelines

A primary goal of our guidelines is to *enable reproducibility and replicability* of empirical SE studies involving LLMs~~in software engineering. While we consider LLM-based studies to have characteristics that differ from traditional empirical studies (e.g., their inherent non-determinism and the fact that truly open models are rare), ~~. As repeating LLM-focused research to verify results lies somewhere between the ACM's definitions of reproducibility (different team, same research artifacts) and repeatability (different team, different research artifacts) [1] due to potential model changes and imperfect research artifacts, we follow Angermeir et al [8] and use the terms interchangeably. While previous guidelines regarding open science and empirical studies still apply~~. Although full reproducibility of LLM study results is very challenging given LLM's~~ , LLM-specific characteristics (e.g. inherent non-determinism [147, 177], opaque and proprietary models) present additional replicability challenges, which, in turn, demand new guidance.

Each guideline below begins with a brief *tl;dr*summary, followed by its **rationale**, ~~transparency on LLM usage, methods, data, and architecture, as~~

~~suggested by our guidelines~~**recommendations**, ~~is an essential prerequisite for future replication studies. The wording of our guidelines (~~**examples**, **benefits**, ~~)follows RFC 2119 [108] and RFC 8174 [109]. Throughout the following sections, we mention the~~ **challenges**, links to the relevant **study types**. The *rationale* articulates the underlying principle, i.e., why the guideline matters, while the *recommendations* provide concrete, actionable practices. Table 4 provides a compact overview of this mapping.

The guidelines further contain **advice for reviewers**. Broadly, reviewers should use our guidelines to help them interrogate the extent to which a manuscript's authors have done what was practically possible to improve reproducibility. We must neither accept research absent reasonable efforts to improve reproducibility, nor reject research for failing to obtain an impossible goal. We borrow this principle of "reasonable efforts" from the SIGSOFT Empirical Standards [123], where it applies to methodological rigor more generally. Of course, papers should acknowledge their limitations, but to determine whether these limitations are reasonable, reviewers should ask "have the authors done what they could to minimize limitations?" Our guidelines attempt to capture what "reasonable efforts" practically means for LLM-based studies.

To distinguish essential criteria from recommendations, our guidelines use two tiers. A MUSTcriterion is a *requirement*. Studies that intend to follow these guidelines are expected to meet all MUSTcriteria. A SHOULDcriterion represents a desired practice that strengthens a study's rigor or transparency. However, there may be valid reasons to deviate in particular circumstances (e.g., resource constraints, inapplicability to a specific study context or type). Nonetheless, studies that deviate from a SHOULDcriterion should briefly justify the deviation and discuss its potential impact (e.g., on validity or reproducibility). Table 2 lists our eight guidelines. Table 3 provides an overview of how each guideline applies to each study type. Cells marked ● indicate that the guideline is a MUSTrequirement for the study type and cells marked ◐ indicate that the guideline SHOULDbe followed. Cells marked – indicate that the guideline is typically not applicable or not feasible for the study type.

The following sections indicate which information we expect researchers to report, and whether it should be in the *paper* ~~and/or in the~~ or *supplementary material*. ~~We are aware that different publication venues have different page limits and that not all aspects can be reported~~ Where a publication venue's page limits hinder reporting all expected elements in the *paper* ~~. If information be reported in the , we explicitly mention this in the specific guidelines. Of course,~~ , it is better to report essential information ~~that we expect in the in the~~ in the *supplementary material* than not at all. The *supplementary material* should be published according to the *ACM SIGSOFT Open Science Policies* [54]. ~~At the beginning of each section, we provide a brief *tl;dr* summary that lists the most important aspects of the corresponding guideline. In addition to our **recommendations**, we provide **examples** from the SE research community and beyond, as well as the **advantages** and potential **challenges** of following the respective guidelines. We conclude each guideline~~

Table 2 Overview of guidelines.

ID	Section	Guideline
G1	5.1	Declare LLM Usage and Role
G2	5.2	Report Model Version, Configuration, and Customizations
G3	5.3	Report Tool Architecture Beyond Models
G4	5.4	Report Prompts, their Development, and Interaction Logs
G5	5.5	Use Human Validation for LLM Outputs
G6	5.6	Use an Open LLM as a Baseline
G7	5.7	Use Suitable Baselines, Benchmarks, and Metrics
G8	5.8	Report Limitations and Mitigations

Table 3 Applicability of guidelines to study types. ● = MUST, ○ = SHOULD. Study type IDs: S1 = *LLMs as Annotators*, S2 = *LLMs as Judges*, S3 = *LLMs for Synthesis*, S4 = *LLMs as Subjects*, S5 = *Studying LLM Usage*, S6 = *LLMs for Tools*, S7 = *Benchmarking LLMs*. Each guideline's study-type-specific guidance is detailed in the corresponding subsections.

	S1	S2	S3	S4	S5	S6	S7
G1: Declare Usage	●	●	●	●	●	●	●
G2: Model Version	●	●	●	●	●	●	●
G3: Architecture	○	○	○	○	○	●	○
G4: Prompts	●	●	●	●	●	●	●
G5: Human Valid.	○	○	○	○	○	○	○
G6: Open LLM	○	○	○	–	–	○	●
G7: Benchmarks	○	○	○	○	○	○	●
G8: Limitations	●	●	●	●	●	●	●

~~by linking it to the **study types**. The remainder of this section presents our eight guidelines and is structured as follows:~~

5.1 Declare LLM Usage and Role (G1)

> ~~*tl;dr:*~~ tl;dr: *Researchers* MUST *disclose any use of LLMs to support empirical studies in their* paper *, specifying which LLM was used, how it was used, and where in the research process it was employed. They* SHOULD *report ~~their purpose, automated tasks , and~~ the exact purpose, the tasks that were automated, and the expected benefits. When the LLM plays a central role, the declaration* SHOULD*be prominent and detailed in the methodology section; for tangential uses, a brief statement in the methodology or acknowledgments suffices.*

5.1.1 ***Rationale:***

Transparency about LLM involvement is a prerequisite for informed assessment of a study's scope, limitations, and potential biases. Without explicit disclosure, readers cannot evaluate how the LLM's characteristics may have influenced the research process or its outcomes.

5.1.2 ***Recommendations:***

When conducting any kind of empirical study involving LLMs, researchers MUST clearly declare that an LLM was used (see *Scope* for what we consider relevant research support). This SHOULD be done in a suitable section of the *paper*, for example, in the introduction or research methods section. For authoring scientific articles, this transparency is, for example, required by the ACM Policy on Authorship: "*The use of generative AI tools and technologies to create content is permitted but must be fully disclosed in the Work*" [11]. ~~Beyond generic authorship declarationsand declarations that LLMs were used as part of the research process~~

Beyond generic declarations, researchers SHOULD report the exact purpose of using an LLM in a study, the tasks it was used to automate, and the expected benefits in the *paper*. A sufficient declaration specifies not only *that* an LLM was used, but also *which* LLM (name and version), *how* it was used (e.g., as an annotator, code generator, or judge), and *where* in the research process it was employed (e.g., data collection, analysis, or synthesis).

When the LLM plays a central role in the study (e.g., as the main tool being evaluated or as a core component of the research method), the declaration SHOULDbe prominent and detailed, appearing in the methodology section with cross-references to the specific guidelines that apply (e.g., Sections *Version and Configuration*, *Architecture*, and *Prompts and Logs*). When the LLM's role is more tangential (e.g., used for a single preprocessing step), a brief but explicit statement in the methodology or acknowledgments section is sufficient. In either case, the disclosure MUSTbe specific enough for readers to assess how the LLM's involvement may affect the study's validity and reproducibility.

5.1.3 ***Example(s):***

The *ACM Policy on Authorship*~~suggests to disclose the usage of GenAI tools in the acknowledgments section of the~~ [11] suggests disclosing GenAI usage in the acknowledgments section of the *paper*, ~~for example: *"ChatGPT was utilized to generate sections of this Work, including text, tables, graphs, code, data, citations"* [11]. Similarly, the acknowledgments section could also be used to disclose GenAI usage for other aspects of the research, if not explicitly described in other sections. The ACM policy further suggests: *"If you are uncertain about the need to disclose the use of a particular tool, err on the side of caution, and include a disclosure in the acknowledgments section of the Work"*~~advising to "err on the side of caution, and include a disclosure in the

acknowledgments section of the Work" when uncertain about the need [11]. For double-blind review, ~~during which the acknowledgments section is usually hidden,~~ researchers can add a temporary "AI Disclosure" section ~~in the same place the acknowledgments section~~ where the acknowledgments would appear. An example of an LLM disclosure beyond writing support can be found in a recent paper by Lubos et al [95], in which they write in the methodology section:

> *"We conducted an LLM-based evaluation of requirements utilizing the Llama 2 language model with 70 billion parameters, fine-tuned to complete chat responses..."*

5.1.4 ***Benefits:***

Transparency in the use of LLMs helps other researchers understand the context and scope of the study, facilitating better interpretation and comparison of the results. Beyond this declaration, we recommend researchers to be explicit about the LLM version they used (see ~~Section~~ *Version and Configuration*) and the LLM's exact role~~(see Section)~~.

5.1.5 ***Challenges:***

~~In general, we expect following our recommendations to be~~ Declaring LLM usage requires only a brief statement and no additional experiments, making compliance straightforward. One challenge might be authors' reluctance to disclose LLM usage for valid use cases, because they fear that AI-generated content makes reviewers think that the authors' work is less original. In fact, there is evidence suggesting that AI disclosure can negatively affect trust in authors [136]. However, the *ACM Policy on Authorship* is very clear in that any use of GenAI tools to create content MUST be disclosed. ~~Before the introduction of LLMs, human proof-reading was possible and allowed and was not required to be declared.~~ Our guidelines focus on research support beyond proof-reading and writing support (see *Scope*)~~. However, over time,~~ , but the threshold of what must be declared ~~and what not will most likely evolve~~ continues to evolve as organizations such as the ACM update their authorship policies.

5.1.6 ***Study Types:***

Researchers MUST follow this guideline for all study types. The specific focus of the declaration varies by study type. For *LLMs as Annotators*, *LLMs as Judges*, *LLMs for Synthesis*, and *LLMs as Subjects*, researchers MUSTdeclare the specific role assigned to the LLM (e.g., annotator, judge, synthesizer, or simulated participant). For *Studying LLM Usage*, researchers MUSTclarify which LLM(s) the observed participants used and under which conditions. For *LLMs for Tools*, researchers MUSTdeclare the LLM's role within the tool architecture and its contribution to the tool's functionality. For *Benchmarking*

LLMs, researchers MUSTdeclare which LLMs were benchmarked and for which tasks.

5.1.7 ***Advice for Reviewers:***

The most common problem with disclosure is incompleteness or vagueness about how the LLM was used. If the paper says "we used LLM X to help with task Y" without specifying how, reviewers should request clarification. Such requests are typically minor revisions unless the missing details may reveal methodological problems.

Using an LLM as a copyeditor becomes problematic when authors do not or cannot take responsibility for the resulting text. If a reviewer finds clear evidence that LLM-generated text was not carefully reviewed (e.g., text like "As a Large Language Model, I..."), this may warrant rejection. If a reviewer suspects an author *cannot* take responsibility for AI-generated text, they should raise the concern with their editor or program chair. Due process requires that authors not be accused of misconduct without clear evidence.

If undisclosed LLM use is suspected, the reviewer should similarly consult their editor or program chair. When the evidence is conclusive, the key question is the degree to which undisclosed use affects the study's contribution, ranging from negligible (e.g., word choice in a single sentence) to severe (e.g., generating ostensibly empirical data or statistical analyses).

5.1.8 ***See Also:***

→ Section *Version and Configuration*: model version and configuration details.

→ Section *Architecture*: system-level role documentation.
→ Section *Prompts and Logs*: prompt documentation and interaction logs.

5.2 Report Model Version, Configuration, and Customizations (G2)

> ~~*tl;dr:*~~ tl;dr: *Researchers* MUST *report the exact LLM model or tool version, configuration, and experiment date in the* paper. *When using quantized models, researchers* SHOULD*report the quantization level and method. For fine-tuned models, they* MUST *describe the fine-tuning goal, dataset, and procedure. Researchers* SHOULD *include default parameters, explain model choices, compare base- and fine-tuned model using suitable metrics and benchmarks, and share fine-tuning data and weights (or alternatively justify why they cannot share them).*

5.2.1 ***Rationale:***

LLMs and LLM-based tools are frequently updated, and configuration parameters such as temperature or seed values affect content generation. This guideline focuses on documenting the *model-specific* aspects of empirical studies involving LLMs~~. While Section addresses how LLMs are integrated into larger systems and tools, here we concentrate~~ , concentrating on the models themselves, their version, configuration parameters, and ~~any direct customizations applied to the model~~ customizations (e.g., fine-tuning). ~~Whether only this guideline or also the following one applies for a particular study depends on the exact study setup and tool architecture. In any event, as soon as an LLM is involved,~~ While the *Architecture*section addresses system-level integration, the information outlined ~~in this guideline is essential to enable reproducibility and replicability~~here is always essential for reproducibility whenever an LLM is involved.

5.2.2 ***Recommendations:***

~~LLMs or LLM-based tools, especially those offered as-a-service, are frequently updated; different versions may produce different results for the same input. Moreover, configuration parameters such as temperature or seed values affect content generation. Therefore, researchers~~ Researchers MUST document in the *paper* which model or tool version they used in their study, along with the date when the experiments were carried out and the configured parameters that affect output generation. Since default values might change over time, researchers SHOULD always report all configuration values, even if they used the defaults. Checksums and fingerprints SHOULD be reported since they identify specific versions and configurations. Depending on the study context, other properties such as the context window size (number of tokens) SHOULD be reported. When using quantized models, researchers SHOULDreport the quantization level (e.g., 4-bit, 8-bit) and method (e.g., GPTQ or AWQ), as different quantization approaches produce different outputs, affecting both output quality and reproducibility. Researchers SHOULD motivate in the *paper* why they selected certain models, versions, and configurations. ~~Potential reasons can be monetary(e.g., no funding to integrate large commercial models), technical(e.g., existing hardware only supports smaller models),~~ Reasons may be monetary, technical, or methodological (e.g., planned comparison to previous work). Depending on the specific study context, additional information regarding the experiment or tool architecture SHOULD be reported~~(see Section)~~.

A common customization approach for existing LLMs is fine-tuning. If a model was fine-tuned, researchers MUST describe the fine-tuning goal (e.g., improving the performance for a specific task), the fine-tuning procedure (e.g., full fine-tuning vs. Low-Rank Adaptation (LoRA), selected hyperparameters, loss function, learning rate, batch size, etc.), and the fine-tuning dataset (e.g., data sources, the preprocessing pipeline, dataset size) in the *paper*. Researchers SHOULD either share the fine-tuning dataset as part of the *supplementary material* or explain in the *paper* why the data cannot be shared (e.g., because

it contains confidential or personal data that could not be anonymized). The same applies to the fine-tuned model weights. Suitable benchmarks and metrics SHOULD be used to compare the base model with the fine-tuned model~~(see Section)~~.

In summary, our recommendation is to report:

(1) Model/tool name and version (MUST in *paper*); (2) All relevant configured parameters that affect output generation (MUST in *paper*); (3) Default values of all available parameters (SHOULD); (4) Checksum/fingerprint of used model version and configuration (SHOULD); (5) Additional properties such as context window size (~~)~~SHOULD); (6) Model quantization level and method, if applicable (SHOULD).

For fine-tuned models, additional recommendations apply:

(1) Fine-tuning goal (MUST in *paper*); (2) Fine-tuning dataset creation and characterization (MUST in *paper*); (3) Fine-tuning parameters and procedure (MUST in *paper*); (4) Fine-tuning dataset and fine-tuned model weights (SHOULD); (5) Validation metrics and benchmarks (SHOULD).

Commercial models (e.g., ~~GPT-4o~~GPT-5) or LLM-based tools (e.g., ChatGPT) might not give researchers access to all required information. Our suggestion is to report what is available and openly acknowledge limitations that hinder reproducibility~~(see also Sections and)~~.

5.2.3 ***Example(s):***

Based on the documentation that OpenAI and Azure provide [101, 112], researchers might, for example, report:

> *"We integrated a* `gpt-4` *model in version* `0125-Preview` *via the Azure OpenAI Service, and configured it with a temperature of 0.7, top_p set to 0.8, a maximum token length of 512, and the seed value* `23487`*. We ran our experiment on 10th January ~~2025' (system fingerprint~~ 2025. The system fingerprint was* `fp_6b68a8204b`*~~)~~.*

Kang et al provide a similar statement in their paper on exploring LLM-based bug reproduction [76]:

> *"We access OpenAI Codex via its closed beta API, using the code-davinci-002 model. For Codex, we set the temperature to 0.7, and the maximum number of tokens to 256."*

Our guidelines additionally suggest to report a checksum/fingerprint and exact dates, but otherwise this example is close to our recommendations.

~~Similar statements can be made for~~ Dhar et al [39] assessed whether LLMs can generate architectural design decisions, detailing the system architecture and the LLM's role within it. They provide information on the fine-tuning approach and datasets, including the source of architectural decision records, preprocessing methods, and data selection criteria.

For self-hosted ~~models. However, when self-hosting~~ models, the *supplementary material* can become a true replication package~~, providing specific~~

~~instructions to reproduce study results~~. For example, for models provisioned using ollama, one can report the specific tag and checksum~~of the model being used~~, e.g., *"llama3.3, tag 70b-instruct-q8_0, checksum d5b5e1b84868."* Given suitable hardware, running the ~~corresponding model in its default configuration~~ model is then as easy as executing ~~one command in the command line (see also Section):~~the following command:

```
ollama run llama3.3:70b-instruct-q8_0
```

~~An example of a study involving fine-tuning is Dhar et al's work [39]. They conducted an exploratory empirical study to assess whether LLMs can generate architectural design decisions. The authors detail the system architecture, including the decision-making framework, the role of the LLM in generating design decisions, and the interaction between the LLM and other components of the system (see Section). The authors provide information on the fine-tuning approach and datasets used for the evaluation, including the source of the architectural decision records, preprocessing methods, and the criteria for data selection.~~

5.2.4 ***Benefits:***

~~Reporting of the information described above~~ Reporting this information is a prerequisite for the verification, reproduction, and replication of LLM-based studies~~under the same or similar conditions. As mentioned before,~~ . While LLMs are inherently non-deterministic~~. However~~, this cannot ~~be an excuse to dismiss the verifiability and reproducibilityof empirical studies involving LLMs~~excuse dismissing reproducibility. Although exact reproducibility is hard to achieve, ~~researchers can do their best to~~ reporting the information outlined in this guideline helps researchers come as close as possible to that ~~gold standard. Part of that effort is reporting the information outlined in this guideline.~~ standard.

5.2.5 ***Challenges:***

Different model providers and modes of operating the models allow for varying degrees of information. For example, OpenAI provides a model version and a system fingerprint describing the backend configuration, which can also influence the output. However, in fact, the fingerprint is intended only to detect changes in the model or its configuration; one cannot go back to a certain fingerprint. As a beta feature, OpenAI lets users set a seed parameter to receive "(mostly) consistent output" [111]. However, the seed value does not allow for full reproducibility and the fingerprint changes frequently. ~~While~~Although, as motivated above, open models significantly simplify re-running experiments, they also come with challenges in terms of reproducibility, as generated outputs can be inconsistent despite setting the temperature to 0 and using a seed value (see GitHub issue for Llama3). Setting the temperature to 0 configures greedy decoding (always selecting the most probable next token), which minimizes

output variability but can degrade quality by producing repetitive text and missing higher-quality responses [62].

Even with a temperature of 0, full determinism is rarely guaranteed: floating-point arithmetic on GPUs causes slight numerical differences that cascade into divergent token selections [177], Sparse Mixture-of-Experts routing amplifies this effect [28], silent backend changes in commercial APIs produce different outputs over time [29], and even self-hosted open models with identical settings do not always yield consistent outputs [8, 147]. Researchers should therefore not treat a temperature of 0 as a guarantee of reproducibility, but as one measure among several, including fixed seed values [111], system fingerprints, and archiving of raw outputs. When a temperature of 0 is chosen primarily for reproducibility, this motivation SHOULDbe stated explicitly, along with an acknowledgment of its potential impact on output quality.

5.2.6 ***Study Types:***

This guideline MUST be followed for all study types for which the researcher has access to (parts of) the model's configuration. They MUST always report the configuration that is visible to them, acknowledging the reproducibility challenges of commercial tools and models that are offered as-a-service. Depending on the specific study type, researchers SHOULD provide additional information on the architecture of a tool they built (see ~~Section~~ *Architecture*), prompts and interactions logs (see ~~Section~~ *Prompts and Logs*), and specific limitations and mitigations (see ~~Section~~ *Limitations and Mitigations*).

For example, when *Studying LLM Usage* by focusing on commercial tools such as ChatGPT or GitHub Copilot, researchers MUST be as specific as possible in describing their study setup. The configured model name, version, and the date when the experiment was conducted MUST always be reported. In those cases, reporting other aspects, such as prompts and interaction logs, is essential.

For *LLMs as Annotators*, *LLMs as Judges*, and *LLMs for Synthesis*, researchers MUSTreport the model configuration used for the respective annotation, judging, or synthesis tasks, including temperature and other sampling parameters that affect output variability. For *LLMs as Subjects*, researchers MUSTreport any persona-related configuration settings and parameters that shape the simulated behavior. For *LLMs for Tools*, researchers MUSTreport the configuration for each model integrated in the tool's architecture, including any model-specific parameter choices. For *Benchmarking LLMs*, researchers MUSTreport the configuration for all benchmarked models to enable fair cross-model comparisons.

5.2.7 ***Advice for Reviewers:***

Missing version, configuration, or parameter information is typically a minor revision request. Before concluding that information is absent, reviewers should check appendices and supplementary materials, as details are sometimes reported

there rather than in the main text. Rejection over missing details is rarely warranted unless the omissions obscure deeper methodological problems.

5.2.8 ***See Also:***

→ Section *Architecture*: system-level architecture beyond the model.
→ Section *Prompts and Logs*: prompt reporting and development.
→ Section *Benchmarks and Metrics*: benchmarks for comparing base and fine-tuned models.
→ Section *Open LLMs*: open models for self-hosted reproducibility.
→ Section *Limitations and Mitigations*: reporting reproducibility limitations.

5.3 Report Tool Architecture Beyond Models (G3)

> ~~*tl;dr:*~~ tl;dr: *Researchers* MUST *describe the full architecture of LLM-based tools that they develop in their* paper. *This includes the role of the LLM, interactions with other components, and the overall system behavior. If autonomous agents are used, researchers* MUST *specify agent roles, reasoning frameworks, and communication flows. Hosting, hardware setup, and latency implications* MUST *be reported. For tools using retrieval or augmentation methods, data sources, integration mechanisms, and update and versioning strategies* MUST *be described. For ensemble architectures, the coordination logic between models* MUST *be explained. Researchers* SHOULD *include architectural diagrams and justify design decisions. Non-disclosed confidential or proprietary components* MUST *be acknowledged as reproducibility limitations.*

5.3.1 ***Rationale:***

LLM-based tools often have *complex software layers* around the model(s) that pre-process data, prepare prompts, filter user requests, or post-process responses. For example, ChatGPT and GitHub Copilot use the same underlying models, but their outputs differ significantly because Copilot automatically adds project context; researchers can also build tools using models directly via APIs. The infrastructure and business logic around the bare model can significantly contribute to the performance of a tool for a given task, for instance, RAG pipelines, output parsers, and retry logic all shape the final output independently of the model itself.

This section addresses the *system-level* aspects of LLM-based tools that researchers develop, complementing Section *Version and Configuration*, which focuses on model-specific details. While standalone LLMs require limited architectural documentation, a more detailed description is required for study setups and tool architectures that integrate LLMs with other components to

create more complex systems. This section provides guidelines for documenting these broader architectures.

5.3.2 ***Recommendations:***

~~Oftentimes, LLM-based tool have *complex software layers* around the model (or models) that pre-processes data, prepares prompts, filters user requests, or post-processes responses. An example is ChatGPT, which allows users to select from different GPT models. GitHub Copilot allows users to select the same models, but its answers may significantly differ from ChatGPT, as GitHub Copilot automatically adds context from the software project in which it is used. Researchers can build their own tools using GPT models directly (*e.g.*, via the OpenAI API). The infrastructure and business logic around the bare model can significantly contribute to the performance of a tool for a given task. Therefore, researchers~~ Researchers MUST clearly describe the tool architecture and what exactly the LLM (or ensemble of LLMs) contributes to the tool or method presented in a research paper. Researchers SHOULDprovide a high-level architectural diagram to improve transparency. To improve clarity, researchers SHOULDexplain design decisions, particularly regarding how the models were hosted and accessed (API-based, self-hosted, etc.) and which retrieval mechanisms were implemented (keyword search, semantic similarity matching, rule-based extraction, etc.). Researchers MUST NOTomit critical architectural details that could affect reproducibility, such as dependencies on proprietary tools that influence tool behavior. Especially for *time-sensitive measurements*, the description of the hosting environment is central, as it can significantly impact the results. Researchers MUSTclarify whether local infrastructure or cloud services were used, including detailed infrastructure specifications and latency considerations.

Agent-Based and Complex Systems: If an LLM is used as a *standalone system*, for example, by sending prompts directly to a GPT-4o model via the OpenAI API without pre-processing the prompts or post-processing the responses, a brief explanation of this approach is usually sufficient. However, if LLMs are integrated into more *complex systems* with pre-processing, retrieval mechanisms, or autonomous agents, researchers MUST provide a detailed description of the system architecture in the *paper*. Aspects to consider are how the LLM interacts with other components such as databases, external APIs, and frameworks. If the LLM is part of an *agent-based system* that autonomously plans ~~, reasons,~~ or executes tasks, researchers MUST describe its exact architecture, including the agents' roles (e.g., planner, executor, coordinator), whether it is a single-agent or multi-agent system, how it interacts with external tools and users, and the reasoning framework used (e.g., chain-of-thought, self-reflection, multi-turn dialogue~~, tool usage).~~

~~Researchers~~). Any agent configuration via context files (e.g., `AGENTS.md`) SHOULD ~~provide a high-level architectural diagram to improve transparency. To improve clarity,~~ also be documented as part of the system architecture (see

Section *Prompts and Logs*for detailed reporting requirements). For agent-based systems that use external tools (e.g., Claude Code and its subagents), researchers ~~explain design decisions, particularly regarding how the models were hosted and accessed (API-based, self-hosted, etc.) and which retrieval mechanisms were implemented (keyword search, semantic similarity matching, rule-based extraction, etc.~~ MUSTdiscuss *agent behavior traceability* by clearly delineating three distinct components: (1) *LLM Input/Output*: Internal deliberation, planning, and interpretation performed by the model. (2) *External tool calls*: Specific invocations of APIs, databases, file systems, or other external services that the agent explicitly triggers (e.g., via the Model Context Protocol (MCP)). (3) *User or system interactions*: Human-in-the-loop feedback, environment responses, or multi-agent communication. Researchers SHOULDreport complete execution traces that show the sequence and causality between these components, including inputs and outputs of all external tool calls. This traceability is essential for determining whether task success is attributable to the LLM's output and tool-calling capabilities, the external tools' functionality, or their interaction patterns. When full tool call logging is not feasible due to privacy or proprietary constraints, researchers SHOULDprovide representative examples or anonymized traces that demonstrate the agent's decision-making process. ~~Researchers omit critical architectural details that could affect reproducibility, such as dependencies on proprietary tools that influence tool behavior. Especially for *time-sensitive measurements*, the previously mentioned description of the hosting environment is central, as it can significantly impact the results. Researchers clarify whether local infrastructure or cloud services were used, including detailed infrastructure specifications and latency considerations.~~

Retrieval, Augmentation, and Ensembles: If *retrieval or augmentation methods* were used (e.g., retrieval-augmented generation (RAG), rule-based retrieval, structured query generation, or hybrid approaches), researchers MUST describe how external data is retrieved, stored, and integrated into the LLM's responses. This includes specifying the type of storage or database used (e.g., vector databases, relational databases, knowledge graphs) and how the retrieved information is selected and used. Stored data used for context augmentation MUST be reported, including details on data preprocessing, versioning, and update frequency. If this data is not confidential, an anonymized snapshot of the data used for context augmentation SHOULD be made available.

For *ensemble models*, in addition to following the *Version and Configuration* guideline for each model, the researchers MUST describe the architecture that connects the models. The *paper* MUST at least contain a high-level description, and details can be reported in the *supplementary material*. Aspects to consider include documenting the logic that determines which model handles which input, the interaction between models, and the architecture for combining outputs (e.g., majority voting, weighted averaging, sequential processing).

5.3.3 ***Example(s):***

~~Some empirical studies involving LLMs in SE have documented the architecture and supplemental data according to our guidelines. In the following, we provide two examples.~~

~~Schäfer et alconducted an empirical evaluation of using~~ Schäfer et al [135] evaluated LLMs for automated unit test generation~~ [135]. The authors provide~~ , providing a comprehensive description of the system architecture ~~, detailing how the LLM is integrated into the software development workflow to analyze codebases and produce the corresponding unit tests. The architecture includes components for~~ including code parsing, prompt formulation, ~~interaction with the LLM~~ LLM interaction, and test suite integration. They also detail the datasets used, including sources, ~~and integration of the generated tests into existing test suites. The paper also elaborates on the datasets utilized for training and evaluating the LLM's performance in unit test generation. It specifies the sourcesof code samples, the~~ selection criteria, and ~~the preprocessing stepsundertaken to prepare the data~~preprocessing steps.

A second example is Yan et al's ~~IVIE~~ *IVIE* tool [175], which integrates LLMs into the VS Code interface. The authors document the tool architecture, detailing the IDE integration, context extraction from code editors, and the formatting pipeline for LLM-generated explanations. This documentation illustrates how architectural components beyond the core LLM affect the overall tool performance and user experience.

5.3.4 ***Benefits:***

~~Usually, researchers implement~~ The software layers around ~~the bare LLMs , using different architectural patterns. These implementations significantly impact the performance of LLM-based tools~~ bare LLMs significantly impact tool performance and hence need ~~to be documented in detail~~detailed documentation. Documenting the architecture and supplemental data of LLM-based systems enhances reproducibility and transparency [94]~~. In empirical software engineering studies, this is essential for~~ , enabling experiment replication, result validation, and ~~benchmarking. A clear documentation of the architecture and the supplemental data that was used enables comparisonand upholds scientific rigor and accountability, fostering reliable and reusable research~~cross-study comparison.

5.3.5 ***Challenges:***

Researchers face challenges in documenting LLM-based architectures, including proprietary APIs and dependencies that restrict disclosure, managing large-scale retrieval databases, and ensuring efficient query execution. They must also balance transparency with data privacy concerns, adapt to the evolving nature of LLM integrations, and, depending on the context, handle the complexity of multi-agent interactions and decision-making logic, all of which can impact reproducibility and system clarity.

5.3.6 ***Study Types:***

This guideline MUST be followed for all studies that involve tools with system-level components beyond bare LLMs, from lightweight wrappers that pre-process user input or post-process model outputs, to systems employing retrieval-augmented methods or complex agent-based architectures.

For *LLMs for Tools*, this guideline is of primary importance: researchers MUSTdescribe the tool's full architecture, including the role of each LLM, interactions with other components, and the overall system behavior. For *Benchmarking LLMs*, researchers SHOULDdescribe the evaluation harness and infrastructure if it goes beyond bare model API calls (e.g., custom sandboxing, orchestration layers, or post-processing pipelines). For *Studying LLM Usage*, researchers SHOULDdescribe the tool architecture of the studied tool to the extent that it is accessible, as architectural details may influence observed usage patterns. For *LLMs as Annotators*, *LLMs as Judges*, *LLMs for Synthesis*, and *LLMs as Subjects*, if the research setup involves a custom pipeline (e.g., retrieval-augmented generation for annotation or chained prompts for synthesis), the architecture SHOULDalso be reported.

5.3.7 ***Advice for Reviewers:***

As with other guidelines, missing architectural information is typically a minor revision request unless so much is missing that methodological rigor cannot be assessed. Reviewers may ask authors to move key details from supplements into the paper body to ensure the main text is self-contained.

Regarding proprietary or confidential components, three principles apply: (1) empirically evaluating an opaque artifact is a valid scientific contribution; (2) the less available and inspectable the artifact, the weaker the contribution; (3) *scientific instruments* including tools, metrics, scales, and experimental materials, must be fully disclosed, even when the object under study cannot be.

5.4 Report Prompts, their Development, and Interaction Logs (G4)

> ~~*tl;dr:*~~ tl;dr: *Researchers MUST publish all prompts, including their struc-*

> *ture, content, formatting, and dynamic components. If full prompt disclosure is not feasible, for example, due to privacy or confidentiality concerns, summaries or examples* SHOULD *be provided. Prompt development strategies (e.g., zero-shot, few-shot), rationale, and selection process* MUST *be described. When prompts are long or complex, input handling and token optimization strategies* MUST *be documented. For dynamically generated or user-authored prompts, generation and collection processes* MUST *be reported. Prompt reuse across models and configurations* MUST *be specified. Researchers* SHOULD *report prompt revisions and pilot testing insights. For agentic systems, developed plans* SHOULD*be reported and interaction logs* SHOULD*be generalized to include human-agent interaction traces. For studies using AI coding agents, all context files used to configure agent behavior* MUST*be reported as part of the* supplementary material. *To address model non-determinism and ensure reproducibility, especially when targeting SaaS-based commercial tools, full interaction logs (prompts and responses)* SHOULD *be included if privacy and confidentiality can be ensured.*

5.4.1 ***Rationale:***

Prompts are critical for any study involving LLMs [142]. Depending on the task, ~~prompts may include various types of content, including~~ they may include instructions, context ~~, and input data, and output indicators. For SE tasks, the context can include~~ (e.g., source code, execution traces, error messages~~, and other forms of natural language content. The output can be unstructured text or a structured format~~), input data, and output indicators, with outputs ranging from unstructured text to structured formats such as JSON.[2] Prompts significantly influence the quality of the output of a model, and understanding how exactly they were formatted and integrated into an LLM-based study is essential to ensure transparency, verifiability, and reproducibility. Sclar et al question the methodological validity of comparing models with "*an arbitrarily chosen, fixed prompt format*", because their research has shown that the performance of different prompt formats only weakly correlates between different models [142]. We remind the reader that these guidelines do not apply when LLMs are used solely for language polishing, paraphrasing, translation, tone or style adaptation, or layout edits (see *Scope*). This implies that our guidelines do not recommend that researcher disclose the prompts they have used for such tasks.

5.4.2 ***Recommendations:***

Prompt Reporting: Researchers MUST report ~~the~~ all prompts that were used for an empirical study, including all instructions, context, input data, and output indicators. The only exception to this is if transcripts might identify anonymous participants or reveal personal or confidential information, for example,

[2] `https://www.promptingguide.ai/introduction/elements`

when working with industry partners. Prompts can be reported using a structured template that contains placeholders for dynamically added content~~(see Section for an example).~~

~~Moreover, specifying the exact output format of the promptsis crucial. For example, when using code snippets , researchers should specify whether they were enclosed in~~ . Moreover, researchers SHOULDspecify the exact formatting of prompts, including how code snippets were enclosed (e.g., markdown-style code blocks~~(indented by four space characters, enclosed in triple backticks, etc.), if line breaks and whitespace were~~ , triple backticks), whether whitespace was preserved, and ~~whether additional annotations such as comments were included. Similarly, for~~ how other artifacts such as error messages ~~, stack traces , researchers should explain how these~~ and stack traces were presented.

Prompt Development: Researchers MUST also explain in the *paper* how they developed the prompts and why they decided to follow certain prompting strategies. If prompts from the early phases of a research project are unavailable, they MUST at least summarize the prompt evolution. However, given that prompts can be stored as plain text, there are established SE techniques such as version control systems that can be used to collect the required provenance information.

Prompt development is often iterative, involving collaboration between human researchers and AI tools. Researchers SHOULD report any instances in which LLMs were used to suggest prompt refinements ~~, as well as~~ and how these suggestions were incorporated. ~~Furthermore, prompts may need to be revised~~ Prompts may need revision in response to failure cases~~where the model produced incorrect or incomplete outputs. Pilot testing and prompt evaluation are~~ , and pilot testing is vital to ensure ~~that prompts yield~~ reliable results. If such testing was conducted, researchers SHOULD summarize key insights, including how different prompt variations affected output quality and which criteria were used to finalize the prompt design. A prompt changelog can ~~help track and report the evolutionof prompts throughout a research project~~track prompt evolution, including key revisions ~~,~~ and reasons for changes ~~, and versioning~~ (e.g., v1.0: initial prompt; ~~v1.2: added output formatting;~~ v2.0: incorporated examples of ideal responses). Since prompt effectiveness varies between models and model versions, researchers MUST make clear which prompts were used for which models in which versions and with which configuration~~(see Section)~~.

Prompting Strategy and Input Handling: Researchers MUST specify whether zero-shot, one-shot, or few-shot prompting was used. For few-shot prompts, the examples provided to the model SHOULD be clearly outlined, along with the rationale for selecting them. If multiple versions of a prompt were tested, researchers SHOULD describe how these variations were evaluated and how the final design was chosen.

When dealing with extensive or complex prompt context, researchers MUST describe the strategies they used to handle input length constraints. Ap-

proaches might include truncating, summarizing, or splitting prompts into multiple parts. Token optimization measures, such as simplifying code formatting or removing unnecessary comments, MUST also be documented if applied.

Dynamic and User-Authored Prompts: In cases where prompts are generated dynamically, such as through preprocessing, template structures, or retrieval-augmented generation (RAG), the process MUST be thoroughly documented. This includes explaining any automated algorithms or rules that influenced prompt generation. For studies involving human participants who create or modify prompts, researchers MUST describe how these prompts were collected and analyzed.

~~To ensure full~~

Supplementary Material and Anonymization: To ensure complete reproducibility, researchers MUST make all prompts and prompt variations publicly available as part of their *supplementary material*. If the complete set of prompts cannot be included in the *paper*, researchers SHOULD provide summaries and representative examples. This also applies if complete disclosure is not possible due to privacy or confidentiality concerns. For prompts containing sensitive information, researchers MUST: (1) anonymize personal identifiers, (ii) replace proprietary code with placeholders, and (iii) clearly highlight modified sections.

Interaction Logs: When trying to verify results, even with the exact same prompts, decoding strategies, and parameters, LLMs can still behave non-deterministically. Non-determinism can arise from batching, input preprocessing, and floating point arithmetic on GPUs [28]. Thus, in order to enable other researchers to verify the conclusions that researchers have drawn from LLM interactions, researchers SHOULD report the full interaction logs (prompts and responses) as part of their *supplementary material*. Reporting this is especially important for studies targeting commercial software-as-a-service (SaaS) solutions such as ChatGPT. The rationale for this is similar to the rationale for reporting interview transcripts in qualitative research. ~~In both cases, it is important to document the entire interaction between the interviewer and the participant.~~ Just as a human participant might give different answers to the same question asked two months apart, the responses from tools such as ChatGPT can also vary over time~~(see also Section). Therefore, keeping a record of the actual conversation is crucial for accuracy and context and shows depth of engagement for transparency~~.

Agentic Systems: For agentic systems that autonomously plan and execute tasks, the developed plans SHOULDbe reported if available, as they document the sequence of actions the system chose to pursue and the intermediate goals it set. Interaction logs SHOULDbe generalized to include full interaction traces between humans and agentic systems, capturing human-in-the-loop feedback, approval or rejection decisions, and iterative refinements. These

traces complement agent behavior traceability requirements and support the evaluation of agentic tool outputs.

Context Files: AI coding agents such as *Claude Code* can be configured using version-controlled Markdown files (e.g., `CLAUDE.md` or `AGENTS.md`) that provide persistent, project-specific instructions to the agent [104]. These *context files* are automatically injected into agent prompts and can significantly influence agent behavior by specifying coding conventions, architectural constraints, tool preferences, and task-specific instructions. Researchers MUSTreport all context files used to configure AI agent behavior as part of their *supplementary material*, as these files are a form of prompt artifact that shapes agent behavior similarly to system prompts.

5.4.3 ***Example(s):***

A paper by Anandayuvaraj et al [7] is a good example of making prompts available online. In that paper, the authors analyze software failures reported in news articles and use prompting to automate tasks such as filtering relevant articles, merging reports, and extracting detailed failure information. Their online appendix contains all the prompts used in the study, providing transparency and supporting reproducibility.

Liang et al's paper is ~~a~~ another good example of comprehensive prompt reporting. The authors make the exact prompts available in their *supplementary material* on Figshare, including details such as code blocks being enclosed in triple backticks. The *paper* thoroughly explains the rationale behind the prompt design and the data output format. It also includes an overview figure and two concrete examples, enabling transparency and reproducibility while keeping the main text concise.

An example of reporting full interaction logs is the study by Ronanki et al [130], for which they reported the full answers of ChatGPT and uploaded them to Zenodo.

5.4.4 ***Benefits:***

~~As motivated above, providing detailed documentation of the prompts improves the~~ Detailed prompt documentation improves verifiability, reproducibility, and comparability of LLM-based studies~~. It allows~~ , allowing other researchers to replicate ~~the study under similar conditions~~studies, refine prompts~~based on documented improvements~~, and evaluate how different ~~types of content (e.g., source code vs. execution traces)~~ content types and formatting choices influence LLM behavior. ~~This transparency also enables a better understanding of how formatting, prompt length, and structure impact results across various studies.~~

~~An advantage of reporting full interaction logs is that, while for human participants conversations~~Unlike human participant conversations, which often cannot be reported due to confidentiality, LLM ~~conversations can . Detailed~~

~~logs enable future reproduction /replication studiesto compare results using the same prompts. This could be valuable for tracking changes in LLM responses~~ interaction logs can be shared. This enables reproduction studies, tracking of response changes over time or across ~~different versions of the model . It would also enable secondary research to study how consistent LLM responses are over model versions and identify any variations in its performance~~ model versions, and secondary research on LLM consistency for specific SE tasks.

5.4.5 ***Challenges:***

One challenge is the complexity of prompts that combine multiple components, such as code, error messages, and explanatory text. Formatting differences (e.g., Markdown vs. plain text) can affect how LLMs interpret input. Additionally, prompt length constraints may require careful context management, particularly for tasks involving extensive artifacts such as large codebases. Privacy and confidentiality concerns can hinder prompt sharing, especially when sensitive data is involved.

Not all systems allow the reporting of complete (system) prompts and interaction logs with ease. This hinders transparency and verifiability. ~~In addition to our recommendation to , we~~ We also recommend exploring open-source tools such as ~~Continue [41]~~ *OpenCode* [113]. For commercial tools, researchers MUST report all available information and acknowledge unknown aspects as limitations~~(see Section)~~. Understanding suggestions of commercial tools such as ~~GitHub Copilot~~ *GitHub Copilot* might require recreating the exact state of the codebase at the time the suggestion was made, a challenging context to report. One solution could be to use version control to capture the exact state of the codebase when a recommendation was made, keeping track of the files that were automatically added as context.

5.4.6 ***Study Types:***

~~Reporting requirements vary depending on the study type. For the study type~~ Researchers MUSTreport the complete prompts that were used, including all instructions, context, and input data. This applies across all study types, but reporting requirements and specific focus areas vary.

For *LLMs for Tools*, researchers MUST explain how prompts were generated and structured within the tool. ~~When~~ For *Studying LLM Usage*, especially for controlled experiments, exact prompts MUST be reported for all conditions. For observational studies, especially the ones targeting commercial tools, researchers MUST report the full ~~interactions~~ interaction logs except for when transcripts might identify anonymous participants or reveal personal or confidential information. As mentioned before, if the complete interaction logs cannot be shared, e.g., because they contain confidential information, the prompts and responses MUST at least be summarized and described in the *paper*.

~~Researchers report the complete prompts that were used, including all instructions, context, and input data. This applies across all study types, but~~

~~specific focus areas vary.~~ For *LLMs as Annotators*, researchers MUST document any predefined coding guides or instructions included in prompts, as these influence how the model labels artifacts. For *LLMs as Judges*, researchers MUST ~~include~~ report the evaluation criteria, scales, and examples embedded in the prompt to ensure a consistent interpretation. For *LLMs for Synthesis* tasks (e.g., summarization, aggregation), researchers MUST document the sequence of prompts used to generate and refine outputs, including follow-ups and clarification queries. For *LLMs as Subjects* (e.g., simulating human participants), researchers MUST report any role-playing instructions, constraints, or personas used to guide LLM behavior. For *Benchmarking LLMs* studies using predefined prompts (e.g., *HumanEval*, ~~*CruxEval*~~*SWE-Bench*), researchers MUST specify the benchmark version and any modifications made to the prompts or evaluation setup. If prompt tuning, retrieval-augmented generation (RAG), or other methods were used to adapt prompts, researchers MUST disclose and justify those changes, and SHOULD make the relevant code publicly available.

5.4.7 ***Advice for Reviewers:***

As with other guidelines, missing prompt information is typically a minor revision request unless so much is missing that methodological rigor cannot be assessed. The most challenging aspect is the description of prompt development, which is inherently iterative and creative. Reviewers should *not* expect justification of each word choice or post-hoc rationalized accounts of prompt generation. Instead, reviewers should focus on whether (1) a new research team could *use* (not reproduce) exactly the same prompts in exactly the same way; and (2) potential biases or validity issues are transparent.

5.4.8 ***See Also:***

→ Section *Version and Configuration*: model versions and configuration for prompt-model mapping.
→ Section *Architecture*: agent behavior traceability requirements.
→ Section *Human Validation*: evaluation of agentic tool outputs.
→ Section *Open LLMs*: open-source tools for transparent prompt logging.
→ Section *Limitations and Mitigations*: reporting limitations of commercial tool transparency.

5.5 Use Human Validation for LLM Outputs (G5)

> ~~*tl;dr:*~~ tl;dr: *If assessing the quality of generated artifacts is important and no*

> *reference datasets or suitable comparison metrics exist, researchers* SHOULD *use human validation for LLM outputs. If they do, they* MUST *define the measured construct (e.g., usability, maintainability) and describe the measurement instrument in the* paper. *Researchers* SHOULD *consider human validation early in the study design (not as an afterthought), build on established reference models for human-LLM comparison, and share their instruments as* supplementary material. *When aggregating LLM judgments, methods and rationale* SHOULD *be reported and inter-rater agreement* SHOULD *be assessed. Confounding factors* SHOULD *be controlled for, and power analysis* SHOULD *be conducted to ensure statistical robustness.*

5.5.1 ***Rationale:***

Automated metrics alone cannot fully capture the validity of subjective or complex constructs. When LLMs are used to automate tasks that involve human judgment, their outputs must be validated against human assessments to ensure that the results are not only reliable but also valid.

5.5.2 ***Recommendations:***

~~Although, in principle, LLMs can automate many tasks in research and software development , which have traditionally been~~ When LLMs are used to automate research or software development tasks previously performed by humans, it is ~~often~~ essential to assess the ~~outcome quality to determine whether the automation was successful. If, for a certain task, there are no~~ LLM's performance. When existing reference datasets or ~~suitable comparison metrics~~ comparison metrics cannot fully capture the target qualities relevant in the study context, researchers SHOULD rely on human judgment to validate the LLM outputs. ~~Human judgment is particularly important if metrics and existing datasets alone cannot fully capture the target qualities relevant in the study context.~~

Study Design Considerations: Integrating human participants in the study design will require additional considerations, including a recruitment strategy, annotation guidelines, training sessions, or ethical approvals. Therefore, researchers SHOULD consider human validation early in the study design, not as an afterthought. ~~In any way, they~~ Authors MUST clearly define the constructs that the human and LLM annotators evaluate [122]. When designing custom instruments to assess LLM output (e.g., questionnaires, scales), researchers MUST share their instruments, for example in the *supplementary material*.

~~Validating the output of an LLM involve the aggregation of inputs~~

Subjective Judgment and Agreement: When the judgment is subjective (i.e., depends on the judge's values or theories), the LLM output SHOULDbe evaluated against judgments aggregated from multiple human judges~~. In these cases,~~ , and researchers SHOULD clearly describe their aggregation method and ~~document their reasoning. When multiple humans are annotating the same artifact,~~

~~researchers validate the agreement between multiple validators with~~ reasoning. The preferred method is to randomly order the objects requiring judgment and put them in groups small enough to judge in one to three hours. Two or three human experts review the first group together, simultaneously judging, discussing, and creating a set of decision rules documenting their reasoning. Then, the experts iterate among rounds of:

1. independently rating one group of objects,
2. calculating inter-rater ~~reliability measures such as *Cohen's Kappa* or *Krippendorff's Alpha*. When human validation is used, additional confounding~~ agreement (IRA) or reliability (IRR),
3. meeting to discuss disagreements and reach consensus,
4. updating the decision rules so the disagreement will not recur.

The goal is to reach a target IRA or IRR (e.g., Krippendorff's $\alpha > 0.8$), indicating sufficient decision rules. Once this target is reached and sustained for two to three groups, it is permissible to continue with a single rater. Researchers SHOULDreport measures of IRA or IRR, preferably broken down by round.

Confounding factors SHOULD be ~~controlled for,~~ discussed and, where feasible, controlled for (e.g., by categorizing participants according to their level of experience or expertise). Where applicable, researchers may perform a power analysis [34, 42] to estimate the required sample size, ensuring sufficient statistical power in their experimental design. However, we also note that to our knowledge, there is limited established guidance specific to determining sample sizes for LLM-human comparisons. In other fields, 100 comparisons are often made without further explanation [155].

Researchers SHOULD use established reference models to compare humans with LLMs. For example, ~~the reference model of~~ Schneider et al [137] ~~provides researchers with an overview of the~~ outline design considerations for studies comparing LLMs with humans. If studies ~~involve the annotation of software artifacts, and the goal is to automate the annotation process using~~ aim to automate annotating software artifacts using an LLM, researchers SHOULD follow systematic approaches to decide whether and how human annotators can be replaced ~~. For example, Ahmed et al [4] suggest a method that involves using~~ (e.g., showing that a jury of three LLMs ~~with 3 to 4 few-shot examples rated by humans, where the~~ exhibit model-to-model agreement ~~on all samples is determined using *Krippendorff's Alpha*. If the agreement is high (alpha *gt* 0.5), a human rating can be replaced with an LLM-generated one. In cases of low model-to-model agreement (alpha $\leq$ 0.5) , they then evaluate the prediction confidence of the model, selectively replacing annotations where the model confidence is high ($\geq$ 0.8)~~comparable to trained human annotators, such as Krippendorff's $\alpha > 0.8$). Model-to-model agreement should not be held to a lower standard than human-to-human agreement, as low thresholds may simply reflect shared model biases rather than annotation quality.

Agentic Tools Besides generating and modifying content, agentic software development tools such as *Claude Code* can autonomously call command-line tools or pull in additional information from MCP servers. For such tools, it makes sense to separate textual output (e.g., file changes) from output related to tool execution. When evaluating agentic tools, researchers SHOULDassess the feedback that users provided for proposed changes, report statistics on how frequently they accepted the content, and how they modified it. This agentic human-in-the-loop interaction approach is essentially a built-in human validation, even though the degrees of freedom are larger than in more traditional experiments that use LLMs directly.

5.5.3 ***Example(s):***

Ahmed et al [4] ~~evaluated how human annotations can be replaced by LLM generated labels. In their study, they explicitly report the calculated agreement metrics between the models and between humans. For example, they wrote that "~~proposed a systematic method for deciding whether LLMs can replace human annotators on a given task, using model-to-model agreement as an initial screening criterion (with a threshold of $\alpha > 0.5$) and model confidence for sample-level decisions. However, their threshold is well below the levels generally considered acceptable for inter-rater agreement. Krippendorff recommends discarding data with $\alpha < 0.667$, considers $0.667 \leq \alpha < 0.8$ sufficient only for tentative conclusions, and requires $\alpha \geq 0.8$ for reliable data [82]. Researchers adopting similar screening approaches should apply thresholds consistent with these established standards. Notably, while three LLMs exhibited higher inter-model agreement than human annotators on some tasks, human-model agreement remained low on others. This illustrates that high model-model ~~agreement is high, for all criteria, especially for the three large models (GPT-4, Gemini, and Claude). Table I in their study indicates that the mean Krippendorff's α is 0.68-0.76. Second, we see that human-model and human-human agreements are in similar ranges, 0.24-0.40 and 0.21-0.48 for the first three categories. "~~reliability does not guarantee alignment with human judgment, reinforcing the need for human validation before assuming that LLM annotations are valid. Moreover, a high model-model agreement could merely indicate that the models share systematic biases and hence reliably agree on the wrong answer.

~~Xue et al [174] conducted a controlled experiment in which they evaluated the impact of ChatGPTon the performance and perceptions of students in an introductory programming course. They used multiple measures to judge the impact of LLM from the human point of view. In their study, they recorded students' screens, evaluated their answers for the given tasks, and distributed a post-study survey to collect their opinions~~Hymel and Johnson [69] [69] evaluated ChatGPT's capability to generate requirements documents by comparing an LLM-generated and a human-generated document based on the same business use case. Domain experts reviewed both documents and attempted to distinguish their origin.

5.5.4 ***Benefits:***

~~For empirical studies involving LLMs , human validation helps ensure the reliability of study results, as LLMs can produce incorrect or biased outputs. For example, in natural language processing tasks, a large-scale study has shown that LLMs have a significant variation in their results, which limits their reliability as a direct substitute for human judges [19]. Moreover, LLMs do not match human annotation in labeling tasks for natural language inference, position detection, semantic change, and hate speech detection [165].~~

~~Incorporating human judgment into the evaluation process adds a layer of quality control and increases~~ Validating LLMs against human judgments builds confidence in the LLM's accuracy and validity, increasing the trustworthiness of ~~the study~~ findings, especially when ~~explicitly reporting inter-rater reliability metrics~~ IRA or IRR metrics are reported [79]. ~~Incorporating feedback from individuals in the target population strengthens external validity by grounding study findings in real-world usage scenarios, which may positively impact the transfer of study results to practice. Researchers might uncover additional opportunities to further~~ Furthermore, incorporating feedback from human judges may help to improve the LLM or LLM-based tool based on the reported experiences.

5.5.5 ***Challenges:***

~~Measurement through human validation can be challenging . Ensuring that the operationalization of a desired construct and the method of measuring it are appropriate~~ Assessing a construct like LLM performance is challenging because ensuring that a construct is defined well and operationalized using an appropriate measurement model requires a deep understanding of (1) the construct, (2) construct validity in general, and (3) ~~systematic approaches for designing measurement instruments [146]. Human judgment is subjective, which can lead to variability between judgments~~ instrumentation [124, 146]. Comparing an LLM to human judges is typically slower and more expensive than machine-generated measures. More fundamentally, neither human judgment nor machine-generated measures provides an objective ground truth against which LLM accuracy can be firmly determined.

Human judgments exhibit variability due to differences in experience, expertise, interpretations, and ~~biases among evaluators~~personal biases [98]. ~~For example, Hicks et al found that [61]. Such biases can be mitigated with a careful selection and combination of multiple judges, but cannot be completely controlled for. Recruiting participants as human validators will always incur additional resources compared to machine-generated measures.~~ When diverse humans rate items reliably given clear decision rules, we assume that reliability implies validity, but it does not. Measuring reliability is *much* easier than measuring validity, and often the best researchers can do is argue conceptually for why their judges, decision rules, and constructs *should* produce valid ratings.

Researchers must weigh the cost and time ~~investment incurred by the recruitment process against the potential benefits for the validity of their study results~~need for human validation against the expected corresponding improvement in validity over machine-generated measures.

5.5.6 ***Study Types:***

~~For~~ This guideline applies to all study types, although the need for human validation varies. For *LLMs as Annotators*, researchers SHOULDvalidate LLM-generated annotations against human annotators to assess labeling quality and identify systematic biases. When using *LLMs as Judges*, researchers SHOULDco-create initial rating criteria with humans and validate a sample of LLM judgments against human expert assessments. For *LLMs for Synthesis*, researchers SHOULDemploy human oversight to verify that qualitative interpretations and synthesized outputs faithfully represent the underlying data. For *LLMs as Subjects*, researchers SHOULDvalidate simulated responses against real human data to assess the fidelity of the simulation. For *Studying LLM Usage*, researchers SHOULD carefully reflect on the validity of their evaluation criteria ~~. In studies where there are arguably objective evaluation criteria (e.g., in~~ and validate subjective assessments with human experts. In developing *LLMs for Tools*, human validation of the LLM output is critical when the output is supposed to match human expectations. For *Benchmarking LLMs* ~~)~~, there is ~~little~~ less need for human validation ~~, since the benchmark has (hopefully) been validated . Where criteria are more unclear~~ when using extensively validated and widely-used benchmarks, but researchers SHOULDemploy human validation when creating or adapting new benchmarks.

5.5.7 ***Advice for Reviewers:***

Human validation may be the most challenging of our guidelines to assess because it often requires evaluating conceptual arguments. If LLM output is validated only by comparison with other LLMs, reviewers should look for *quantitative empirical evidence* that such comparison is reliable *and* valid. High inter-model agreement alone is insufficient, as reliability does not imply validity. Similarly, reviewers should expect evidence that any employed benchmarks are reliable and valid. Absent such evidence, human validation is warranted. A single human judge is appropriate only when judgments depend on widely accepted theories and involve limited value conflict (e.g., ~~, , or), human validation is important, and this guideline should be followed. When using , it is common to start with humans who co-create intial rating criteria. In developing , human validation of the LLM output is criticalwhen the output is supposed to match human expectations.~~

tagging method names containing abbreviations). For multiple judges, reviewers should expect IRA/IRR improvement techniques as described in the recommendations above (experienced raters, organized rounds, consensus meetings, updated decision rules). Low IRA or IRR (e.g., Krippendorff's $\alpha < 0.8$) without these

techniques is a concern. Conversely, if authors have followed best practices and still obtained mediocre results (e.g., $0.66 < \alpha < 0.8$), this should be noted as a limitation. Beyond reliability, reviewers should expect authors to explain conceptually why their human judgments should be valid, considering construct definitions, decision rules, and judge expertise. As with other guidelines, missing information is typically a revision request. Absent judge instructions, instruments, decision rules, or construct definitions may prevent assessment of rigor and validity, while missing recruitment details are less critical. Clarification requests about construct definitions are routine and should not alone warrant rejection.

5.6 Use an Open LLM as a Baseline (G6)

> ~~*tl;dr:*~~ tl;dr*: Researchers* SHOULD *include an open LLM as a baseline when using commercial models and report inter-model agreement. By open LLM, we mean a model that everyone with the required hardware can deploy and operate. Such models are usually "open weight." A full replication package with step-by-step instructions* SHOULD *be provided as part of the* supplementary material.

5.6.1 ***Rationale:***

Reproducibility depends on access to the model under study. When research relies exclusively on proprietary models, other researchers cannot independently verify or build upon the findings. Including an open LLM as a baseline ensures that at least part of the study can be fully replicated.

5.6.2 ***Recommendations:***

Empirical studies using LLMs in SE, especially those that target commercial tools or models, SHOULD incorporate an open LLM as a baseline and report established metrics for inter-model agreement~~(see Section)~~. We acknowledge that including an open LLM baseline might not always be possible, for example, if the study involves human participants, and letting them work on the tasks using two different models might not be feasible. Using an open model as a baseline is also not necessary if the use of the LLM is tangential to the study goal.

Open models allow other researchers to verify research results and build upon them, even without access to commercial models. A comparison of commercial and open models also allows researchers to contextualize model performance. Researchers SHOULD provide a complete replication package as part of their *supplementary material*, including clear step-by-step instructions on how to verify and reproduce the results reported in the paper.

Open LLMs are available on platforms such as *Hugging Face* ~~. Depending on their size and the available compute power, open LLMs~~ and can be hosted ~~on a local computer or server~~ locally using frameworks such as *Ollama* or *LM Studio*~~. They can also be run on cloud-based~~ , or on cloud services such as *Together AI*~~or on large hyperscalers such as~~ , AWS, Azure, ~~Alibaba Cloud,~~ and Google Cloud.

The term "open" can have different meanings in the context of LLMs. Widder et al discuss three types of openness ~~:~~ (transparency, reusability, and extensibility~~ [166]. They also discuss what openness in AI~~) and what openness can and cannot provide~~. Moreover, the~~ [166]. The *Open Source Initiative* (OSI)~~[110] provides a definition of~~ [110] defines open-source AI ~~that serves as a useful framework for evaluating the openness of AI models. In simple terms, according to OSI, open-source AI means that one has~~ as having access to everything needed to ~~use the AI, which includes that it is possible to~~ understand, modify, share, retrain, and recreate ~~it~~the model.

Researchers can also explore open-source tools such as Continue, Cline, and opencode as alternatives to commercial tools like GitHub Copilot and Claude Code, enabling instrumentation, architectural transparency, and detailed telemetry collection.

5.6.3 ***Example(s):***

~~Numerous~~ An increasing number of studies have adopted open LLMs as baseline models. For example, Wang et al evaluated seven advanced LLMs, six of which were open-source, testing 145 API mappings drawn from eight popular Python libraries across 28,125 completion prompts aimed at detecting deprecated API usage in code completion [162]. Moumoula et al compared four LLMs on a cross-language code clone detection task [107]. Three evaluated models were open-source. Gonçalves et al fine-tuned the open LLM *LLaMA* 3.2 on a refined version of the *DiverseVul* dataset to benchmark vulnerability detection performance [53]. ~~Many papers have used CodeBERT as an LLM, which is a bimodal (code + NL) Transformer~~ *CodeBERT*, a bimodal transformer pre-trained by Microsoft Research~~and released under the MIT license~~, has been widely used as an open baseline [25, 148, 170, 176]~~. Its~~ , with model weights, source code, and data-processing scripts ~~are all openly~~ published on GitHub [100].

5.6.4 ***Benefits:***

Using a true open LLM as a baseline improves the reproducibility of scientific research by providing ~~full~~ access to model architectures ~~, training data,~~ and parameter settings~~(see Section),~~ , and ideally training data, thereby allowing independent reconstruction and verification of experimental results. Moreover, by adopting an open-source baseline, researchers can directly compare novel methods against established performance metrics without the variability introduced by proprietary systems~~(see also Section)~~. The transparent nature of

these models allows for detailed inspection of data processing pipelines and decision-making routines, which is essential for identifying potential sources of bias and delineating model limitations~~(see Section)~~. Furthermore, unlike closed-source alternatives, which can be withdrawn or altered without notice, open LLMs ensure long-term accessibility and stability, preserving critical resources for future studies. Finally, the licensing requirements associated with open-source implementations lower financial barriers, making advanced language models attainable for research groups operating under constrained budgets.

5.6.5 ***Challenges:***

Open-source LLMs face several notable challenges~~(see also Section)~~. First, they often lag behind the most advanced proprietary ~~systems~~ "frontier" models in common benchmarks, making it difficult for researchers to demonstrate clear improvements when evaluating new methods using open LLMs alone~~(see Section)~~. Additionally, deploying and experimenting with these models typically requires substantial hardware resources, in particular high-performance GPUs, which may be beyond reach for many academic groups~~(see Section).~~

. The notion of "openness" ~~itself~~ remains in flux: ~~although numerous models are described as open, many release only the~~ many models release only trained weights without ~~disclosing the underlying~~ training data or methodological details (~~see Section), a practice sometimes referred to as~~ "open weight" openness~~[51]. To address this gap, in our recommendations, we have referenced the open-source AI definition proposed by the OSI as an initial framework for what constitutes genuinely open-source AI software~~) [51], which is why we reference the OSI definition in our recommendations [110].

Finally, unlike ~~managed cloud~~ APIs provided by proprietary vendors ~~,~~ (e.g., the OpenAI API), installing, configuring, and fine-tuning open-source models can be technically demanding. Documentation is often sparse or fragmented, placing a high barrier to entry for researchers without specialized engineering support~~(see also Section)~~.

5.6.6 ***Study Types:***

This guideline applies primarily to study types in which the researcher controls which LLM is used. In formal benchmarking studies and controlled experiments (see *Benchmarking LLMs*), an open LLM MUSTbe one of the models under evaluation. When evaluating *LLMs for Tools*, researchers SHOULD use an open LLM as a baseline whenever it is technically feasible; if integration proves too complex, they SHOULD report the initial benchmarking results of open models~~(see Section). In formal benchmarking studies and controlled experiments (see Section), an open LLM be one of the models under evaluation. For observational studies in which~~. For *LLMs as Annotators*and *LLMs as Judges*, researchers SHOULDcompare annotation or judgment quality from open vs. commercial models to assess the extent to which results depend on a specific

proprietary model. For *LLMs for Synthesis*, researchers SHOULDcompare synthesis results from open and commercial models to evaluate robustness of the findings. For *Studying LLM Usage*, using an open LLM ~~is impossible~~as a baseline is often not feasible when the study observes participants using specific commercial tools; in such cases, investigators SHOULD explicitly acknowledge its absence and discuss how this limitation might affect their conclusions~~(see Section). Finally, when using , where an LLM serves to explore qualitative data or contrast alternative interpretations, researchers report results~~ . For *LLMs as Subjects*, using an open LLM as a baseline may similarly be impractical if the study design requires the capabilities of a specific model; researchers SHOULDacknowledge this limitation when applicable.

5.6.7 ***Advice for Reviewers:***

Reviewers should distinguish between LLM use that is central to the research (e.g., building LLM-driven SE tools, using LLMs for synthesis) and use that is tangential (e.g., generating recruiting materials). An open LLM baseline is expected only when LLM use is central; otherwise, its absence need not be justified. When an open baseline is expected, reviewers should look for either the use of an open LLM ~~.~~or a convincing argument for why it is impractical. If authors claim openness, some justification for that characterization is appropriate. Where practical, reviewers should examine whether the replication package contains sufficiently detailed instructions. Reviewers should *not* penalize studies for performance differences between open and proprietary models, as this is beyond the authors' control.

5.6.8 ***See Also:***

→ Section *Version and Configuration*: model version, configuration, and parameter settings.
→ Section *Architecture*: hardware and hosting requirements for open models.

→ Section *Benchmarks and Metrics*: metrics for inter-model agreement and benchmarking.
→ Section *Limitations and Mitigations*: reporting limitations when open baselines are absent.

5.7 Use Suitable Baselines, Benchmarks, and Metrics (G7)

> ~~*tl;dr:*~~ tl;dr: *Researchers* MUST *justify all benchmark and metric choices in*

> *the* paper *and* SHOULD *summarize benchmark structure, task types, and limitations. Where possible, traditional (non-LLM) baselines* SHOULD *be used for comparison. Researchers* MUST *explain why the selected metrics are suitable for the specific study. They* SHOULD *report established metrics to make study results comparable, but can report additional metrics that they consider appropriate. Due to the inherent non-determinism of LLMs, experiments* SHOULD *be repeated; the result distribution* SHOULD *then be reported using descriptive statistics.*

5.7.1 ***Rationale:***

Meaningful evaluation requires well-understood, valid measurement instruments. Without justified benchmarks and metrics, claims about LLM performance lack the rigor needed for scientific comparison and cumulative progress. Benchmarks, baselines, and metrics play an important role in assessing the effectiveness of LLMs ~~or~~ and LLM-based tools. ~~Benchmarks are model- and tool-independent standardized tests used to assess the performance of LLMs on specific tasks such as code summarization or code generation. A benchmark consists of multiple standardized test cases, each with at least one task and a corresponding expected result. Metrics are used to quantify performance on benchmark tasks, enabling a comparison~~A *benchmark* is a "standard tool for the competitive evaluation and comparison of competing systems or components according to specific characteristics, such as performance, dependability, or security" [80]. Meanwhile, "a *metric* is a method, algorithm, or procedure for assigning one or more numbers to a phenomenon" [124]. A baseline ~~represents~~ is a reference point~~for the measured LLM. Since LLMs require substantial hardware resources, baselines serve as a comparison to assess their performance~~ , enabling comparison of LLMs against traditional algorithms with lower computational costs.

5.7.2 ***Recommendations:***

When selecting benchmarks and metrics, it is important to fully understand the benchmark tasks~~and the expected results because they determine what the benchmark actually assesses. Researchers~~ , what exactly is being measured, and how it relates to the (often latent) variables researchers actually care about. If one or more metrics or benchmarks are used, researchers:

- MUST briefly ~~summarize why they selected certain benchmarks in the~~ justify (in the *paper* ~~, They~~) why selected benchmarks and metrics are suitable for the given task or study;
- MUSTdiscuss the reliability and validity (especially construct validity) of selected metrics and benchmarks;
- SHOULD ~~also~~ summarize the structure and tasks of the selected benchmark(s), including the programming language(s) and descriptive statistics such as the number of contained tasks and test cases~~. Researchers~~ ;

- SHOULD ~~also~~ discuss the limitations of the selected benchmark(s) ~~. For example, many benchmarks focus heavily on isolated Python functions . This assesses~~ (e.g. widely used benchmarks such as *HumanEval* [30] and *MBPP* [12] Python functions assess a very specific part of software development, which is ~~certainly~~ not representative of the full breadth of ~~software engineering~~ SE work [27]~~.~~ ~~Researchers~~).
- SHOULD include an example of a task and the corresponding test case(s) to illustrate the structure of the benchmark.

If multiple benchmarks exist for the same task, researchers SHOULD compare performance between benchmarks. When selecting only a subset of all available benchmarks, researchers SHOULD use the most specific benchmarks given the context.

~~The use of LLMs might not always be reasonable if traditional approaches achieve similar performance. For many tasks for which LLMs are being evaluated, there exist traditional non-LLM-based approaches~~ Furthermore, researchers SHOULDcheck whether a less resource-intensive approach (e.g., for static analysis tasks or program repair) ~~that~~ can serve as a baseline. If so, the LLM or LLM-based tool should be compared with such baselines using suitable metrics. Even if LLM-based tools ~~perform better, the question is~~ outperform baselines, researchers SHOULDdiscuss whether the resources consumed justify the ~~potentially marginalimprovements. Researchers always check whether such traditional baselines exist and, if they do, compare them with the LLM or LLM-based tool using suitable metrics.~~ (often marginal) improvements [99].

To compare traditional and LLM-based approaches or different LLM-based tools, researchers SHOULD report established metrics whenever possible, as this allows secondary research. They can, of course, report additional metrics that they consider appropriate. ~~In any event~~We briefly discuss common metrics in the Example(s) subsection below. As mentioned, researchers MUST ~~argue why the selected metrics are suitable for the given task or~~ motivate why they chose a certain metric or variant thereof for their particular study.

~~If a study evaluates an LLM-based tool that is supposed to support humans, a relevant metric is the acceptance rate, meaning the ratio of all accepted artifacts~~ Due to LLM non-determinism, researchers SHOULDrepeat experiments and report descriptive statistics of model or tool performance (e.g., ~~test cases, code snippets)in relation to all artifacts that were generated and presented to the user. Another way of evaluating LLM-based toolsis calculating inter-model agreement (see also Section Use an Open LLM as a Baseline) . This allows researchers to assess how dependent a tool's performance is on specific models and versions. Metrics used to measure inter-model agreements include general agreement (percentage), *Cohen's kappa,* and *Scott's Pi coefficient.*~~ arithmetic mean, median, confidence intervals, standard deviations) [2, 21]. When comparing models or tools, researchers SHOULDuse appropriate inferential statistics (e.g., hypothesis tests such as the Mann-Whitney U test or bootstrap-based comparisons,

along with effect sizes) rather than relying solely on differences in means or other summary statistics.

~~LLM-based generation is non-deterministic by design. Due to this non-determinism,~~ The number of required repetitions depends on factors such as the study type, the variability of the task, and the desired precision of estimates. As with sample sizes for human validation (*Human Validation*), researchers SHOULD ~~repeat experiments to statistically assess the performance of a model or tool, e.g., using the arithmetic mean, confidence intervals, and standard deviations~~justify their chosen number of repetitions, for example, through a power analysis [21, 34] or by monitoring the convergence of descriptive statistics across incremental runs [22]. A pilot study can help estimate the expected variability and inform this decision.

From a measurement perspective, researchers SHOULD reflect on the theories, values, and measurement models on which the benchmarks and metrics they have selected for their study are based. For example, the phenomena labeled "bugs" in a large open dataset of software bugs ~~is a collection of bugs~~ are according to a certain theory of what constitutes a bug, and from the values and ~~perspective~~ perspectives of the people who labeled the dataset. Reflecting on the context in which these labels were assigned and discussing whether and how the labels generalize to a new study context is crucial.

5.7.3 ***Example(s):***

~~According to Hou et al, main problem types for LLMs are classification, recommendation and generation problems. Each of these problem types requires a different set of metrics. Hu et al conducted a comprehensive structured literature review on LLM benchmarks related to software engineering tasks. They analyzed 191 benchmarks and categorized them according to the specific task they address, making the paper a valuable resource for identifying existing metrics. Metrics used to assess generation tasks include *BLEU*, *pass@k*, *Accuracy*, *Accuracy@k*, and *Exact Match*. The most common metric for recommendation tasks is *Mean Reciprocal Rank*. For classification tasks, classical machine learning metrics such as *Precision*, *Recall*, *F1-score*, and *Accuracy* are often reported.~~

Common Metrics: ~~In the following, we briefly discuss two~~ Two common metrics used for generation tasks ~~:~~ are *BLEU-N* and *pass@k*. *BLEU-N*~~[118] is a similarity score based on~~ [118] was originally developed for evaluating machine translation quality by measuring n-gram precision between ~~two strings~~a candidate and reference text, ranging from 0 (dissimilar) to 1 ~~. Values close to 0 represent dissimilarcontent, values closer to 1 represent similarcontent, indicating that a model is more capable of generating the expected output. *BLEU-N* has multiple variations .~~ (similar). It has been widely adopted in SE for code generation tasks, though its validity in this context is debatable: n-gram overlap does not capture functional correctness, and syntactically different code can be semantically equivalent (see Challenges below). Code-specific variations such as *CodeBLEU* [127] and *CrystalBLEU*~~[43] are the most notable ones~~

~~tailored to code. They introduce~~ [43] attempt to address these limitations by introducing additional heuristics such as AST matching. ~~As mentioned, researchers motivate why they chose a certain metric or variant thereof for their particular study.~~

The metric *pass@k* reports the likelihood that a model correctly completes a code snippet at least once within ~~*k* attempts. To our knowledge, the basic concept of~~ *k* attempts. Originally introduced as ~~*pass@kwas first reported by Kulal et alto evaluate code synthesis [83] . They named the concept* success rate at B, *where B denotes the trial "budget." The term* pass@k *was later*~~ *success rate at B* by Kulal et al [83] and popularized by Chen et al~~as a metric for code generation correctness [30]. The exact definition of correctness varies depending on the task. For code generation, correctness is often defined based on test cases: A passing test implies that the solution is correct. The resulting pass rates range~~ [30], it ranges from 0 ~~to 1. A pass rate of 0 indicates that the model was unable to generate a single correct solution within~~ (no correct solution in *k* tries~~; a rate of~~) to 1 ~~indicates that the model successfully generated~~ (at least one correct solution~~in *k* tries~~), with correctness typically defined by passing test cases.

The *pass@k* metric is defined as: ~~$1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$, where~~

$$\text{pass@}k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}\text{, where:}$$

- n is the total number of generated samples per prompt,
- c is the number of correct samples among n, and
- k is the number of samples ~~.~~ drawn.

~~Choosing an appropriate value for *k*~~ The choice of *k* depends on the downstream task~~of the model and how end-users interact with it. A high pass rate for~~ : *pass@1* is ~~highly desirable in tasks where the system presents only one solution or if a single solution requires high computational effort. For example, code completiondepends on a single prediction, since end users typically see only a single suggestion. Pass rates for~~ critical for single-suggestion scenarios like code completion, while higher *k* values (e.g., 2, 5, 10) ~~indicate how well a model can solve a given task in multiple attempts. The *pass@k* metric is frequently referenced in papers on code generation~~ assess multi-attempt capability [57, 67, 89, 131]. However, *pass@k* is not a universal metric suitable for all ~~tasks. For instance, for creative or~~ generation tasks. It requires a binary notion of correctness, making the metric appropriate for code synthesis evaluated via unit tests, but unsuitable for open-ended ~~tasks,~~ generation tasks such as comment generation, ~~it is not a suitable metric since more than one correct result exists~~where multiple valid outputs exist.

~~Two benchmarks used for code generation are~~ If a study evaluates an LLM-based tool for supporting humans, a relevant metric is the acceptance rate, meaning the ratio of all accepted artifacts (e.g., test cases, code snippets)

in relation to all artifacts that were generated and presented to the user. Another way of evaluating LLM-based tools is calculating inter-model agreement. This allows researchers to assess how dependent a tool's performance is on specific models and versions. Metrics used to measure inter-model agreements include general agreement (percentage), *Cohen's kappa*, and *Krippendorff's* α (see *Human Validation*for recommended thresholds and best practices for measuring agreement).

Common problem types for LLM-based studies are classification, recommendation, and generation, each requiring different metrics [63]. Hu et al [64] categorized 191 LLM benchmarks by SE task, providing a valuable reference. Common metrics include *BLEU*, *pass@k*, *Accuracy@k*, and *Exact Match* for generation; *Mean Reciprocal Rank* for recommendation; and *Precision*, *Recall*, *F1-score*, and *Accuracy* for classification.

Benchmark Examples: Benchmarks used for code generation include *HumanEval* (available on GitHub~~) [118] and~~) [30], *MBPP* (available on Hugging Face~~) [12] . Both benchmarks consist of code snippets written in Python sourced from publicly available repositories. Each snippet consists of four parts: a prompt containing a function definition and a corresponding description of what the function should accomplish, a canonical solution, an entry point for execution, and test cases. The input of the LLM is the entire prompt. The output of the LLM is then evaluated against the canonical solution using metrics or against a test suite. Other benchmarks for code generation include~~) [12], *ClassEval* (available on GitHub) [40], *LiveCodeBench* (available on GitHub) [71], and *SWE-bench* (available on GitHub) [74]. An example of a code translation benchmark is *TransCoder* [84] (available on GitHub).

Most benchmarks focus on generation tasks, but benchmarks for classification and recommendation also exist. For classification, *DiverseVul* (available on GitHub) [31] provides vulnerable and non-vulnerable functions evaluated using standard classification metrics. For recommendation, *CodeSearchNet* (available on GitHub) [68] contains code-documentation pairs evaluated using *Mean Reciprocal Rank*.

~~Benchmarks are an essential tool for assessing model performance for SE tasks. Reproducible benchmarks measure and assess the performance of models for a specific SE tasks, enabling comparison. That~~

5.7.4 ***Benefits:***

Benchmarks and metrics improve reproducibility and comparability across studies, leading to faster improvements in the cumulative body of knowledge. It is simply easier to assess a new system against one or a few benchmarks than hundreds of competing systems, and when most systems are assessed against the same benchmarks and metrics, we can see which approaches work best (at least, on those benchmarks). This comparison enables progress tracking~~,~~ ; for example, when researchers iteratively improve a new LLM-based tool and

test it against benchmarks after significant changes. For practitioners, leaderboards ~~, i.e.,~~ (published benchmark results of models~~, support the selection of~~) support selecting models for downstream tasks. Meanwhile, baselines help us understand just how much LLMs improve performance over non-LLM-enabled alternatives.

5.7.5 ***Challenges:***

~~A general challenge with benchmarks for LLMs is that the most prominent ones, such as *HumanEval*~~ Computer scientists have a long history of using oversimplified, unvalidated metrics [122] (e.g., lines of code, CPU time) as proxies for complex, multidimensional latent variables (e.g., system size, environmental sustainability). In fields such as psychology, where measurement theory has a longer tradition [23], metrics are backed by foundational theories or extensive empirical validation of their psychometric properties. The fundamental challenge with AI metrics and benchmarks is that many have neither firmly understood theoretical underpinnings nor extensive empirical validation of their construct, measurement, and ~~*MBPP*, use Python, introducing a bias toward this specific programming language and its idiosyncrasies. Since modelperformance ismeasured against these benchmarks, researchers often optimize for them. As a result, performance may degrade if programming languages other than Python are used.~~ ecological validity.

~~Many closed-source models, such as those released by *OpenAI*, achieve exceptional performance on certain tasks but lack transparency and reproducibility [40, 88, 183] . Benchmark leaderboards, particularly for code generation~~For example, *pass@k* is intended to measure a model's *functional correctness*, that is, ~~are led by closed-source models [40, 183]. While researchers compare performance against these models, they must consider that providers might discontinue them or apply undisclosed pre- or post-processing beyond the researchers ' control (see)~~its ability to generate code that produces the expected output for a given specification. However, functional correctness is only one dimension of code quality. *pass@k* does not capture maintainability, readability, security, or efficiency of the generated code, all of which are critical for downstream use. Furthermore, "correctness" is defined entirely by test suites, whose coverage is itself unvalidated: a solution that passes all provided tests may still be incorrect for untested inputs. Whether a test suite adequately operationalizes correctness for a given task is a subjective judgment that is rarely examined.

Similarly, *HumanEval* is intended to measure a model's ability to *synthesize short Python functions from docstring specifications*. This operationalizes a narrow slice of "code generation ability": it covers neither multi-file tasks, nor debugging, nor the use of existing codebases—tasks that dominate real-world software engineering [27]. Notably, neither *pass@k* nor *HumanEval* has undergone rigorous empirical validation of its construct validity; their widespread adoption rests on face validity and convenience rather than on evidence that they reliably measure what researchers intend them to measure [26].

~~The challenges of individual metrics include that, for example,~~ Benchmarks and metrics also have generalizability problems. Prominent LLM benchmarks such as *HumanEval* and *MBPP* use Python, so researchers can optimize for Python's idiosyncrasies, creating illusory improvements that do not generalize to other languages. Similarly, the metric *BLEU-N* is a syntactic metric~~and therefore does not measure semantic or structural correctness. Thus, a high *BLEU-N* score does not directly indicate that the generated code is executable. While alternatives exist, they often come with their own limitations. For example, *Exact Match* is a strict measurement that~~ , so code can score highly without being executable. The metric *Exact Match*, meanwhile, does not account for functional equivalence of syntactically different code. Both *BLEU-N* and *Exact Match* are influenced by code formatting, which confounds their intended use.

Execution-based metrics such as *pass@k* directly evaluate correctness by running test cases, but they require a setup with an execution environment. When researchers observe unexpected values for certain metrics, the specific results should be investigated in more detail to uncover potential problems. ~~These problems can be related to formatting, since code formatting is known to influence metrics such as *BLEU-N* or *Exact Match*.~~

~~Another challenge to consider is that metrics usually capture one specific aspect of a task or solution. For example, metrics such as *pass@k* do not reflect qualitative aspects of the generated source code, including its maintainability or readability. However, these aspects are critical for many downstream tasks. Moreover, benchmarks are isolated test sets and may not fully represent real-world applications. For example, benchmarks such as *HumanEval* synthesize code based on written specifications. However, such explicit descriptions are rare in real-world applications. Thus, evaluating model performance with benchmarks might not reflect real-world tasks and end-user performance.~~

Finally, benchmark data ~~contamination [172] continues to be a major challenge. In many cases, the LLM training data is not released. However, the benchmark itself could be~~ contamination—where the benchmark itself is part of the training ~~data. Such benchmark contamination may lead to~~ data—may lead to artificially high performance if the model remembers the ~~model remembering the~~ solution from the training data rather than ~~solving the new~~ actually solving the task based on the input data~~. This leads to artificially high performance on benchmarks. However, for unforeseen scenarios, the model might perform much worse.~~ [172]. Therefore, for proprietary LLMs that do not release their training data, researchers SHOULDconsider using human validation, curating new data, or refactoring existing data [172].

To mitigate contamination, researchers can create new benchmark datasets by collecting data after a specified cutoff date; researchers MUSTdisclose data sources and collection dates for each release. However, this temporal approach requires continuous updates as new models may include the benchmark in future training data. Alternatively, keeping the benchmark private prevents inclusion in training sets, but requires trust in the benchmark creator and a

system to execute it without leaking data. Researchers can also evaluate how contaminated their benchmark is [32].

5.7.6 ***Study Types:***

This guideline MUST be followed for all study types that automatically evaluate the performance of LLMs or LLM-based tools. The design of a benchmark and the selection of appropriate metrics are highly dependent on the specific study type and research goal. Recommending specific metrics for specific study types is beyond the scope of these guidelines, but Hu et al provide a good overview of existing metrics for evaluating LLMs [64]. ~~In addition, our Section provides an overview of the integration of human evaluation in LLM-based studies~~

For *Benchmarking LLMs*, this guideline is of primary importance: researchers MUSTuse established benchmarks or rigorously justify the creation of new ones, and MUSTreport standard metrics to enable cross-study comparison. For *LLMs as Annotators*, the research goal might be to assess which model comes close to a ground truth dataset created by human annotators. Especially for open annotation tasks, selecting suitable metrics to compare LLM-generated and human-generated labels is important. In general, annotation tasks can vary significantly. Are multiple labels allowed for the same sequence? Are the available labels predefined, or should the LLM generate a set of labels independently? Due to this task dependence, researchers MUST justify their metric choice, explaining what aspects of the task it captures together with known limitations. For *LLMs for Tools*, researchers SHOULDbenchmark the tool against suitable baselines using appropriate metrics that capture the tool's intended contribution. For *LLMs as Judges*, researchers SHOULDreport inter-rater agreement metrics and validity measures to demonstrate the reliability and quality of LLM judgments. For *LLMs for Synthesis*, researchers SHOULDspecify metrics for comparing synthesized outputs (e.g., coverage, faithfulness) and justify their appropriateness for the synthesis task. For *Studying LLM Usage*, researchers SHOULDjustify the measurement instruments and metrics used for studying LLM usage patterns, including any survey scales or behavioral measures. For *LLMs as Subjects*, researchers SHOULDcompare simulated and real human responses using appropriate metrics to assess simulation fidelity. If researchers assess a well-established task such as code generation, they SHOULD report standard metrics such as *pass@k* and compare the performance between models. If non-standard metrics are used, researchers MUST state their reasoning.

5.7.7 ***Advice for Reviewers:***

Reviewers should expect manuscripts to: 1. clearly identify the constructs or variables the study aims to measure (e.g., LLM performance, quality of generated code), including independent, dependent, and control variables; 2. present their measurement model, i.e., which metrics, benchmarks, or baselines are used and how they relate to the target constructs; 3. justify the selection

of any metrics, benchmarks, and baselines used; 4. discuss *in detail* the assumptions, reliability, and validity—*especially construct validity*—of each benchmark and metric; 5. articulate any limitations regarding construct and measurement validity. As with other guidelines, missing information about baselines or metrics is typically a revision request. However, vague descriptions that conflate broad concerns (e.g., effectiveness, quality) with specific counting methods should be questioned. Ubiquity of a benchmark does not imply validity or appropriateness for a given context. Manuscripts should convey a solid understanding of construct and measurement validity by explaining and justifying their measurement models.

5.7.8 ***See Also:***

→ Section *Open LLMs*: open models for inter-model comparison.
→ Section *Human Validation*: human evaluation to complement automated metrics.
→ Section *Limitations and Mitigations*: reporting construct and measurement validity limitations.

5.8 Report Limitations and Mitigations (G8)

> ~~*tl;dr:*~~ tl;dr: *Researchers* MUST *transparently report study limitations, including the impact of non-determinism and generalizability constraints. The* paper MUST *specify whether generalization across LLMs or across time was assessed, and discuss model and version differences. Authors* MUST *describe measurement constructs and methods, disclose any data leakage risks, and avoid leaking evaluation data into LLM improvement pipelines. They* MUST *provide model outputs, discuss sensitive data handling,* ~~*ethics approvals,*~~ *and justify LLM usage in light of its resource demands. Mitigation strategies such as replication packages, human validation, longitudinal re-runs, triangulation, and sensitivity analysis* SHOULD *be employed and reported where applicable. Where full data sharing is not possible, a subset of the validation data* SHOULD *be included to enable partial replication.*

5.8.1 ***Rationale:***

When using LLMs for empirical studies in SE, researchers face unique challenges and potential limitations that can influence the validity, reliability, and reproducibility of their findings [133]. It is important to openly discuss these limitations and explain how their impact was mitigated. ~~All this is relative to the current capabilities of LLMs and the current state of the art in terms of tool architectures. If and how the performance of LLMs in a particular study context will improve with future model generations or new architectural patterns is beyond what researchers can and should discuss~~

~~as part~~ These limitations are relative to current LLM capabilities and tool architectures; speculating about future improvements is beyond the scope of a paper's limitation section. Nevertheless, risk management and threat mitigation ~~are important tasks during the design of an empirical study . They should not happen~~ should be planned during study design, not as an afterthought.

5.8.2 ***Recommendations:***

~~A cornerstone of open science is the ability to reproduce research results. Although the inherent non-deterministic nature of LLM is a strength in many use cases, its impact on *reproducibility* is problematic. To enable reproducibility, researchers disclose a replication package for their study. They perform multiple repetitions of their experiments (see)to account for non-deterministic outputs. In some cases, researchers reduce the output variability by setting the temperature to a value close to 0 and setting a fixed seed value. However, configuring a lower temperature can negatively impact task performance, and not all models allow the configuration of seed values. Besides the inherent non-determinism~~Researchers MUST make every effort to present the limitations of their work clearly without defensiveness or obfuscation. These limitations may concern diverse topics including generalizability, internal validity (e.g. data leakage), ~~the behavior of an LLM depends on many external factors such as prompt variations and model evolution. To ensure reproducibility ,~~ reliability (i.e. non-determinism), and reproducibility (e.g. resource requirements). When deterministic reproducibility is structurally unattainable (e.g., SaaS-based models with opaque versioning), researchers SHOULD ~~follow all previous guidelines and further discuss the generalization challenges outlined below.~~

~~Although the topic of *generalizability* is not new, it has gained new relevance with increasing interest in LLMs. In~~ adopt trustworthiness criteria from qualitative research to substantiate dependability and confirmability of findings. LLM-based studies ~~, generalizability boils down to two main concerns:~~ may involve many kinds of generalization including the following:

1. From tested LLMs to others; that is, do the performance characteristics of the LLM(s) studied generalize to LLM(~~1) First, are the results specific to an LLM, or can they be achieved with other LLMs as well? (2) Second, will these results still be valid in the future? If generalizability to other LLMs isnot in the scope of the research, this be clearly explained in the or, if generalizability is in scope, researchers compare their results or subsets of the results (e.g., due to computational cost) with other LLMs that are, e.g., similar in size, to assess the generalizability of their findings (see also Section) . Multiple studies (e.g., [29, 87]) found that the performance of proprietary LLMs (e.g., GPT) decreased over time for certain tasks using the same model version (e.g. GPT-4o). Reporting the model version and configuration is not sufficient in such cases. To date,~~ s) not included in the study.
2. From tested configurations to others; that is, how sensitive are the results to the ~~only way to mitigate this limitation is the usage of an open LLM having~~

~~a versioning schema and archiving (see) . Hence, researchers employ open LLMs to establish a reproducible baseline and, if the use of an open LLMis not possible, researchers test and report their results over an extended period of time as a proxy of the stability of the results over time.~~ specific configuration(s) of the LLM under test.

3. From research to practice; for instance, just because researchers can get a certain level of code generation performance under ideal conditions does not mean software developers, given the same tools, will get similar results without having the researchers present to show them exactly what to do.
4. From a sample of people (e.g. human validators; human participants) to a larger population of people.
5. From one time period to another; i.e., does the same LLM, or other versions of the same LLM, produce similar results at a different point in time?

~~*Data leakage, contamination, or overfitting* occurs when information outside the training data influences model performance. With the growing reliance on big datasets, the risks of~~ The following concerns present an overview that has to be tailored to the individual study context. This section does not repeat requirements from other recommendations.

Internal Validity: The primary threats to internal validity are:

- Data leakage and contamination, including inter-dataset duplication~~increases (see, e.g., [6, 92]). In the context of LLMs for SE, this can manifest itself, for example, as training data samples that appear in the fine-tuning or evaluation datasets, potentially compromising the validity of the evaluation results [93]. Moreover, ChatGPT's functionality to "improve the model for everyone" can result in unintentional~~ , potentially resulting in a training–evaluation overlaps, leading to overly optimistic evaluation results.
- Unintended inclusion of evaluation data in model-improvement pipelines, contributing to data leakage. ~~Hence, to ensure the validity of the evaluation results, researchers carefully curate the fine-tuning and evaluation datasets to prevent inter-dataset duplication and leak their fine-tuned or evaluation datasets into LLM improvement processes. When publishing the results, researchers of course still follow open science practices and publish the datasets as part of their . If information about the training data of the employed LLM is available, researchers evaluate the inter-dataset duplication and~~ This is especially relevant for longitudinal studies including LLMs.
- Incomplete architecture, prompt, or pipeline reporting, posing hidden confounding factors.

Inter-dataset duplication is prevalent in SE, particularly for code-related benchmarks. As transparency on training data is limited for LLMs, researchers MUST discuss potential data leakage ~~in the . When training an LLM from scratch, researchers consider using open datasets such as *RedPajama (Together AI)* [36] , which are already built with deduplication in mind (with the positive side effect of potentially improving performance [86])~~effects and their impact on results.

Construct Validity: The primary threats to construct validity are:

- Metric–construct mismatch (e.g., BLEU/ROUGE vs. functional correctness). More traditional metrics might not capture all relevant SE-specific aspects.
- Over-reliance on benchmark-specific metrics. Optimizing for a single benchmark might result in dataset-specific heuristics rather than the intended construct, overstating real-world utility.
- Benchmark scope limitations. Benchmarks commonly ignore runtime behaviors, security implications, readability, testability, and maintainability, yielding results that may not transfer to realistic development settings.

If constructs are based on subjective interpretations, purely automated metrics are insufficient. Researchers MUSTdiscuss how they ensured quality of subjective results, similarly to qualitative research.

~~Conducting studies with LLMs is a~~

External Validity: The primary threats to external validity are:

- Cross-model transfer limitations. Results obtained with one LLM (or family of LLMs) may not generalize to others due to differences in training data, architecture, and capabilities.
- Tool-architecture specificity. Tools built around a specific LLM's API or capabilities may not transfer to other models without significant re-engineering.
- Limited domain coverage. Studies often focus on a narrow set of programming languages, task types, or application domains, limiting generalizability to other SE contexts.
- Limited participant diversity. Study participants (e.g., human validators, developers) may not represent the broader population in terms of expertise, geographic location, or cultural background.
- Cross-time instability (model evolution). Performance of proprietary models can change over time, leading to non-generalizable study outcomes [29, 87] .

Generalizability is particularly critical for proprietary and non-deterministic systems whose behavior is subject to drift (i.e., silent changes in model output over time) [29]. The limitations and mitigations used for external validity MUSTbe discussed by the researchers.

Reliability & Reproducibility: The primary threats to reliability and reproducibility are:

- Non-deterministic outputs. Identical prompts and configurations can yield different outputs across runs due to factors such as floating-point arithmetic, batching, and stochastic decoding strategies.
- Infrastructure dependence. Results may vary depending on the hardware, software stack, and hosting environment used, making exact replication challenging across different infrastructure setups.

- Resource inequality preventing replication. LLM research is resource-intensive ~~process. For self-hosted LLMs,~~ and hence remains predominantly in the ~~respective hardware must be provided, and for managed LLMs, the service *costs* must be considered. The challenge becomes more pronounced as LLMs grow larger, research architectures become more complex, and experiments become more computationally expensive. For example, multiple repetitions to assess model or tool performance (see) multiply the cost and impact *scalability*. Consequently, resource-intensive research remains predominantly the~~ domain of private companies or well-funded research institutions~~, hindering researchers with limited resources in reproducing, replicating, or extending study results. Hence, for transparency reasons, researchers report the cost associated with executing an LLM-based study. If the study used self-hosted LLMs, researchers report the specific hardware used. If the study used managed LLMs, the service cost be reported. To ensure the validity and reproducibility of the research results, researchers~~ [3, 141], excluding researchers from under-resourced institutions.

Researchers MUST ~~provide the LLM outputs as evidence (see Section). Depending on the architecture (see Section), these outputs be reported on different architectural levels~~ discuss the measures taken to increase reliability and reproducibility. However, non-deterministic reproducibility is not inherently disqualifying. Qualitative traditions such as ethnography, grounded theory, and action research ensure trustworthiness through *credibility*, *transferability*, *dependability*, and *confirmability* [55]. SaaS-based LLM research faces analogous conditions, as providers frequently deprecate model versions without guaranteeing stable behavior.

Ethical & Regulatory Boundaries: The primary concerns for ethical and regulatory matters are:

- Use of sensitive or proprietary data. Studies involving proprietary code, confidential business data, or personally identifiable information may face restrictions on data sharing that limit reproducibility.
- Jurisdictional obligations. Data protection regulations (e.g., ~~outputs of individual LLMs in multi-agent systems) . Additionally,~~ GDPR, CCPA) and institutional policies may impose constraints on data collection, processing, and sharing in LLM-based studies.
- Implicit model bias. Especially for qualitative research LLM might "*reinforce dominant paradigms and biases*" and "*identify, replicate and reinforce dominant language and patterns*" [75].

Studies involving sensitive data MUSTdiscuss data governance mechanisms tailored towards LLM environments, compliant with applicable juristical obligations. If applicable, researchers SHOULD ~~include a subset of the employed validation dataset, selected using an accepted sampling strategy, to allow partial replication of the results.~~ discuss how model biases potentially impact the study outcomes and how those biases were evaluated.

~~*Sensitive data* can range from personal to proprietary data , each with its own set of *ethical concerns*. As mentioned above, a big threat to proprietary LLMs and sensitive data is its use for model improvement. Hence, using sensitive data can lead to privacy and intellectual property (IP) violations. Another threat is the implicit bias of LLMs that could lead to discrimination or unfair treatment of individuals or groups. To mitigate these concerns, researchers~~

Environmental & Sustainability Constraints:

- Energy consumption. With growing model size, the environmental impact of experiments with LLMs increases. Considering the environmental impact in the study design is important given the substantial energy costs of LLM experiments [151].
- Trade-off between repetition and sustainability. Increased reliability through repetition and the aim of cautious energy consumption pose an inherent tension field to navigate during the study design.

The study findings SHOULD ~~minimize the sensitive data used in their studies, follow applicable regulations (e.g., GDPR) and individual processing agreements, create a data management plan outlining how the data is handled and protected against leakage and discrimination, and apply for approval from the ethics committee of their organization (if required).~~ be discussed in the context of the resource-intensiveness of LLMs to justify the increased resource consumption over traditional approaches.

~~Although *metrics* such as *BLEU* or *ROUGE* are commonly used to evaluate the performance of LLMs (see Section), they may not capture other relevant SE-specific aspects such as functional correctness or runtime performance of automatically generated source code [91]. Given the high resource demand of LLMs , in addition to traditional metrics such as accuracy, precision, and recall or more contemporary metrics such as *pass@k*, *resource consumption* has to become a key indicator of performance. While research has predominantly focused on energy consumption during the early phases of building LLMs (e.g., data center manufacturing, data acquisition, training), inference, that is LLM usage, often becomes similarly or even more resource-intensive [46, 73, 102, 158, 169] . Hence, researchers~~

Mitigation Strategies The following mitigation strategies can help address the threats described above, depending on the study scope and feasibility.

- *Replication Packages* covering prompt and architecture specifications, model outputs, and representative examples for partial replicability, accompanied by an implementation using an open model for long-term stability.
- *Human Validation* of subjective constructs, following quality criteria known from qualitative research.
- *Longitudinal Re-Runs* to repeat experiments with LLMs over time, complemented by statistical analyses.

- *Methodological Trustworthiness Measures.* Researchers SHOULD ~~aim for lower resource consumption on the model side. This can be achieved~~ consider triangulation, reflexivity, audit trails, and peer debriefing as complementary measures when deterministic reproduction is structurally impossible.
- *Triangulation* via multiple models (e.g., proprietary and open models), multiple independent datasets, and multiple complementary metrics.
- *Cost Accounting* through reporting of input/output tokens, token and service costs, or hardware specifications.
- *Energy Preservation* by selecting smaller ~~(e.g., GPT-4o-mini instead of GPT-4o) or newer models or by~~ or newer less resource-intensive models and employing techniques such as input/output token reduction, model pruning, quantization, or knowledge distillation [102] ~~.~~ where feasible. Carbon footprint estimation is desirable, but still difficult.
- *Ethical and Regulatory Considerations* through data governance mechanisms, ethical reviews, and bias assessment procedures.
- *Sensitivity Analysis* through variation of LLM configurations, prompts, architecture decisions, datasets, and if applicable human participant backgrounds.

5.8.3 ***Study Types:***

Researchers ~~further reduce resource consumption by restricting the number of queries, input tokens, or output tokens [102], by using different prompt engineering techniques (e.g., zero-shot promptsseem to emit less CO2 than chain-of-thought prompts), or by carefully sampling smaller datasetsfor fine-tuning and evaluation instead of using large datasets. Reducing resource consumption involves a trade-off with our recommendation to perform multiple experimental runs to account for non-determinism, as suggested in . To report the environmental impact of a study,~~ MUSTfollow this guideline for all study types. Transparently reporting limitations and mitigations is a universal requirement, but specific concerns vary by study type. For *LLMs as Annotators*, researchers ~~use software such as CodeCarbon or Experiment Impact Tracker to track and quantify the carbon footprint of the study or report an estimate of the carbon footprint through tools such as MLCO2 Impact. They detail the LLM version and configuration as described in , state the hardware or hosting provider of the model as described in and report the total number of requests, accumulated input and output tokens. Researchers~~ MUST ~~justify why LLMs were chosen over existing approaches and~~ discuss potential biases in label assignment, label reliability limitations, and sensitivity of annotations to prompt wording and model choice. For *LLMs as Judges*, researchers MUSTaddress measurement validity concerns, known biases such as position bias or verbosity bias, and the extent to which LLM judgments align with human expert assessments. For *LLMs for Synthesis*, researchers MUST discuss the ~~achieved performance in relation to their potentially higher resource consumption.~~ risk of contextual misinterpretation, potential loss of nuance in summarized or aggregated outputs, and reflexivity limitations inherent in using an LLM for qualitative interpretation.

For *LLMs as Subjects*, researchers MUSTdiscuss the fundamental inability of LLMs to truly simulate human behavior, the risk of stereotype amplification, and the limited ecological validity of simulated responses. For *Studying LLM Usage*, researchers MUSTdiscuss generalizability constraints across different tools and user populations, and acknowledge how observed usage patterns may not transfer to other contexts. For *LLMs for Tools*, researchers MUSTdiscuss replicability constraints arising from dependencies on commercial models, the impact of model updates on tool behavior, and limitations of the evaluation setup. For *Benchmarking LLMs*, researchers MUSTdiscuss potential data contamination, benchmark scope limitations, and the extent to which benchmark performance generalizes to real-world tasks.

5.8.4 ***Advice for Reviewers:***

~~An example highlighting the need for caution around *replicability* is the study of Staudinger et al [150] who attempted to replicate an LLM study. The authors were unable to reproduce the exact results, even though they saw similar trends as in the original study.~~

~~To analyze whether the results of proprietary LLMs transfer to open LLMs, Staudinger et al benchmarked previous results using GPT-3.5 and GPT4 against Mistral and Zephyr [150]. They found that open-source models could not deliver the same performance as proprietary models. This paper is also an example of a study reporting *costs*: . Tinnes et al is an example of balancing dataset size between the need for manual semantic analysis and computational resource consumption [157].~~

~~Individual studies have already begun to highlight the uncertainty about the *generalizability* of their results in the future. Jesse et al acknowledge the issue that LLMs evolve over time and that this evolution could impact the study results [72]. Since a lot of research in SE evolves around code, inter-dataset code duplication has been extensively researched over the years to curate de-duplicated datasets (see, e.g., [6, 77, 92, 93]). The issue of inter-dataset duplication has also attracted interest in other disciplines. For example, in biology, Lakiotaki et al acknowledge and address the overlap between multiple common disease datasets [85]. In the domain of code generation, Coignion et al evaluated the performance of LLMs to produce LeetCode [35]. To mitigate the issue of inter-dataset duplication, they only used LeetCode problems published after 2023-01-01, reducing the likelihood of LLMs having seen those problems before. Further, they discuss the performance differences of LLMs on different datasets in light of potential inter-dataset duplication. Zhou et al performed an empirical evaluation of data leakage in 83 software engineering benchmarks [181] . Although most benchmarks suffer from minimal leakage, very few showed a leakage of up to 100%. The authors found a high impact of data leakage on the performance evaluation. A starting point for studies that aim to assess and mitigate inter-dataset duplication are the Falcon LLMs, for which parts of its training data are available on Hugging Face [156]. Through this dataset, it is possible to reduce the overlap between the traning and evaluation data,~~

~~improving the validity of the evaluation results. A starting point to prevent active data leakage into a LLM improvement process is to ensure that~~Reviewers should verify that the ~~data is not used to train the model~~ limitation section is comprehensive and appropriate for the specific study type, checking that: (1) limitations address the specific threats relevant to the study type (e.g., ~~via OpenAI's data control functionality) [14].~~

~~Finally, bias can occur in LLM training datasets, resulting in various types of discrimination. Gallegos et al propose metrics to quantify biases in various tasks (e.g., text generation, classification, question answering) [47].~~

~~The *reproduction* or *replication* of study results under similar conditions by different parties greatly increases the validity of the results. Independent verification is of particular importance for studies involving LLMs, due to the~~ label reliability for annotation studies, simulation fidelity for studies using LLMs as subjects); (2) mitigations are concrete and correspond to identified limitations rather than being generic statements; (3) the impact of LLM non-determinism ~~of their outputs and the potential for biases in their training, fine-tuning, and evaluation datasets. Mitigating threats to *generalizability* of a study through the integration of an open LLM as a baseline or the reportingof results over an extended period of time can increase the validity, reliability, and replicability of a study's results. Assessing and mitigating the effects of *inter-dataset duplication* strengthens a study's validity and reliability, as it prevents overly optimistic performance estimates that do not apply to previously unknown samples. Reporting the *costs* of performing a study not only increases transparency but also provides context for secondary literature. Providing *replication packages* with LLMoutput and data samples for partial replicability are paramount steps toward open and inclusive research in light of resource inequality between researchers. Considering and justifying the usage of LLMs over other approaches can lead to more efficient and sustainable solutions for SE problems. Reporting the *environmental impact* of LLM usage also sets the stage for more sustainable research practices in SE.~~ on findings is discussed; (4) generalizability constraints—across models, configurations, time periods, and populations—are acknowledged. When important limitations are missing, reviewers should request they be added. The absence of a limitation section, or one that is formulaic or insufficiently specific, is a more serious concern than any individual missing limitation and may warrant a major revision.

~~With commercial LLMs evolving over time, the *generalizability* of results to future model versions is uncertain. Employing open LLMs as a baseline can mitigate this limitation, but may not always be feasible due to computational cost. Most LLM providers do not publicly offer information about their training data, impeding the assessment of *inter-dataset duplication* effects. Consistently keeping track of and reporting the *costs* involved in a research endeavor is challenging. Building a coherent *replication package* that includes LLM outputs and samples for partial replicability requires additional effort and resources. Finding the right *metrics* to evaluate the performance of LLMs in SE for specific tasks is challenging. Defining all requirements beforehand to ensure the~~

~~use of suitable metrics can be a challenge, especially in exploratory research. Our Section~~

5.8.5 ***See Also:***

→ Section *Version and Configuration*: model configuration, cost accounting, and sensitivity analysis.
→ Section *Architecture*: architecture-level triangulation and sensitivity analysis.

→ Section *Prompts and Logs*: prompt reporting, audit trails, and longitudinal re-runs.
→ Section *Benchmarks and Metrics* ~~and its references can serve as a starting point. Ensuring *compliance* across jurisdictions is difficult with different regions having different regulations and requirements (e.g., GDPR and the AI Act in the EU, CCPA in California). Selecting datasets and models with fewer *biases* is challenging, as biases are often unknown. Measuring or estimating the *environmental impact* of a study is challenging and might not always be feasible. Especially in exploratory research, the impact is hard to estimate in advance, making it difficult to justify the usage of LLMs over other approaches.~~ : metric selection and benchmark limitations.
→ Section *Human Validation*: human validation of subjective constructs.

~~Limitations and mitigations be discussed for all study types in relation to the context of the individual study. This section provides a starting point and lists aspects along which researchers can reflect on the limitations of their study design and potential mitigations that exist.~~

6 Conclusion

~~This paper organizes the landscape of empirical studies involving LLMs in software engineering along seven study types and distills eight guidelines aimed at improving their validity, transparency, and reproducibility.~~ LLM non-determinism, opaque training data, and rapidly evolving models threaten the reproducibility and replicability of empirical SE studies. This paper presents a taxonomy of seven study types that organizes how LLMs are used in SE research, together with eight guidelines for designing and reporting such studies. Each guideline distinguishes requirements (MUST) from recommended practices (SHOULD) and is contextualized by the study types it applies to. Our guidelines recommend that researchers: (1) ~~explicitly declare when and how LLMs are used~~declare LLM usage and role; (2) report model versions, ~~configuration, and fine-tuning details~~configurations, and customizations; (3) ~~describe the complete~~ document the tool architecture beyond the model; (4) ~~release~~ disclose prompts, their development, and, where ~~possible~~feasible, interaction logs; (5) validate LLM outputs with humans; (6) include an open LLM as a baseline; (7) ~~select appropriate~~ use suitable baselines, benchmarks, and metrics; and (8) ~~report~~

Table 4 Rationale and key recommendations for each guideline. ● = MUST, ○ = SHOULD.

ID	Rationale	Core Recommendations
G1	Transparency enables informed assessment of scope and limitations.	●Declare which LLM, how it was used, and where in the research process.
G2	Reproducibility requires precise identification of the system used in a study.	●Report exact version, date, configuration, and fine-tuning details.
G3	Complex systems require holistic documentation beyond the model itself.	●Describe full architecture, agent roles, and infrastructure.
G4	Prompts shape LLM behavior and must be verifiable.	●Publish all prompts, context files, and development strategy. ○Share interaction logs where feasible.
G5	Automated metrics alone cannot ensure validity of subjective constructs.	○Validate against human judgment with inter-rater reliability, where applicable.
G6	Reproducibility depends on access to the model under study.	○Include open LLM baseline; provide replication package.
G7	Meaningful evaluation requires reasoned valid measurement.	●Justify metric and benchmark choices; discuss their validity. ○Repeat experiments and report descriptive statistics.
G8	Honest acknowledgment of threats strengthens a study.	●Report threats by validity type. ○Employ mitigations where possible.

~~study~~ articulate limitations and mitigations, including costs, potential biases, and environmental impact. An applicability matrix maps guidelines to study types, and a reporting checklist (Section A) supports authors and reviewers. The study types and guidelines were derived collaboratively by 22 co-authors, following a process similar to that of guidelines in other disciplines (see, e.g., [20]).

Following the quality dimensions of the *SIGSOFT Empirical Standards* [123], we consider our guidelines (1) *comprehensive*, as they cover seven study types that we identified and were derived from 22 co-authors' collective expertise; (2) *useful*, as they provide actionable recommendations with an applicability matrix and a concrete reporting checklist (Section A); (3) *well-argued*, as each guideline is motivated by a dedicated rationale and supported by references to the literature; and (4) *integrated with previous work*, as they build on and complement established empirical SE standards while addressing LLM-specific challenges. Adopting these practices will not eliminate LLM non-determinism, but it ~~will~~ should make claims auditable ~~,~~ and results easier to replicate and compare across studies. ~~By consistently applying our recommendations, the community can produce more reliable evidence on what LLM can and~~

~~cannot deliver for software engineering research and practice. Our guidelines complement existing empirical software engineering guidance while addressing LLM-specific challenges.~~ Because models and tools evolve, we maintain the guidelines as a living resource and invite the community to refine and extend them ~~:~~ at llm-guidelines.org.

Acknowledgments

Our study types and guidelines are based on discussion sessions with researchers at the *2024 International Software Engineering Research Network* (ISERN) meeting and at the *2nd Copenhagen Symposium on Human-Centered Software Engineering AI* (supported by the Carlsberg Foundation with grant CF24-0693 and the Alfred P. Sloan Foundation with grant G-2024-22586 to Daniel Russo). ~~ChatGPT with~~ LLM-based tools, including ChatGPT (GPT-5~~was used to generate initial versions of the abstract and conclusion~~) and Claude Code (Opus 4.6), were used interactively for language editing and structural revision suggestions across all sections of this paper. All LLM-generated suggestions were reviewed and revised by the authors.

Appendix A Checklist

The following checklist, inspired by CONSORT [140], summarizes actionable items from the guidelines based on the *tl;dr* sections. The checklist is organized along typical paper sections. Items marked ● are requirements (MUST); items marked ○ are recommendations (SHOULD). Each item references its source guideline (G1–G8).

Introduction

- ☐ ● Disclose any use of LLMs, specifying which LLM, how, and where it was used (G1).
- ☐ ○ Report the purpose of using LLMs, automated tasks, and expected benefits (G1).

Research Design and Methods

Model Selection and Configuration

- ☐ ● Report the exact LLM model or tool version, configuration, and experiment date in the *paper* (G2).
- ☐ ● For fine-tuned models, describe the fine-tuning goal, dataset, and procedure (G2).

- ☐ ○ Report default parameters and explain model and version choices (G2).
- ☐ ○ For quantized models, report the quantization level and method (G2).
- ☐ ○ Compare base and fine-tuned models using suitable metrics and benchmarks; share fine-tuning data and weights (or justify why they cannot be shared) (G2).

Architecture

- ☐ ●Describe the full architecture of LLM-based tools in the *paper*, including the role of the LLM, interactions with other components, and overall system behavior (G3).
- ☐ ●If autonomous agents are used, specify agent roles, reasoning frameworks, and communication flows (G3).
- ☐ ●Report hosting, hardware setup, and latency implications (G3).
- ☐ ●For tools using retrieval or augmentation methods, describe data sources, integration mechanisms, and update and versioning strategies (G3).
- ☐ ●For ensemble architectures, explain the coordination logic between models (G3).

- ☐ ○Include architectural diagrams and justify design decisions (G3).

Prompts and Interactions

- ☐ ●Publish all prompts, including their structure, content, formatting, and dynamic components (G4).
- ☐ ●Describe prompt development strategies (e.g., zero-shot, few-shot), rationale, and selection process (G4).
- ☐ ●Document input handling and token optimization strategies when prompts are long or complex (G4).
- ☐ ●Report generation and collection processes for dynamically generated or user-authored prompts (G4).
- ☐ ●Specify prompt reuse across models and configurations (G4).
- ☐ ●Report all context files (e.g., `CLAUDE.md`, `.cursorrules`) used to configure AI coding agents as *supplementary material* (G4).
- ☐ ○If full prompt disclosure is not feasible, provide summaries or examples (G4).

- ☐ ○Report prompt revisions and pilot testing insights (G4).
- ☐ ○For agentic systems, report developed plans and generalize interaction logs to include human-agent interaction traces (G4).
- ☐ ○Include full interaction logs (prompts and responses) if privacy and confidentiality can be ensured (G4).

Human Validation

- ☐ ●If using human validation, define the measured construct (e.g., usability, maintainability) and describe the measurement instrument in the *paper* (G5).

- ☐ ○Consider human validation early in the study design, build on established reference models for human-LLM comparison, and share instruments as *supplementary material* (G5).
- ☐ ○When aggregating LLM judgments, report methods and rationale and assess inter-rater agreement (G5).
- ☐ ○Control for confounding factors and conduct power analysis to ensure statistical robustness (G5).

Benchmarks and Metrics

- ☐ ●Justify all benchmark and metric choices in the *paper* (G7).
- ☐ ●Explain why the selected metrics are suitable for the specific study (G7).

- ☐ ○Include an open LLM as a baseline when using commercial models and report inter-model agreement (G6).
- ☐ ○Summarize benchmark structure, task types, and limitations (G7).

Reproducibility, Ethics, and Resources

- ☐ ●Provide model outputs and discuss sensitive data handling (G8).
- ☐ ●Justify LLM usage in light of its resource demands (G8).
- ☐ ○Provide a full replication package with step-by-step instructions as part of the *supplementary material* (G6).
- ☐ ○Include a subset of the validation data for partial replication (G8).

Results

- ☐ ○Report established metrics to make study results comparable; additional metrics may be reported as appropriate (G7).
- ☐ ○Repeat experiments due to the inherent non-determinism of LLMs and report the result distribution using descriptive statistics (G7).
- ☐ ○Use traditional (non-LLM) baselines for comparison where possible (G7).

Limitations and Threats to Validity

- ☐ ●Acknowledge non-disclosed confidential or proprietary components as reproducibility limitations (G3).
- ☐ ●Transparently report study limitations, including the impact of non-determinism and generalizability constraints (G8).
- ☐ ●Specify whether generalization across LLMs or across time was assessed, and discuss model and version differences (G8).
- ☐ ●Describe measurement constructs and methods; disclose any data leakage risks and avoid leaking evaluation data into LLM improvement pipelines (G8).

- ☐ ○Employ and report mitigation strategies where applicable, such as replication packages, human validation, longitudinal re-runs, triangulation, and sensitivity analysis (G8).

7 Statements and Declarations

Competing Interests: The authors declare no competing interests. Multiple co-authors are members of the EMSE Editorial Board, which is disclosed here for transparency.

Authorship: Each author contributed or reviewed content and, most importantly, read the complete guidelines and confirmed that they stand behind all recommendations, not only their own contributions.

Funding: No funding was received for writing this article. However, the *2nd Copenhagen Symposium on Human-Centered Software Engineering AI*, which played a central role in forming the team of co-authors, was supported by the Carlsberg Foundation with grant CF24-0693 and the Alfred P. Sloan Foundation with grant G-2024-22586 to Daniel Russo.

Ethical Approval: No ethics approval was required for writing this article.

Data Availability: This article has no associated data. The guidelines are, however, hosted online and maintained on GitHub.

References

1. ACM Publications Board (2020) Artifact review and badging – current. `https://www.acm.org/publications/policies/artifact-review-and-badging-current`, accessed 2025-01-13
2. Agarwal R, Schwarzer M, Castro PS, Courville AC, Bellemare MG (2021) Deep reinforcement learning at the edge of the statistical precipice. In: Ranzato M, Beygelzimer A, Dauphin YN, Liang P, Vaughan JW (eds) Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual, pp 29304–29320, URL `https://proceedings.neurips.cc/paper/2021/hash/f514cec81cb148559cf475e7426eed5e-Abstract.html`
3. Ahmed N, Wahed M, Thompson NC (2023) The growing influence of industry in AI research. Science 379(6635):884–886, DOI 10.1126/science.ade2420
4. Ahmed T, Devanbu PT, Treude C, Pradel M (2025) Can llms replace manual annotation of software engineering artifacts? In: 22nd IEEE/ACM International Conference on Mining Software Repositories, MSR@ICSE 2025, Ottawa, ON, Canada, April 28-29, 2025, IEEE, pp 526–538, DOI 10.1109/MSR66628.2025.00086
5. Ahuja S, Gumma V, Sitaram S (2024) Contamination report for multilingual benchmarks. CoRR abs/2410.16186, DOI 10.48550/ARXIV.2410.16186, URL `https://doi.org/10.48550/arXiv.2410.16186`, `2410.16186`
6. Allamanis M (2019) The adverse effects of code duplication in machine learning models of code. In: Masuhara H, Petricek T (eds) Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software, Onward! 2019, ACM, pp 143–153, DOI 10.1145/3359591.3359735
7. Anandayuvaraj D, Campbell M, Tewari A, Davis JC (2024) FAIL: analyzing software failures from the news using llms. In: Filkov V, Ray B, Zhou M (eds) Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE 2024, Sacramento,

CA, USA, October 27 - November 1, 2024, ACM, pp 506–518, DOI 10.1145/3691620.3695022

8. Angermeir F, Amougou M, Kreitz M, Bauer A, Linhuber M, Fucci D, Constante FM, Méndez D, Gorschek T (2025) Reflections on the reproducibility of commercial LLM performance in empirical software engineering studies. CoRR abs/2510.25506, DOI 10.48550/ARXIV.2510.25506, URL https://doi.org/10.48550/arXiv.2510.25506, 2510.25506
9. Argyle LP, Busby EC, Fulda N, Gubler J, Rytting CM, Wingate D (2022) Out of one, many: Using language models to simulate human samples. CoRR abs/2209.06899, DOI 10.48550/ARXIV.2209.06899, URL https://doi.org/10.48550/arXiv.2209.06899, 2209.06899
10. Association AP (2018) Apa dictionary of psychology: subject. https://dictionary.apa.org/subject, accessed 2025-08-15
11. Association for Computing Machinery (2023) ACM Policy on Authorship. https://www.acm.org/publications/policies/new-acm-policy-on-authorship, accessed 2025-01-13
12. Austin J, Odena A, Nye MI, Bosma M, Michalewski H, Dohan D, Jiang E, Cai CJ, Terry M, Le QV, Sutton C (2021) Program synthesis with large language models. CoRR abs/2108.07732, URL https://arxiv.org/abs/2108.07732, 2108.07732
13. Azanza M, Pereira J, Irastorza A, Galdos A (2024) Can llms facilitate onboarding software developers? an ongoing industrial case study. In: 36th International Conference on Software Engineering Education and Training, CSEE&T 2024, IEEE, pp 1–6, DOI 10.1109/CSEET62301.2024.10662989
14. Balloccu S, Schmidtová P, Lango M, Dusek O (2024) Leak, cheat, repeat: Data contamination and evaluation malpractices in closed-source llms. In: Graham Y, Purver M (eds) Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics, EACL 2024 - Volume 1: Long Papers, St. Julian's, Malta, March 17-22, 2024, Association for Computational Linguistics, pp 67–93, URL https://aclanthology.org/2024.eacl-long.5
15. Bano M, Zowghi D, Whittle J (2023) Exploring qualitative research using llms. CoRR abs/2306.13298, DOI 10.48550/ARXIV.2306.13298, URL https://doi.org/10.48550/arXiv.2306.13298, 2306.13298
16. Bano M, Hoda R, Zowghi D, Treude C (2024) Large language models for qualitative research in software engineering: exploring opportunities and challenges. Autom Softw Eng 31(1):8, DOI 10.1007/S10515-023-00407-8, URL https://doi.org/10.1007/s10515-023-00407-8
17. Bano M, Gunatilake H, Hoda R (2025) What does a software engineer look like? exploring societal stereotypes in llms. In: 47th IEEE/ACM International Conference on Software Engineering: Software Engineering in Society, ICSE-SEIS 2025, Ottawa, ON, Canada, April 27 - May 3, 2025, IEEE, pp 173–184, DOI 10.1109/ICSE-SEIS66351.2025.00023

18. Barros CF, Azevedo BB, Neto VVG, Kassab M, Kalinowski M, do Nascimento HAD, Bandeira MCGSP (2025) Large language model for qualitative research: A systematic mapping study. In: IEEE/ACM International Workshop on Methodological Issues with Empirical Studies in Software Engineering, WSESE@ICSE 2025, Ottawa, ON, Canada, May 3, 2025, IEEE, pp 48–55, DOI 10.1109/WSESE66602.2025.00015
19. Bavaresco A, Bernardi R, Bertolazzi L, Elliott D, Fernández R, Gatt A, Ghaleb E, Giulianelli M, Hanna M, Koller A, Martins AFT, Mondorf P, Neplenbroek V, Pezzelle S, Plank B, Schlangen D, Suglia A, Surikuchi AK, Takmaz E, Testoni A (2024) Llms instead of human judges? A large scale empirical study across 20 NLP evaluation tasks. CoRR abs/2406.18403, DOI 10.48550/ARXIV.2406.18403, URL `https://doi.org/10.48550/arXiv.2406.18403`, `2406.18403`
20. Begg C, Cho M, Eastwood S, Horton R, Moher D, Olkin I, Pitkin R, Rennie D, Schulz KF, Simel D, Stroup DF (1996) Improving the quality of reporting of randomized controlled trials. The CONSORT statement. JAMA 276(8):637–639, DOI 10.1001/jama.276.8.637
21. Bjarnason BH, Silva A, Monperrus M (2026) On randomness in agentic evals. URL `https://arxiv.org/abs/2602.07150`, `2602.07150`
22. Blackwell RE, Barry J, Cohn AG (2024) Towards reproducible llm evaluation: Quantifying uncertainty in llm benchmark scores. URL `https://arxiv.org/abs/2410.03492`, `2410.03492`
23. Borsboom D (2005) Measuring the Mind: Conceptual Issues in Contemporary Psychometrics. Cambridge University Press, DOI 10.1017/CBO9780511490026
24. Byun C, Vasicek P, Seppi KD (2023) Dispensing with humans in human-computer interaction research. In: Schmidt A, Väänänen K, Goyal T, Kristensson PO, Peters A (eds) Extended Abstracts of the 2023 CHI Conference on Human Factors in Computing Systems, CHI EA 2023, ACM, pp 413:1–413:26, DOI 10.1145/3544549.3582749
25. Cai Y, Yadavally A, Mishra A, Montejo G, Nguyen TN (2024) Programming assistant for exception handling with codebert. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14-20, 2024, ACM, pp 94:1–94:13, DOI 10.1145/3597503.3639188
26. Cao J, Chan YK, Ling Z, Wang W, Li S, Liu M, Qiao R, Han Y, Wang C, Yu B, He P, Wang S, Zheng Z, Lyu MR, Cheung SC (2025) How should we build a benchmark? revisiting 274 code-related benchmarks for LLMs. CoRR abs/2501.10711, `2501.10711`
27. Chandra S (2025) Benchmarks for AI in Software Engineering (BLOG@CACM). `https://cacm.acm.org/blogcacm/benchmarks-for-ai-in-software-engineering/`, accessed 2025-08-24
28. Chann S (2023) Non-determinism in GPT-4 is caused by Sparse MoE. `https://152334h.github.io/blog/non-determinism-in-gpt-4/`, accessed 2025-01-13

29. Chen L, Zaharia M, Zou J (2024) How is chatgpt's behavior changing over time? Harvard Data Science Review 6(2), DOI 10.1162/99608f92.5317da47
30. Chen M, Tworek J, Jun H, Yuan Q, de Oliveira Pinto HP, Kaplan J, Edwards H, Burda Y, Joseph N, Brockman G, Ray A, Puri R, Krueger G, Petrov M, Khlaaf H, Sastry G, Mishkin P, Chan B, Gray S, Ryder N, Pavlov M, Power A, Kaiser L, Bavarian M, Winter C, Tillet P, Such FP, Cummings D, Plappert M, Chantzis F, Barnes E, Herbert-Voss A, Guss WH, Nichol A, Paino A, Tezak N, Tang J, Babuschkin I, Balaji S, Jain S, Saunders W, Hesse C, Carr AN, Leike J, Achiam J, Misra V, Morikawa E, Radford A, Knight M, Brundage M, Murati M, Mayer K, Welinder P, McGrew B, Amodei D, McCandlish S, Sutskever I, Zaremba W (2021) Evaluating large language models trained on code. CoRR abs/2107.03374, URL `https://arxiv.org/abs/2107.03374`, `2107.03374`
31. Chen Y, Ding Z, Alowain L, Chen X, Wagner DA (2023) Diversevul: A new vulnerable source code dataset for deep learning based vulnerability detection. In: Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses, RAID 2023, Hong Kong, China, October 16-18, 2023, ACM, pp 654–668, DOI 10.1145/3607199.3607242
32. Choi HK, Khanov M, Wei H, Li Y (2025) How contaminated is your benchmark? quantifying dataset leakage in large language models with kernel divergence. CoRR abs/2502.00678, DOI 10.48550/ARXIV.2502.00678, URL `https://doi.org/10.48550/arXiv.2502.00678`, `2502.00678`
33. Cohen J (1960) A coefficient of agreement for nominal scales. Educational and Psychological Measurement 20(1):37–46, DOI 10.1177/001316446002000104
34. Cohen J (1992) A power primer. Psychological Bulletin 112(1):155–159, DOI 10.1037/0033-2909.112.1.155
35. Coignion T, Quinton C, Rouvoy R (2024) A performance study of llm-generated code on leetcode. In: Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering, EASE 2024, Salerno, Italy, June 18-21, 2024, ACM, pp 79–89, DOI 10.1145/3661167.3661221
36. Computer T (2023) Redpajama: an open dataset for training large language models. `https://github.com/togethercomputer/RedPajama-Data`
37. Crowell R (2023) Why AI's diversity crisis matters, and how to tackle it. Nature Career Feature DOI 10.1038/d41586-023-01689-4, URL `https://doi.org/10.1038/d41586-023-01689-4`
38. De Paoli S (2024) Performing an inductive thematic analysis of semi-structured interviews with a large language model: An exploration and provocation on the limits of the approach. Social Science Computer Review 42(4):997–1019

39. Dhar R, Vaidhyanathan K, Varma V (2024) Can llms generate architectural design decisions? - an exploratory empirical study. In: 21st IEEE International Conference on Software Architecture, ICSA 2024, Hyderabad, India, June 4-8, 2024, IEEE, pp 79–89, DOI 10.1109/ICSA59870.2024.00016
40. Du X, Liu M, Wang K, Wang H, Liu J, Chen Y, Feng J, Sha C, Peng X, Lou Y (2023) Classeval: A manually-crafted benchmark for evaluating llms on class-level code generation. CoRR abs/2308.01861, DOI 10.48550/ARXIV.2308.01861, URL https://doi.org/10.48550/arXiv.2308.01861, 2308.01861
41. Dunn T (2023) It's time to collect data on how you build software. https://blog.continue.dev/its-time-to-collect-data-on-how-you-build-software/, accessed 2025-08-15
42. Dybå T, Kampenes VB, Sjøberg DIK (2006) A systematic review of statistical power in software engineering experiments. Inf Softw Technol 48(8):745–755, DOI 10.1016/J.INFSOF.2005.08.009, URL https://doi.org/10.1016/j.infsof.2005.08.009
43. Eghbali A, Pradel M (2022) Crystalbleu: Precisely and efficiently measuring the similarity of code. In: 37th IEEE/ACM International Conference on Automated Software Engineering, ASE 2022, ACM, pp 28:1–28:12, DOI 10.1145/3551349.3556903
44. El-Hajjami A, Salinesi C (2025) How good are synthetic requirements? evaluating llm-generated datasets for AI4RE. CoRR abs/2506.21138, DOI 10.48550/ARXIV.2506.21138, URL https://doi.org/10.48550/arXiv.2506.21138, 2506.21138
45. Elangovan A, Ko J, Xu L, Elyasi M, Liu L, Bodapati S, Roth D (2024) Beyond correlation: The impact of human uncertainty in measuring the effectiveness of automatic evaluation and llm-as-a-judge. CoRR abs/2410.03775, DOI 10.48550/ARXIV.2410.03775, URL https://doi.org/10.48550/arXiv.2410.03775, 2410.03775
46. Fu Z, Chen F, Zhou S, Li H, Jiang L (2024) LLMCO2: advancing accurate carbon footprint prediction for LLM inferences. CoRR abs/2410.02950, DOI 10.48550/ARXIV.2410.02950, URL https://doi.org/10.48550/arXiv.2410.02950, 2410.02950
47. Gallegos IO, Rossi RA, Barrow J, Tanjim MM, Kim S, Dernoncourt F, Yu T, Zhang R, Ahmed NK (2023) Bias and fairness in large language models: A survey. CoRR abs/2309.00770, DOI 10.48550/ARXIV.2309.00770, URL https://doi.org/10.48550/arXiv.2309.00770, 2309.00770
48. Gallegos IO, Rossi RA, Barrow J, Tanjim MM, Kim S, Dernoncourt F, Yu T, Zhang R, Ahmed N (2024) Bias and fairness in large language models: A survey. Computational Linguistics 50:1097–1179, DOI 10.1162/coli_a_00524
49. Gallifant J, Afshar M, Ameen S, Aphinyanaphongs Y, Chen S, Cacciamani G, Demner-Fushman D, Dligach D, Daneshjou R, Fernandes C, Hansen LH, Landman A, Lehmann L, McCoy LG, Miller T, Moreno

A, Munch N, Restrepo D, Savova G, Umeton R, Gichoya JW, Collins GS, Moons KGM, Celi LA, Bitterman DS (2025) The tripod-llm reporting guideline for studies using large language models. Nature Medicine 31(1):60–69, DOI 10.1038/s41591-024-03425-5

50. Gerosa MA, Trinkenreich B, Steinmacher I, Sarma A (2024) Can AI serve as a substitute for human subjects in software engineering research? Autom Softw Eng 31(1):13, DOI 10.1007/S10515-023-00409-6, URL https://doi.org/10.1007/s10515-023-00409-6
51. Gibney E (2024) Not all 'open source' AI models are actually open. Nature News DOI 10.1038/d41586-024-02012-5
52. Gilardi F, Alizadeh M, Kubli M (2023) ChatGPT outperforms crowd workers for text-annotation tasks. Proceedings of the National Academy of Sciences 120(30):e2305016120, DOI 10.1073/pnas.2305016120
53. Gonçalves J, Silva M, Cabral B, Dias T, Maia E, Praça I, Severino R, Ferreira LL (2025) Evaluating llama 3.2 for software vulnerability detection. In: Praça I, Bernardi S, Inácio PRM (eds) Cybersecurity - 9th European Interdisciplinary Cybersecurity Conference, EICC 2025, Rennes, France, June 18-19, 2025, Proceedings, Springer, Communications in Computer and Information Science, vol 2500, pp 38–51, DOI 10.1007/978-3-031-94855-8_3
54. Graziotin D (2024) acmsigsoft/open-science-policies: v1.0.0. DOI 10.5281/zenodo.10796477
55. Guba EG (1981) Criteria for assessing the trustworthiness of naturalistic inquiries. ECTJ 29(2):75–91, DOI 10.1007/BF02766777
56. Gundersen OE, Cappelen O, Mølnå M, Nilsen NG (2024) The unreasonable effectiveness of open science in AI: A replication study. CoRR abs/2412.17859, DOI 10.48550/ARXIV.2412.17859, URL https://doi.org/10.48550/arXiv.2412.17859, 2412.17859
57. Guo D, Zhu Q, Yang D, Xie Z, Dong K, Zhang W, Chen G, Bi X, Wu Y, Li YK, Luo F, Xiong Y, Liang W (2024) Deepseek-coder: When the large language model meets programming - the rise of code intelligence. CoRR abs/2401.14196, DOI 10.48550/ARXIV.2401.14196, URL https://doi.org/10.48550/arXiv.2401.14196, 2401.14196
58. Harding J, D'Alessandro W, Laskowski NG, Long R (2024) AI language models cannot replace human research participants. AI Soc 39(5):2603–2605, DOI 10.1007/S00146-023-01725-X, URL https://doi.org/10.1007/s00146-023-01725-x
59. He J, Shi J, Zhuo TY, Treude C, Sun J, Xing Z, Du X, Lo D (2025) Llm-as-a-judge for software engineering: Literature review, vision, and the road ahead. CoRR abs/2510.24367, DOI 10.48550/ARXIV.2510.24367, URL https://doi.org/10.48550/arXiv.2510.24367, 2510.24367
60. He Z, Huang C, Ding CC, Rohatgi S, Huang TK (2024) If in a crowdsourced data annotation pipeline, a GPT-4. In: Mueller FF, Kyburz P, Williamson JR, Sas C, Wilson ML, Dugas POT, Shklovski I (eds) Proceedings of the CHI Conference on Human Factors in Computing Systems, CHI 2024, ACM, pp 1040:1–1040:25, DOI 10.1145/3613904.

3642834
61. Hicks CM, Lee CS, Foster-Marks K (2025) The new developer: AI skill threat, identity change & developer thriving in the transition to AI-assisted software development. PsyArXiv DOI 10.31234/osf.io/2gej5_v2, URL https://osf.io/preprints/psyarxiv/2gej5_v2
62. Holtzman A, Buys J, Du L, Forbes M, Choi Y (2020) The curious case of neural text degeneration. In: 8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020, OpenReview.net, URL https://openreview.net/forum?id=rygGQyrFvH
63. Hou X, Zhao Y, Liu Y, Yang Z, Wang K, Li L, Luo X, Lo D, Grundy J, Wang H (2024) Large language models for software engineering: A systematic literature review. ACM Trans Softw Eng Methodol 33(8):1–79, DOI 10.1145/3695988
64. Hu X, Niu F, Chen J, Zhou X, Zhang J, He J, Xia X, Lo D (2025) Assessing and advancing benchmarks for evaluating large language models in software engineering tasks. CoRR abs/2505.08903, DOI 10.48550/ARXIV.2505.08903, URL https://doi.org/10.48550/arXiv.2505.08903, 2505.08903
65. Huang F, Kwak H, An J (2023) Is chatgpt better than human annotators? potential and limitations of chatgpt in explaining implicit hate speech. In: Ding Y, Tang J, Sequeda JF, Aroyo L, Castillo C, Houben G (eds) Companion Proceedings of the ACM Web Conference 2023, WWW 2023, ACM, pp 294–297, DOI 10.1145/3543873.3587368
66. Huang J, Yi B, Sun W, Wan B, Xu Y, Feng Y, Ye W, Qin Q (2024) Enhancing review classification via llm-based data annotation and multi-perspective feature representation learning. SSRN Electronic Journal pp 1–15, DOI 10.2139/ssrn.5002351, URL https://ssrn.com/abstract=5002351
67. Hui B, Yang J, Cui Z, Yang J, Liu D, Zhang L, Liu T, Zhang J, Yu B, Dang K, Yang A, Men R, Huang F, Ren X, Ren X, Zhou J, Lin J (2024) Qwen2.5-coder technical report. CoRR abs/2409.12186, DOI 10.48550/ARXIV.2409.12186, URL https://doi.org/10.48550/arXiv.2409.12186, 2409.12186
68. Husain H, Wu H, Gazit T, Allamanis M, Brockschmidt M (2019) Codesearchnet challenge: Evaluating the state of semantic code search. CoRR abs/1909.09436, URL http://arxiv.org/abs/1909.09436, 1909.09436
69. Hymel C, Johnson H (2025) Analysis of llms vs human experts in requirements engineering. CoRR abs/2501.19297, DOI 10.48550/ARXIV.2501.19297, URL https://doi.org/10.48550/arXiv.2501.19297, 2501.19297
70. Jahic J, Sami A (2024) State of practice: Llms in software engineering and software architecture. In: 21st IEEE International Conference on Software Architecture, ICSA 2024 - Companion, Hyderabad, India, June 4-8, 2024, IEEE, pp 311–318, DOI 10.1109/ICSA-C63560.2024.00059

71. Jain N, Han K, Gu A, Li W, Yan F, Zhang T, Wang S, Solar-Lezama A, Sen K, Stoica I (2024) Livecodebench: Holistic and contamination free evaluation of large language models for code. CoRR abs/2403.07974, DOI 10.48550/ARXIV.2403.07974, URL https://doi.org/10.48550/arXiv.2403.07974, 2403.07974
72. Jesse K, Ahmed T, Devanbu PT, Morgan E (2023) Large language models and simple, stupid bugs. In: 20th IEEE/ACM International Conference on Mining Software Repositories, MSR 2023, IEEE, pp 563–575, DOI 10.1109/MSR59073.2023.00082
73. Jiang P, Sonne C, Li W, You F, You S (2024) Preventing the immense increase in the life-cycle energy and carbon footprints of llm-powered intelligent chatbots. Engineering 40:202–210, DOI 10.1016/j.eng.2024.04.002, URL https://www.sciencedirect.com/science/article/pii/S2095809924002315
74. Jimenez CE, Yang J, Wettig A, Yao S, Pei K, Press O, Narasimhan KR (2024) Swe-bench: Can language models resolve real-world github issues? In: The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024, OpenReview.net, URL https://openreview.net/forum?id=VTF8yNQM66
75. Jowsey T, Braun V, Clarke V, Clarke V, Lupton D, Fine M (2025) We reject the use of generative artificial intelligence for reflexive qualitative research. SSRN DOI 10.2139/ssrn.5676462, URL https://ssrn.com/abstract=5676462
76. Kang S, Yoon J, Yoo S (2023) Large language models are few-shot testers: Exploring llm-based general bug reproduction. In: 45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, IEEE, pp 2312–2323, DOI 10.1109/ICSE48619.2023.00194
77. Karmakar A, Allamanis M, Robbes R (2023) JEMMA: an extensible java dataset for ml4code applications. Empir Softw Eng 28(2):54, DOI 10.1007/S10664-022-10275-7, URL https://doi.org/10.1007/s10664-022-10275-7
78. Khojah R, Mohamad M, Leitner P, de Oliveira Neto FG (2024) Beyond code generation: An observational study of chatgpt usage in software engineering practice. Proc ACM Softw Eng 1(FSE):1819–1840, DOI 10.1145/3660788
79. Khraisha Q, Put S, Kappenberg J, Warraitch A, Hadfield K (2024) Can large language models replace humans in systematic reviews? evaluating gpt-4's efficacy in screening and extracting data from peer-reviewed and grey literature in multiple languages. Research Synthesis Methods 15(4):616–626, DOI 10.1002/jrsm.1715, URL https://onlinelibrary.wiley.com/doi/abs/10.1002/jrsm.1715, https://onlinelibrary.wiley.com/doi/pdf/10.1002/jrsm.1715
80. v Kistowski J, Arnold JA, Huppler K, Lange KD, Henning JL, Cao P (2015) How to build a benchmark. In: Proceedings of the 6th ACM/SPEC International Conference on Performance Engineering, Association for Computing Machinery, New York, NY, USA, ICPE '15, p 333–336, DOI

10.1145/2668930.2688819
81. Kong A, Zhao S, Chen H, Li Q, Qin Y, Sun R, Zhou X (2023) Better zero-shot reasoning with role-play prompting. CoRR abs/2308.07702, DOI 10.48550/ARXIV.2308.07702, URL https://doi.org/10.48550/arXiv.2308.07702, 2308.07702
82. Krippendorff K (2018) Content Analysis: An Introduction to Its Methodology, 4th edn. SAGE Publications
83. Kulal S, Pasupat P, Chandra K, Lee M, Padon O, Aiken A, Liang P (2019) Spoc: Search-based pseudocode to code. CoRR abs/1906.04908, URL http://arxiv.org/abs/1906.04908, 1906.04908
84. Lachaux M, Rozière B, Chanussot L, Lample G (2020) Unsupervised translation of programming languages. CoRR abs/2006.03511, URL https://arxiv.org/abs/2006.03511, 2006.03511
85. Lakiotaki K, Vorniotakis N, Tsagris M, Georgakopoulos G, Tsamardinos I (2018) Biodataome: a collection of uniformly preprocessed and automatically annotated datasets for data-driven biology. Database J Biol Databases Curation 2018:bay011, DOI 10.1093/DATABASE/BAY011, URL https://doi.org/10.1093/database/bay011
86. Lee K, Ippolito D, Nystrom A, Zhang C, Eck D, Callison-Burch C, Carlini N (2022) Deduplicating training data makes language models better. In: Muresan S, Nakov P, Villavicencio A (eds) Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022, Dublin, Ireland, May 22-27, 2022, Association for Computational Linguistics, pp 8424–8445, DOI 10.18653/V1/2022.ACL-LONG.577, URL https://doi.org/10.18653/v1/2022.acl-long.577
87. Li D, Gupta K, Bhaduri M, Sathiadoss P, Bhatnagar S, Chong J (2024) Comparing gpt-3.5 and gpt-4 accuracy and drift in radiology diagnosis please cases. Radiology 310(1):e232411, DOI 10.1148/radiol.232411, https://doi.org/10.1148/radiol.232411
88. Li J, Li G, Zhang X, Zhao Y, Dong Y, Jin Z, Li B, Huang F, Li Y (2024) Evocodebench: An evolving code generation benchmark with domain-specific evaluations. In: Globersons A, Mackey L, Belgrave D, Fan A, Paquet U, Tomczak JM, Zhang C (eds) Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, URL http://papers.nips.cc/paper_files/paper/2024/hash/6a059625a6027aca18302803743abaa2-Abstract-Datasets_and_Benchmarks_Track.html
89. Li R, Allal LB, Zi Y, Muennighoff N, Kocetkov D, Mou C, Marone M, Akiki C, Li J, Chim J, Liu Q, Zheltonozhskii E, Zhuo TY, Wang T, Dehaene O, Davaadorj M, Lamy-Poirier J, Monteiro J, Shliazhko O, Gontier N, Meade N, Zebaze A, Yee M, Umapathi LK, Zhu J, Lipkin B, Oblokulov M, Wang Z, V RM, Stillerman J, Patel SS, Abulkhanov D, Zocca M, Dey M, Zhang Z, Moustafa-Fahmy N, Bhattacharyya U, Yu W, Singh S, Luccioni S, Villegas P, Kunakov M, Zhdanov F, Romero M, Lee T, Timor

N, Ding J, Schlesinger C, Schoelkopf H, Ebert J, Dao T, Mishra M, Gu A, Robinson J, Anderson CJ, Dolan-Gavitt B, Contractor D, Reddy S, Fried D, Bahdanau D, Jernite Y, Ferrandis CM, Hughes S, Wolf T, Guha A, von Werra L, de Vries H (2023) Starcoder: may the source be with you! CoRR abs/2305.06161, DOI 10.48550/ARXIV.2305.06161, URL https://doi.org/10.48550/arXiv.2305.06161, 2305.06161

90. Liang JT, Badea C, Bird C, DeLine R, Ford D, Forsgren N, Zimmermann T (2024) Can gpt-4 replicate empirical software engineering research? Proceedings of the ACM on Software Engineering 1(FSE):1330–1353
91. Liu J, Xia CS, Wang Y, Zhang L (2023) Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. In: Oh A, Naumann T, Globerson A, Saenko K, Hardt M, Levine S (eds) Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, URL http://papers.nips.cc/paper_files/paper/2023/hash/43e9d647ccd3e4b7b5baab53f0368686-Abstract-Conference.html
92. Lopes CV, Maj P, Martins P, Saini V, Yang D, Zitny J, Sajnani H, Vitek J (2017) Déjàvu: a map of code duplicates on github. Proc ACM Program Lang 1(OOPSLA):84:1–84:28, DOI 10.1145/3133908
93. López JAH, Chen B, Saad M, Sharma T, Varró D (2025) On inter-dataset code duplication and data leakage in large language models. IEEE Trans Software Eng 51(1):192–205, DOI 10.1109/TSE.2024.3504286
94. Lu Q, Zhu L, Xu X, Xing Z, Whittle J (2024) Toward responsible AI in the era of generative AI: A reference architecture for designing foundation model-based systems. IEEE Softw 41(6):91–100, DOI 10.1109/MS.2024.3406333
95. Lubos S, Felfernig A, Tran TNT, Garber D, Mansi ME, Erdeniz SP, Le V (2024) Leveraging llms for the quality assurance of software requirements. In: Liebel G, Hadar I, Spoletini P (eds) 32nd IEEE International Requirements Engineering Conference, RE 2024, Reykjavik, Iceland, June 24-28, 2024, IEEE, pp 389–397, DOI 10.1109/RE59067.2024.00046
96. Madampe K, Grundy J, Hoda R, Obie HO (2024) The struggle is real! the agony of recruiting participants for empirical software engineering studies. In: 2024 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC), Liverpool, UK, September 2-6, 2024, IEEE, pp 417–422, DOI 10.1109/VL/HCC60511.2024.00065
97. Mathis WS, Zhao S, Pratt N, Weleff J, De Paoli S (2024) Inductive thematic analysis of healthcare qualitative interviews using open-source large language models: How does it compare to traditional methods? Computer Methods and Programs in Biomedicine 255:108356
98. McDonald N, Schoenebeck S, Forte A (2019) Reliability and inter-rater reliability in qualitative research: Norms and guidelines for CSCW and HCI practice. Proc ACM Hum Comput Interact 3(CSCW):72:1–72:23, DOI 10.1145/3359174
99. Menzies T (2025) The case for compact AI. Commun ACM 68(9):6–7, DOI 10.1145/3746057

100. Microsoft (2023) Codebert on github. https://github.com/microsoft/CodeBERT, accessed 2025-08-15
101. Microsoft (2025) Azure OpenAI Service models. https://learn.microsoft.com/en-us/azure/ai-services/openai/concepts/models, accessed 2025-01-13
102. Mitu NE, Mitu GT (2024) The hidden cost of AI: Carbon footprint and mitigation strategies. Revista de Stiinte Politice Revue des Sciences Politiques 84:9–16, DOI 10.2139/ssrn.5036344
103. Mohamed S, Parvin A, Parra E (2024) Chatting with AI: deciphering developer conversations with chatgpt. In: Spinellis D, Bacchelli A, Constantinou E (eds) 21st IEEE/ACM International Conference on Mining Software Repositories, MSR 2024, Lisbon, Portugal, April 15-16, 2024, ACM, pp 187–191, DOI 10.1145/3643991.3645078
104. Mohsenimofidi S, Galster M, Treude C, Baltes S (2026) Context engineering for AI agents in open-source software. In: Proceedings of the 23rd IEEE/ACM International Conference on Mining Software Repositories (MSR 2026), URL https://arxiv.org/abs/2510.21413, 2510.21413
105. Montes CM, Feldt R, Martos CM, Ouhbi S, Premanandan S, Graziotin D (2025) Large language models in thematic analysis: Prompt engineering, evaluation, and guidelines for qualitative software engineering research. CoRR abs/2510.18456, DOI 10.48550/ARXIV.2510.18456, URL https://doi.org/10.48550/arXiv.2510.18456, 2510.18456
106. de Morais Leça M, Valença L, Santos R, de Souza Santos R (2025) Applications and implications of large language models in qualitative analysis: A new frontier for empirical software engineering. In: IEEE/ACM International Workshop on Methodological Issues with Empirical Studies in Software Engineering, WSESE@ICSE 2025, Ottawa, ON, Canada, May 3, 2025, IEEE, pp 36–43, DOI 10.1109/WSESE66602.2025.00013
107. Moumoula MB, Kaboré AK, Klein J, Bissyandé TF (2024) Large language models for cross-language code clone detection. CoRR abs/2408.04430, DOI 10.48550/ARXIV.2408.04430, URL https://doi.org/10.48550/arXiv.2408.04430, 2408.04430
108. Network Working Group (1997) RFC 2119. https://www.rfc-editor.org/rfc/rfc2119, accessed 2025-08-15
109. Network Working Group (2017) RFC 8174. https://www.rfc-editor.org/rfc/rfc8174, accessed 2025-08-15
110. Open Source Initiative (OSI) (2025) Open Source AI Definition 1.0. https://opensource.org/ai/open-source-ai-definition, accessed 2025-08-26
111. OpenAI (2023) How to make your completions outputs consistent with the new seed parameter. https://cookbook.openai.com/examples/reproducible_outputs_with_the_seed_parameter, accessed 2025-01-13
112. OpenAI (2025) OpenAI API Introduction. https://platform.openai.com/docs/api-reference/chat/streaming, accessed 2025-01-13

113. OpenCode Contributors (2025) OpenCode: The open source AI coding agent. https://opencode.ai/, accessed 2026-02-27
114. Ornelas T, Araújo AA, Araújo J, Araújo M, Trinkenreich B, Kalinowski M (2025) Llm-assisted thematic analysis: Opportunities, limitations, and recommendations. CoRR abs/2511.14528, DOI 10.48550/ARXIV.2511.14528, URL https://doi.org/10.48550/arXiv.2511.14528, 2511.14528
115. Pan Q, Ashktorab Z, Desmond M, Cooper MS, Johnson J, Nair R, Daly E, Geyer W (2024) Human-Centered Design Recommendations for LLM-as-a-Judge. In: ACL 2024 Workshop HuCLLM, arXiv, DOI 10.48550/arXiv.2407.03479, 2407.03479
116. Pangakis N, Wolken S, Fasching N (2023) Automated annotation with generative AI requires validation. CoRR abs/2306.00176, DOI 10.48550/ARXIV.2306.00176, URL https://doi.org/10.48550/arXiv.2306.00176, 2306.00176
117. Panickssery A, Bowman S, Feng S (2024) Llm evaluators recognize and favor their own generations. In: Globerson A, Mackey L, Belgrave D, Fan A, Paquet U, Tomczak J, Zhang C (eds) Advances in Neural Information Processing Systems, Curran Associates, Inc., vol 37, pp 68772–68802, URL https://proceedings.neurips.cc/paper_files/paper/2024/file/7f1f0218e45f5414c79c0679633e47bc-Paper-Conference.pdf
118. Papineni K, Roukos S, Ward T, Zhu W (2002) Bleu: a method for automatic evaluation of machine translation. In: Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics, ACL, pp 311–318, DOI 10.3115/1073083.1073135, URL https://aclanthology.org/P02-1040/
119. Peters U, Chin-Yee B (2025) Generalization bias in large language model summarization of scientific research. R Soc Open Sci 12(12241776), DOI 10.1098/rsos.241776
120. Pezeshkpour P, Hruschka E (2024) Large language models sensitivity to the order of options in multiple-choice questions. In: Duh K, Gómez-Adorno H, Bethard S (eds) Findings of the Association for Computational Linguistics: NAACL 2024, Mexico City, Mexico, June 16-21, 2024, Association for Computational Linguistics, Findings of ACL, vol NAACL 2024, pp 2006–2017, DOI 10.18653/V1/2024.FINDINGS-NAACL.130, URL https://doi.org/10.18653/v1/2024.findings-naacl.130
121. Rabbi MF, Champa AI, Zibran MF, Islam MR (2024) AI writes, we analyze: The chatgpt python code saga. In: Spinellis D, Bacchelli A, Constantinou E (eds) 21st IEEE/ACM International Conference on Mining Software Repositories, MSR 2024, Lisbon, Portugal, April 15-16, 2024, ACM, pp 177–181, DOI 10.1145/3643991.3645076
122. Ralph P, Tempero ED (2018) Construct validity in software engineering research and software metrics. In: Rainer A, MacDonell SG, Keung JW (eds) Proceedings of the 22nd International Conference on Evaluation and Assessment in Software Engineering, EASE2018, ACM, pp 13–23, DOI 10.1145/3210459.3210461

123. Ralph P, bin Ali N, Baltes S, Bianculli D, Diaz J, Dittrich Y, Ernst N, Felderer M, Feldt R, Filieri A, de França BBN, Furia CA, Gay G, Gold N, Graziotin D, He P, Hoda R, Juristo N, Kitchenham B, Lenarduzzi V, Martínez J, Melegati J, Mendez D, Menzies T, Molleri J, Pfahl D, Robbes R, Russo D, Saarimäki N, Sarro F, Taibi D, Siegmund J, Spinellis D, Staron M, Stol K, Storey MA, Taibi D, Tamburri D, Torchiano M, Treude C, Turhan B, Wang X, Vegas S (2021) Empirical standards for software engineering research. URL https://arxiv.org/abs/2010.03525, 2010.03525
124. Ralph P, Kuutila M, Arif H, Ayoola B (2024) Teaching software metrology: The science of measurement for software engineering. In: Méndez D, Avgeriou P, Kalinowski M, Ali NB (eds) Handbook on Teaching Empirical Software Engineering, Springer Nature Switzerland, pp 101–154, DOI 10.1007/978-3-031-71769-7_5
125. Rasheed Z, Waseem M, Ahmad A, Kemell K, Wang X, Nguyen-Duc A, Abrahamsson P (2024) Can large language models serve as data analysts? A multi-agent assisted approach for qualitative data analysis. CoRR abs/2402.01386, DOI 10.48550/ARXIV.2402.01386, URL https://doi.org/10.48550/arXiv.2402.01386, 2402.01386
126. Reiss MV (2023) Testing the reliability of chatgpt for text annotation and classification: A cautionary remark. CoRR abs/2304.11085, DOI 10.48550/ARXIV.2304.11085, URL https://doi.org/10.48550/arXiv.2304.11085, 2304.11085
127. Ren S, Guo D, Lu S, Zhou L, Liu S, Tang D, Sundaresan N, Zhou M, Blanco A, Ma S (2020) Codebleu: a method for automatic evaluation of code synthesis. CoRR abs/2009.10297, URL https://arxiv.org/abs/2009.10297, 2009.10297
128. Renze M (2024) The effect of sampling temperature on problem solving in large language models. In: Findings of the Association for Computational Linguistics: EMNLP 2024, pp 7346–7356, DOI 10.18653/v1/2024.findings-emnlp.432
129. Richards J, Wessel M (2024) What you need is what you get: Theory of mind for an llm-based code understanding assistant. In: IEEE International Conference on Software Maintenance and Evolution, ICSME 2024, IEEE, pp 666–671, DOI 10.1109/ICSME58944.2024.00070
130. Ronanki K, Berger C, Horkoff J (2023) Investigating chatgpt's potential to assist in requirements elicitation processes. In: 2023 49th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), IEEE, pp 354–361
131. Rozière B, Gehring J, Gloeckle F, Sootla S, Gat I, Tan XE, Adi Y, Liu J, Remez T, Rapin J, Kozhevnikov A, Evtimov I, Bitton J, Bhatt M, Canton-Ferrer C, Grattafiori A, Xiong W, Défossez A, Copet J, Azhar F, Touvron H, Martin L, Usunier N, Scialom T, Synnaeve G (2023) Code llama: Open foundation models for code. CoRR abs/2308.12950, DOI 10.48550/ARXIV.2308.12950, URL https://doi.org/10.48550/arXiv.2308.12950, 2308.12950

132. Russo D (2024) Navigating the complexity of generative ai adoption in software engineering. ACM Transactions on Software Engineering and Methodology 33(5):1–50
133. Sallou J, Durieux T, Panichella A (2024) Breaking the silence: the threats of using llms in software engineering. In: Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results, NIER@ICSE 2024, Lisbon, Portugal, April 14-20, 2024, ACM, pp 102–106, DOI 10.1145/3639476.3639764
134. Santos A, Vegas S, Oivo M, Juristo N (2021) A procedure and guidelines for analyzing groups of software engineering replications. IEEE Trans Software Eng 47(9):1742–1763, DOI 10.1109/TSE.2019.2935720
135. Schäfer M, Nadi S, Eghbali A, Tip F (2024) An empirical evaluation of using large language models for automated unit test generation. IEEE Trans Software Eng 50(1):85–105, DOI 10.1109/TSE.2023.3334955
136. Schilke O, Reimann M (2025) The transparency dilemma: How ai disclosure erodes trust. Organizational Behavior and Human Decision Processes 188:104405, DOI 10.1016/j.obhdp.2025.104405, URL https://www.sciencedirect.com/science/article/pii/S0749597825000172
137. Schneider K, Fotrousi F, Wohlrab R (2025) A reference model for empirically comparing llms with humans. In: 47th IEEE/ACM International Conference on Software Engineering: Software Engineering in Society, ICSE-SEIS 2025, Ottawa, ON, Canada, April 27 - May 3, 2025, IEEE, pp 130–134, DOI 10.1109/ICSE-SEIS66351.2025.00018
138. Schröder S, Morgenroth T, Kuhl U, Vaquet V, Paaßen B (2025) Large language models do not simulate human psychology. CoRR abs/2508.06950, DOI 10.48550/ARXIV.2508.06950, URL https://doi.org/10.48550/arXiv.2508.06950, 2508.06950
139. Schroeder K, Wood-Doughty Z (2024) Can you trust LLM judgments? reliability of llm-as-a-judge. CoRR abs/2412.12509, DOI 10.48550/ARXIV.2412.12509, URL https://doi.org/10.48550/arXiv.2412.12509, 2412.12509
140. Schulz KF, Altman DG, Moher D (2010) CONSORT 2010 statement: updated guidelines for reporting parallel group randomised trials. BMJ 340:c332, DOI 10.1136/bmj.c332
141. Schwartz R, Dodge J, Smith NA, Etzioni O (2020) Green AI. Communications of the ACM 63(12):54–63, DOI 10.1145/3381831
142. Sclar M, Choi Y, Tsvetkov Y, Suhr A (2024) Quantifying language models' sensitivity to spurious features in prompt design or: How I learned to start worrying about prompt formatting. In: The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024, OpenReview.net, URL https://openreview.net/forum?id=RIu5lyNXjT
143. Shull F, Singer J, Sjøberg DIK (eds) (2008) Guide to Advanced Empirical Software Engineering. Springer, DOI 10.1007/978-1-84800-044-5
144. Silva A, Monperrus M (2025) Repairbench: Leaderboard of frontier models for program repair. In: IEEE/ACM International Work-

shop on Large Language Models for Code, LLM4Code@ICSE 2025, Ottawa, ON, Canada, May 3, 2025, IEEE, pp 9–16, DOI 10.1109/LLM4CODE66737.2025.00006, URL https://doi.org/10.1109/LLM4Code66737.2025.00006

145. Siska C, Marazopoulou K, Ailem M, Bono J (2024) Examining the robustness of LLM evaluation to the distributional assumptions of benchmarks. In: Ku L, Martins A, Srikumar V (eds) Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024, Association for Computational Linguistics, pp 10406–10421, DOI 10.18653/V1/2024.ACL-LONG.560, URL https://doi.org/10.18653/v1/2024.acl-long.560
146. Sjøberg DIK, Bergersen GR (2023) Construct validity in software engineering. IEEE Trans Software Eng 49(3):1374–1396, DOI 10.1109/TSE.2022.3176725
147. Song Y, Wang G, Li S, Lin BY (2025) The good, the bad, and the greedy: Evaluation of llms should not ignore non-determinism. In: Chiruzzo L, Ritter A, Wang L (eds) Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL 2025 - Volume 1: Long Papers, Association for Computational Linguistics, pp 4195–4206, DOI 10.18653/V1/2025.NAACL-LONG.211, URL https://doi.org/10.18653/v1/2025.naacl-long.211
148. Sonnekalb T, Gruner B, Brust C, Mäder P (2022) Generalizability of code clone detection on codebert. In: 37th IEEE/ACM International Conference on Automated Software Engineering, ASE 2022, ACM, pp 143:1–143:3, DOI 10.1145/3551349.3561165
149. Stack Overflow (2025) 2025 developer survey. https://survey.stackoverflow.co/2025/, 49,009 respondents from 166 countries, May–June 2025
150. Staudinger M, Kusa W, Piroi F, Lipani A, Hanbury A (2024) A reproducibility and generalizability study of large language models for query generation. In: Sakai T, Ishita E, Ohshima H, Hasibi F, Mao J, Jose JM (eds) Proceedings of the 2024 Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region, SIGIR-AP 2024, Tokyo, Japan, December 9-12, 2024, ACM, pp 186–196, DOI 10.1145/3673791.3698432
151. Strubell E, Ganesh A, McCallum A (2019) Energy and policy considerations for deep learning in NLP. In: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, pp 3645–3650, DOI 10.18653/v1/P19-1355
152. Sullivan GM, Sargeant J (2011) Qualities of qualitative research: part I. J Grad Med Educ 3(4):449–452
153. Sumers TR, Yao S, Narasimhan K, Griffiths TL (2024) Cognitive architectures for language agents. Trans Mach Learn Res 2024, URL https://openreview.net/forum?id=1i6ZCvflQJ

154. Takerngsaksiri W, Pasuksmit J, Thongtanunam P, Tantithamthavorn C, Zhang R, Jiang F, Li J, Cook E, Chen K, Wu M (2025) Human-in-the-loop software development agents. In: 47th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice, SEIP@ICSE 2025, Ottawa, ON, Canada, April 27 - May 3, 2025, IEEE, pp 342–352, DOI 10.1109/ICSE-SEIP66354.2025.00036
155. Tam TYC, Sivarajkumar S, Kapoor S, Stolyar AV, Polanska K, McCarthy KR, Osterhoudt H, Wu X, Visweswaran S, Fu S, et al (2024) A framework for human evaluation of large language models in healthcare derived from literature review. NPJ digital medicine 7(1):258
156. Technology Innovation Institute (2023) falcon-refinedweb (revision 184df75). `https://huggingface.co/datasets/tiiuae/falcon-refinedweb`, DOI 10.57967/hf/0737
157. Tinnes C, Welter A, Apel S (2025) Software model evolution with large language models: Experiments on simulated, public, and industrial datasets. In: 47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, IEEE, pp 950–962, DOI 10.1109/ICSE55347.2025.00112
158. de Vries A (2023) The growing energy footprint of artificial intelligence. Joule 7(10):2191–2194
159. Wagner S, Barón MM, Falessi D, Baltes S (2025) Towards evaluation guidelines for empirical studies involving llms. In: IEEE/ACM International Workshop on Methodological Issues with Empirical Studies in Software Engineering, WSESE@ICSE 2025, May 3, 2025, IEEE, pp 24–27, DOI 10.1109/WSESE66602.2025.00011
160. Wan M, Safavi T, Jauhar SK, Kim Y, Counts S, Neville J, Suri S, Shah C, White RW, Yang L, Andersen R, Buscher G, Joshi D, Rangan N (2024) Tnt-llm: Text mining at scale with large language models. In: Baeza-Yates R, Bonchi F (eds) Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2024, ACM, pp 5836–5847, DOI 10.1145/3637528.3671647
161. Wang A, Morgenstern J, Dickerson JP (2025) Large language models that replace human participants can harmfully misportray and flatten identity groups. Nature Machine Intelligence 7:400–411, DOI 10.1038/s42256-025-00986-z
162. Wang C, Huang K, Zhang J, Feng Y, Zhang L, Liu Y, Peng X (2024) How and why llms use deprecated apis in code completion? an empirical study. CoRR abs/2406.09834, DOI 10.48550/ARXIV.2406.09834, URL `https://doi.org/10.48550/arXiv.2406.09834`, `2406.09834`
163. Wang F, Arora C, Liu Y, Huang K, Tantithamthavorn C, Aleti A, Sambathkumar D, Lo D (2025) Multi-modal requirements data-based acceptance criteria generation using llms. In: 40th IEEE/ACM International Conference on Automated Software Engineering, ASE 2025, Seoul, Korea, Republic of, November 16-20, 2025, IEEE, pp 3334–3345, DOI 10.1109/ASE63991.2025.00275

164. Wang S, Liu Y, Xu Y, Zhu C, Zeng M (2021) Want to reduce labeling cost? GPT-3 can help. In: Moens M, Huang X, Specia L, Yih SW (eds) Findings of the Association for Computational Linguistics: EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 16-20 November, 2021, Association for Computational Linguistics, Findings of ACL, vol EMNLP 2021, pp 4195–4205, DOI 10.18653/V1/2021.FINDINGS-EMNLP.354, URL https://doi.org/10.18653/v1/2021.findings-emnlp.354
165. Wang X, Kim H, Rahman S, Mitra K, Miao Z (2024) Human-llm collaborative annotation through effective verification of LLM labels. In: Mueller FF, Kyburz P, Williamson JR, Sas C, Wilson ML, Dugas POT, Shklovski I (eds) Proceedings of the CHI Conference on Human Factors in Computing Systems, CHI 2024, ACM, pp 303:1–303:21, DOI 10.1145/3613904.3641960
166. Widder DG, Whittaker M, West SM (2024) Why 'open'ai systems are actually closed, and why this matters. Nature 635(8040):827–833
167. Wiesinger J, Marlow P, Vuskovic V (2025) Agents. Google DeepMind URL https://gemini.google.com, contributors: Evan Huang, Emily Xue, Olcan Sercinoglu, Sebastian Riedel, Satinder Baveja, Antonio Gulli, Anant Nawalgaria
168. Wohlin C, Runeson P, Höst M, Ohlsson MC, Regnell B, Wesslén A (2024) Experimentation in Software Engineering, Second Edition. Springer, DOI 10.1007/978-3-662-69306-3
169. Wu C, Raghavendra R, Gupta U, Acun B, Ardalani N, Maeng K, Chang G, Behram FA, Huang J, Bai C, Gschwind M, Gupta A, Ott M, Melnikov A, Candido S, Brooks D, Chauhan G, Lee B, Lee HS, Akyildiz B, Balandat M, Spisak J, Jain R, Rabbat M, Hazelwood KM (2022) Sustainable AI: environmental implications, challenges and opportunities. In: Marculescu D, Chi Y, Wu C (eds) Proceedings of the Fifth Conference on Machine Learning and Systems, MLSys 2022, mlsys.org, URL https://proceedings.mlsys.org/paper_files/paper/2022/hash/462211f67c7d858f663355eff93b745e-Abstract.html
170. Xia Y, Shao H, Deng X (2024) Vulcobert: A codebert-based system for source code vulnerability detection. In: 2024 International Conference on Generative Artificial Intelligence and Information Security, GAIIS 2024, Kuala Lumpur, Malaysia, May 10-12, 2024, ACM, DOI 10.1145/3665348.3665391
171. Xiao T, Treude C, Hata H, Matsumoto K (2024) Devgpt: Studying developer-chatgpt conversations. In: Spinellis D, Bacchelli A, Constantinou E (eds) 21st IEEE/ACM International Conference on Mining Software Repositories, MSR 2024, Lisbon, Portugal, April 15-16, 2024, ACM, pp 227–230, DOI 10.1145/3643991.3648400
172. Xu C, Guan S, Greene D, Kechadi MT (2024) Benchmark data contamination of large language models: A survey. CoRR abs/2406.04244, DOI 10.48550/ARXIV.2406.04244, URL https://doi.org/10.48550/arXiv.2406.04244, 2406.04244

173. Xu R, Sun Y, Ren M, Guo S, Pan R, Lin H, Sun L, Han X (2024) AI for social science and social science of AI: A survey. Inf Process Manag 61(2):103665, DOI 10.1016/J.IPM.2024.103665, URL https://doi.org/10.1016/j.ipm.2024.103665
174. Xue Y, Chen H, Bai GR, Tairas R, Huang Y (2024) Does chatgpt help with introductory programming?an experiment of students using chatgpt in CS1. In: Proceedings of the 46th International Conference on Software Engineering: Software Engineering Education and Training, SEET@ICSE 2024, Lisbon, Portugal, April 14-20, 2024, ACM, pp 331–341, DOI 10.1145/3639474.3640076, URL https://doi.org/10.1145/3639474.3640076
175. Yan L, Hwang A, Wu Z, Head A (2024) Ivie: Lightweight anchored explanations of just-generated code. In: Mueller FF, Kyburz P, Williamson JR, Sas C, Wilson ML, Dugas POT, Shklovski I (eds) Proceedings of the CHI Conference on Human Factors in Computing Systems, CHI 2024, ACM, pp 140:1–140:15, DOI 10.1145/3613904.3642239
176. Yang G, Zhou Y, Chen X, Zhang X, Han T, Chen T (2023) Exploitgen: Template-augmented exploit code generation based on codebert. J Syst Softw 197:111577, DOI 10.1016/J.JSS.2022.111577, URL https://doi.org/10.1016/j.jss.2022.111577
177. Yuan J, Li H, Ding X, Xie W, Li YJ, Zhao W, Wan K, Shi J, Hu X, Liu Z (2025) Understanding and mitigating numerical sources of nondeterminism in LLM inference. In: Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2025, NeurIPS 2025, URL https://openreview.net/forum?id=Q3qAsZAEZw, 2506.09501
178. Zhao C, Habule M, Zhang W (2025) Large language models (llms) as research subjects: Status, opportunities and challenges. New Ideas in Psychology 79:101167, DOI 10.1016/j.newideapsych.2025.101167, URL https://www.sciencedirect.com/science/article/pii/S0732118X25000236
179. Zhou R, Chen L, Yu K (2024) Is LLM a reliable reviewer? A comprehensive evaluation of LLM on automatic paper reviewing tasks. In: Calzolari N, Kan M, Hoste V, Lenci A, Sakti S, Xue N (eds) Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation, LREC/COLING 2024, 20-25 May, 2024, Torino, Italy, ELRA and ICCL, pp 9340–9351, URL https://aclanthology.org/2024.lrec-main.816
180. Zhou W, Jiang YE, Li L, Wu J, Wang T, Qiu S, Zhang J, Chen J, Wu R, Wang S, Zhu S, Chen J, Zhang W, Zhang N, Chen H, Cui P, Sachan M (2023) Agents: An open-source framework for autonomous language agents. CoRR abs/2309.07870, DOI 10.48550/ARXIV.2309.07870, URL https://doi.org/10.48550/arXiv.2309.07870, 2309.07870
181. Zhou X, Weyssow M, Widyasari R, Zhang T, He J, Lyu Y, Chang J, Zhang B, Huang D, Lo D (2025) Lessleak-bench: A first investigation of data leakage in llms across 83 software engineering benchmarks. CoRR

abs/2502.06215, DOI 10.48550/ARXIV.2502.06215, URL https://doi.org/10.48550/arXiv.2502.06215, 2502.06215

182. Zhu Y, Zhang P, ul Haq E, Hui P, Tyson G (2023) Can chatgpt reproduce human-generated labels? A study of social computing tasks. CoRR abs/2304.10145, DOI 10.48550/ARXIV.2304.10145, URL https://doi.org/10.48550/arXiv.2304.10145, 2304.10145

183. Zhuo TY, Vu MC, Chim J, Hu H, Yu W, Widyasari R, Yusuf INB, Zhan H, He J, Paul I, Brunner S, Gong C, Hoang T, Zebaze AR, Hong X, Li W, Kaddour J, Xu M, Zhang Z, Yadav P, Jain N, Gu A, Cheng Z, Liu J, Liu Q, Wang Z, Lo D, Hui B, Muennighoff N, Fried D, Du X, de Vries H, von Werra L (2024) Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. CoRR abs/2406.15877, DOI 10.48550/ARXIV.2406.15877, URL https://doi.org/10.48550/arXiv.2406.15877, 2406.15877